Std Code Library(Temp version1.0)

tingyx

Nanjing University of Science and Technology

October 19, 2024

Contents

二分图

Graph Coloring I(奇环判断染色)

```cpp
#include <bits/stdc++.h>
#define endl '\n'
using namespace std;
typedef long double db;
typedef long long ll;

const ll N = 3e5 + 10;
const ll mod = 998244353;
const ll inf32 = 0x3f3f3f3f;
const ll inf64 = 5e18;

vector<int> G[N];

bool col[N], err;
int st[N], pos[N], top;

void dfs(int x, int fa, int c){
    if (!err && pos[x] && pos[x] < top && col[x] != c){
        err = 1;
        cout << top - pos[x] + 1 << endl;
        for (int i = pos[x]; i <= top; ++i)
            cout << st[i] << " \n"[i == top];
    }
    if (err || pos[x]) return;
    st[++top] = x;
    pos[x] = top;
    col[x] = c;
    for (auto y : G[x]){
        if (y == fa) continue;
        dfs(y, x, c ^ 1);
    }
    --top;
}

void solve(){
    int n, m;
    cin >> n >> m;
    for (int i = 1; i <= m; ++i){
        int u, v;
        cin >> u >> v;
        G[u].push_back(v);
        G[v].push_back(u);
    }
    dfs(1, 0, 0);
    if (!err){
        cout << 0 << endl;
        for (int i = 1; i <= n; ++i) cout << col[i] << " \n"[i == n];
    }
}

signed main(){
    ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
    int t = 1;
    //cin >> t;
    while(t--) solve();
    return 0;
}
```

增广路算法 (矩阵游戏)

矩阵进行行列交换，能否把对角线都变成 1

```cpp
#include <bits/stdc++.h>
#define endl '\n'
using namespace std;
typedef long double db;
typedef long long ll;
```

```cpp

const ll N = 2e5 + 10;
const ll mod = 998244353;
const ll inf32 = 0x3f3f3f3f;
const ll inf64 = 5e18;

struct augment_path
{
    vector<vector<int>> g;
    vector<int> pa; // 匹配
    vector<int> pb;
    vector<int> vis; // 访问
    int n, m;        // 两个点集中的顶点数量
    int dfn;         // 时间戳记
    int res;         // 匹配数

    augment_path(int _n, int _m) : n(_n), m(_m)
    {
        assert(0 <= n && 0 <= m);
        pa = vector<int>(n, -1);
        pb = vector<int>(m, -1);
        vis = vector<int>(n);
        g.resize(n);
        res = 0;
        dfn = 0;
    }

    void add(int from, int to)
    {
        assert(0 <= from && from < n && 0 <= to && to < m);
        g[from].push_back(to);
    }

    bool dfs(int v)
    {
        vis[v] = dfn;
        for (int u : g[v])
        {
            if (pb[u] == -1)
            {
                pb[u] = v;
                pa[v] = u;
                return true;
            }
        }
        for (int u : g[v])
        {
            if (vis[pb[u]] != dfn && dfs(pb[u]))
            {
                pa[v] = u;
                pb[u] = v;
                return true;
            }
        }
        return false;
    }

    int solve()
    {
        while (true)
        {
            dfn++;
            int cnt = 0;
            for (int i = 0; i < n; i++)
            {
                if (pa[i] == -1 && dfs(i))
                {
                    cnt++;
                }
            }
            if (cnt == 0)
```

```
            {
                break;
            }
            res += cnt;
        }
        return res;
    }
};

void solve()
{
    int n;
    cin >> n;
    augment_path ap(n, n);
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j){
            int x;
            cin >> x;
            if (x) ap.g[i].push_back(j);
        }
    int res = ap.solve();
    if (res == n){
        cout << "Yes" << endl;
        return;
    }
    cout << "No" << endl;
}

signed main()
{
    ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
    int t = 1;
    cin >> t;
    while (t--)
        solve();
    return 0;
}
```

3. 宿舍的假期 (简略版 AG)

```
#include <bits/stdc++.h>
#define endl '\n'
using namespace std;
typedef long double db;
typedef long long ll;

const ll N = 2e5 + 10;
const ll mod = 998244353;
const ll inf32 = 0x3f3f3f3f;
const ll inf64 = 5e18;

struct augment_path
{
    vector<vector<int>> g;
    vector<int> pa; // 匹配
    vector<int> pb;
    vector<int> vis; // 访问
    int n, m;        // 两个点集中的顶点数量
    int dfn;         // 时间戳记
    int res;         // 匹配数

    augment_path(int _n, int _m) : n(_n), m(_m)
    {
        assert(0 <= n && 0 <= m);
        pa = vector<int>(n, -1);
        pb = vector<int>(m, -1);
        vis = vector<int>(n);
        g.resize(n);
        res = 0;
        dfn = 0;
    }

```

```
    void add(int from, int to)
    {
        assert(0 <= from && from < n && 0 <= to && to < m);
        g[from].push_back(to);
    }

    bool dfs(int v)
    {
        vis[v] = dfn;
        for (int u : g[v])
        {
            if (pb[u] == -1)
            {
                pb[u] = v;
                pa[v] = u;
                return true;
            }
        }
        for (int u : g[v])
        {
            if (vis[pb[u]] != dfn && dfs(pb[u]))
            {
                pa[v] = u;
                pb[u] = v;
                return true;
            }
        }
        return false;
    }

    int solve()
    {
        while (true)
        {
            dfn++;
            int cnt = 0;
            for (int i = 0; i < n; i++)
            {
                if (pa[i] == -1 && dfs(i))
                {
                    cnt++;
                }
            }
            if (cnt == 0)
            {
                break;
            }
            res += cnt;
        }
        return res;
    }
};

void solve()
{
    int n;
    cin >> n;
    augment_path ap(n, n);
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j){
            int x;
            cin >> x;
            if (x) ap.g[i].push_back(j);
        }
    int res = ap.solve();
    if (res == n){
        cout << "Yes" << endl;
        return;
    }
    cout << "No" << endl;
}
```

```cpp

signed main()
{
    ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
    int t = 1;
    cin >> t;
    while (t--)
        solve();
    return 0;
}
```

串串

本质不同子序列数量

```cpp
vector<int> dp(n + 1);
map<char, int> vis;
for (int i = pl + 1; i < pr; ++i){
        if(!vis.count(s[i])){
            vis[s[i]] = 1;
            dp[i] = (dp[i - 1] * 2 + 1) % P;
        }else{
            dp[i] = (((dp[i - 1] * 2) % P - dp[vis[s[i]] - 1] + P) % P + P) % P;
        }
        vis[s[i]] = i;
    }
```

子序列自动机

```cpp
#include <bits/stdc++.h>
#define endl '\n'
#define pll pair<i64, i64>
#define tll tuple<i64, i64, i64>
#define all(a) a.begin() + 1, a.end()
using namespace std;
using i64 = long long;
using db = long double;
const i64 N = 1e5 + 10;
const i64 mod = 998244353;
const i64 inf32 = 1e9;
const i64 inf64 = 5e18;

int typ, n, q, m;
int root[N];
struct PersistentTree{
    int ls[N << 5], rs[N << 5], nxt[N << 5], tot = 0;
    //单点修改
    inline void update(int & rt, int old, int l, int r, int p, int k){
        rt = ++tot;
        ls[rt] = ls[old], rs[rt] = rs[old], nxt[rt] = nxt[old];
        if (l == r){
            nxt[rt] = k;
            return;
        }
        int mid = l + r >> 1;
        if (p <= mid) update(ls[rt], ls[old], l, mid, p, k);
        else update(rs[rt], rs[old], mid + 1, r, p, k);
    }
    //单点查询
    inline int query(int rt, int l, int r, int p){
        if (l == r) return nxt[rt];
        int mid = l + r >> 1;
        if (p <= mid) return query(ls[rt], l, mid, p);
        else return query(rs[rt], mid + 1, r, p);
    }
    //建树
    inline void build(vector<int> a){
        int n = a.size() - 1;
        for (int i = n; i >= 1; --i){
            update(root[i], root[i + 1], 1, m, a[i], i + 1);
```

```
        }
    }
}T;

void solve(){
    cin >> typ >> n >> q >> m;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; ++i) cin >> a[i];
    T.build(a);
    while (q--){
        int k;
        cin >> k;
        bool ok = true;
        int rt = 1;
        while (k--){
            int p;
            cin >> p;
            int to = T.query(root[rt], 1, m, p);
            if (!to) ok = false;
            if (to) rt = to;
        }
        if (ok) cout << "Yes" << endl;
        else cout << "No" << endl;
    }
}

int main(){
    ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
    int t = 1;
    //cin >> t;
    while(t--) solve();
    return 0;
}
```

SAM

codeforces edu101 E

```
#include <bits/stdc++.h>
#define endl '\n'

using ll = long long;

using namespace std;

constexpr int N = 1e6 + 10;
constexpr int mod = 998244353;

struct SuffixAutomaton {
    struct State {
        int len, link;
        map<char, int> next;
    };

    vector<State> st;
    int last;

    SuffixAutomaton(const string& s) {
        st.reserve(2 * s.size());
        st.push_back({0, -1});
        last = 0;
        for (char c : s) {
            extend(c);
        }
    }

    void extend(char c) {
        int cur = st.size();
        st.push_back({st[last].len + 1});
        int p = last;
        while (p != -1 && !st[p].next.count(c)) {
```

```
            st[p].next[c] = cur;
            p = st[p].link;
        }
        if (p == -1) {
            st[cur].link = 0;
        } else {
            int q = st[p].next[c];
            if (st[p].len + 1 == st[q].len) {
                st[cur].link = q;
            } else {
                int clone = st.size();
                st.push_back({st[p].len + 1, st[q].link, st[q].next});
                while (p != -1 && st[p].next[c] == q) {
                    st[p].next[c] = clone;
                    p = st[p].link;
                }
                st[q].link = st[cur].link = clone;
            }
        }
        last = cur;
    }

    bool contains(const string& t) {
        int cur = 0;
        for (char c : t) {
            if (!st[cur].next.count(c)) {
                return false;
            }
            cur = st[cur].next[c];
        }
        return true;
    }
};


void solve(){
    int n, k;
    string s;
    cin >> n >> k >> s;
    if (n == k) {
        cout << "YES" << endl;
        bool ok = 0;
        for (int i = 0; i < n - 1; ++i) {
            if (s[i] == '0') {
                ok = 1;
            }
            s[i] = '0';
        }
        if (ok) s[n - 1] = '0';
        cout << s << endl;
        return;
    }
    int m = log2(n - k) + 1;
    SuffixAutomaton sam(s);
    string ans;
    ans.resize(k);
    auto dfs = [&](auto self, int cur) -> bool {
        if (cur == k) {
            return !sam.contains(ans);
        }
        ans[cur] = '1';
        if (self(self, cur + 1)) return 1;
        ans[cur] = '0';
        return self(self, cur + 1);
    };
    if (!dfs(dfs, 0)) {
        cout << "NO" << endl;
        return;
    }
    cout << "YES" << endl;
    for (auto &x : ans) {
```

```
        if (x == '0') x = '1';
        else x = '0';
    }
    cout << ans << endl;
}

signed main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    cin >> t;
    while(t--) solve();
    return 0;
}
```

Trie 板子

```
#include <bits/stdc++.h>
#define endl '\n'

using ll = long long;

constexpr int N = 1e5 + 10;
constexpr int mod = 998244353;

using namespace std;

int tr[N << 5][26], idx = 0;
bool vis[N << 5];

void insert(string s) {
    int p = 0;
    for (auto ch : s) {
        int c = ch - 'a';
        if (!tr[p][c]) tr[p][c] = ++idx;
        p = tr[p][c];
    }
    vis[p] = 1;
}

vector<int> G[26];
int in[26];

bool query(string s) {
    auto topo = [&]() -> bool {
        queue<int> q;
        int cnt = 0;
        for (int i = 0; i < 26; ++i) if (!in[i]) q.push(i);
        while (!q.empty()) {
            auto u = q.front();
            q.pop();
            cnt++;
            for (auto v : G[u]) {
                if (--in[v] == 0) q.push(v);
            }
        }
        return cnt == 26;
    };
    int p = 0;
    for (int i = 0; i < s.size(); ++i) {
        int c = s[i] - 'a';
        for (int j = 0; j < 26; ++j) {
            if (j == c || !tr[p][j]) continue;
            G[c].push_back(j);
            in[j]++;
        }
        p = tr[p][c];
        if (vis[p] && i != s.size() - 1) return false;
    }
    return topo();

```

```
}

void solve(){
    int n;
    cin >> n;
    vector<string> s(n + 1);
    for (int i = 1; i <= n; ++i) cin >> s[i], insert(s[i]);
    vector<string> ans;
    for (int i = 1; i <= n; ++i) {
        memset(in, 0, sizeof in);
        for (int j = 0; j < 26; ++j) G[j].clear();
        if (query(s[i])) ans.push_back(s[i]);
    }
    cout << ans.size() << endl;
    for (auto x : ans) {
        cout << x << endl;
    }
}

signed main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    //cin >> t;
    while(t--) solve();
    return 0;
}
```

异或最大值

```
int tr[maxm][2], idx, n;

void insert(int x){
    int p = 0;
    for (int i = 31; i >= 0; --i){
        int c = x >> i & 1;
        if (!tr[p][c]) tr[p][c] = ++idx;
        p = tr[p][c];
    }
}

int query(int x){
    int res = 0, p = 0;
    for (int i = 31; i >= 0; --i){
        int c = x >> i & 1;
        if (tr[p][c ^ 1]){
            p = tr[p][c ^ 1];
            res += 1 << i;
        }else
            p = tr[p][c];
    }
    return res;
}
```

Border Tree1

Tricks:

1. 每个前缀 i 的所有 Border 为 i 到根结点的链
2. 哪些前缀有长度为 x 的 Border：x 的子树
3. 求两个前缀的公共 Border 等价于求 LCA

本题求出现次数大于等于 k 的 border

```
#include <bits/stdc++.h>
#define endl '\n'

using ll = long long;

constexpr int N = 1e6 + 10;
```

```cpp
constexpr int mod = 998244353;

using namespace std;

vector<int> G[N];
int sz[N];

vector<int> prefix_function(string s){
    G[0].push_back(1);
    int n = (int) s.length();
    vector<int> pi(n);
    for(int i = 2; i < n; i++) {
        pi[i] = pi[i - 1];
        while(pi[i] && s[i] != s[pi[i] + 1])
            pi[i] = pi[pi[i]];
        pi[i] += (s[i] == s[pi[i] + 1]);
        G[pi[i]].push_back(i);
    }
    return pi;
}

void dfs(int u) {
    sz[u] = 1;
    for(auto v : G[u]) {
        dfs(v);
        sz[u] += sz[v];
    }
}

void solve(){
    int n, k;
    cin >> n >> k;
    string s;
    cin >> s;
    s = " " + s;
    auto nxt = prefix_function(s);
    dfs(0);
    int u = n;
    while (u && sz[u] < k) u = nxt[u];
    if (!u) cout << -1 << endl;
    else cout << s.substr(1, u) << endl;
}

signed main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    //cin >> t;
    while(t--) solve();
    return 0;
}
```

Border Tree2

本题可以动态在字符串末尾加一个字符

```cpp
#include <bits/stdc++.h>
#define endl '\n'

using ll = long long;

constexpr int N = 1e6 + 10;
constexpr int mod = 998244353;

using namespace std;

struct BIT {
    int tr[N];
    int lowbit(int x) {return x & -x;}
    void add(int p, int v) {
        for (; p < N; p += lowbit(p)) tr[p] += v;
```

```
    }
    int query(int p) {
        int res = 0;
        for (; p; p -= lowbit(p)) {
            res += tr[p];
        }
        return res;
    }
}tr;

vector<int> G[N];

vector<int> prefix_function(string s){
    G[0].push_back(1);
    int n = (int) s.length();
    vector<int> pi(n);
    for(int i = 2; i < n; i++) {
        pi[i] = pi[i - 1];
        while(pi[i] && s[i] != s[pi[i] + 1])
            pi[i] = pi[pi[i]];
        pi[i] += (s[i] == s[pi[i] + 1]);
        G[pi[i]].push_back(i);
    }
    return pi;
}

pair<int, int> qry[N];
int sz[N], fa[N][21], dfn[N], low[N], tot;

void dfs(int u) {
    dfn[u] = ++tot;
    for (auto v : G[u]) {
        fa[v][0] = u;
        for (int i = 1; i <= 20; ++i) fa[v][i] = fa[fa[v][i - 1]][i - 1];
        dfs(v);
    }
    low[u] = tot;
}

void solve(){
    int n, q;
    cin >> n >> q;
    string s;
    cin >> s; s = " " + s;
    for (int i = 1; i <= q; ++i) {
        auto &[x, y] = qry[i];
        cin >> x;
        if (x == 2) cin >> y;
        else {
            char ch;
            cin >> ch;
            y = ch;
            s += ch;
        }
    }
    auto nxt = prefix_function(s);
    dfs(0);
    for (int i = 1; i <= n; ++i) {
        tr.add(dfn[i], 1);
    }
    for (int i = 1; i <= q; ++i) {
        auto [x, y] = qry[i];
        if (x == 1) tr.add(dfn[++n], 1);
        else {
            int cur = n;
            for (int j = 20; j >= 0; --j) {
                int k = y;
                int p = fa[cur][j];
                if (tr.query(low[p]) - tr.query(dfn[p] - 1) < k) {
                    cur = p;
                }
```

```
            }
            int ans = -1;
            if (fa[cur][0]) ans = fa[cur][0];
            cout << ans << endl;
        }
    }
}

signed main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    //cin >> t;
    while(t--) solve();
    return 0;
}
```

Manacher 区间回文串数量查询

```
#include <bits/stdc++.h>
#define endl '\n'

using ll = long long;

constexpr int N = 4e5 + 10;
constexpr int mod = 998244353;
constexpr ll linf = 2e18;

using namespace std;

struct Manacher {
    int r[N], p[N], n;
    void clear() {
        memset(r, 0, sizeof r);
        memset(p, 0, sizeof p);
    }

    void calc(string s, int tn){
        n = tn;
        int mx(0), center;
        r[0] = -2;
        for (int i = 1; i <= n; i++)
            r[2 * i] = s[i];
        for (int i = 1; i <= n; i++)
            r[2 * i - 1] = -1;
        r[2 * n + 1] = -1;
        for (int i = 1; i <= 2 * n + 1; i++){
            if (mx >= i)
                p[i] = min(p[2 * center - i], mx - i + 1);
            else
                p[i] = 1;
            while (r[i - p[i]] == r[i + p[i]])
                p[i]++;
            if (i + p[i] - 1 > mx){
                mx = i + p[i] - 1;
                center = i;
            }
        }
    }
}mnc;

struct SegmentTree{
    ll mn[N << 2], mx[N << 2], sum[N << 2], add[N << 2], set[N << 2], L[N << 2], R[N << 2];
    void maketag_set(ll o, ll v){
        add[o] = 0;
        set[o] = v;
        mx[o] = mn[o] = v;
        sum[o] = (R[o] - L[o] + 1) * v;
    }
    void maketag_add(ll o, ll v){
        add[o] += v;
```

```
        mx[o] += v, mn[o] += v;
        sum[o] += (R[o] - L[o] + 1) * v;
    }
    void pushdown(ll o){
        if (L[o] == R[o])
            return;
        if (~set[o]){
            maketag_set(o << 1, set[o]);
            maketag_set(o << 1 | 1, set[o]);
            set[o] = -1;
        }
        if (add[o]){
            maketag_add(o << 1, add[o]);
            maketag_add(o << 1 | 1, add[o]);
            add[o] = 0;
        }
    }
    void pushup(ll o){
        mx[o] = max(mx[o << 1], mx[o << 1 | 1]);
        mn[o] = min(mn[o << 1], mn[o << 1 | 1]);
        sum[o] = sum[o << 1] + sum[o << 1 | 1];
    }
    void build(ll o, ll l, ll r, ll *array = NULL){
        ll mid((l + r) >> 1);
        L[o] = l, R[o] = r;
        add[o] = 0;
        set[o] = -1;
        if (l == r)
        {
            if (array)
                mn[o] = mx[o] = sum[o] = array[l];
            else
                mn[o] = mx[o] = sum[o] = 0;
            return;
        }
        build(o << 1, l, mid, array);
        build(o << 1 | 1, mid + 1, r, array);
        pushup(o);
    }
    void Set(ll o, ll l, ll r, ll v)
    {
        ll mid((L[o] + R[o]) >> 1);
        if (l <= L[o] and r >= R[o])
        {
            maketag_set(o, v);
            return;
        }
        pushdown(o);
        if (l <= mid)
            Set(o << 1, l, r, v);
        if (r > mid)
            Set(o << 1 | 1, l, r, v);
        pushup(o);
    }
    void Add(ll o, ll l, ll r, ll v)
    {
        ll mid((L[o] + R[o]) >> 1);
        if (l <= L[o] and r >= R[o])
        {
            maketag_add(o, v);
            return;
        }
        pushdown(o);
        if (l <= mid)
            Add(o << 1, l, r, v);
        if (r > mid)
            Add(o << 1 | 1, l, r, v);
        pushup(o);
    }
    ll Sum(ll o, ll l, ll r)
    {
```

```
        pushdown(o);
        ll mid((L[o] + R[o]) >> 1), ans(0);
        if (l <= L[o] and r >= R[o])
            return sum[o];
        if (l <= mid)
            ans += Sum(o << 1, l, r);
        if (r > mid)
            ans += Sum(o << 1 | 1, l, r);
        return ans;
    }
    ll Min(ll o, ll l, ll r)
    {
        ll mid((L[o] + R[o]) >> 1), ans(linf);
        if (l <= L[o] and r >= R[o])
            return mn[o];
        pushdown(o);
        if (l <= mid)
            ans = min(ans, Min(o << 1, l, r));
        if (r > mid)
            ans = min(ans, Min(o << 1 | 1, l, r));
        return ans;
    }
    ll Max(ll o, ll l, ll r)
    {
        ll mid((L[o] + R[o]) >> 1), ans(-linf);
        if (l <= L[o] and r >= R[o])
            return mx[o];
        pushdown(o);
        if (l <= mid)
            ans = max(ans, Max(o << 1, l, r));
        if (r > mid)
            ans = max(ans, Max(o << 1 | 1, l, r));
        return ans;
    }
} segtree;

void solve(){
    int n, q;
    cin >> n >> q;
    string s;
    cin >> s; s = " " + s;
    mnc.calc(s, n);
    vector<int> l(q + 1), r(q + 1), id(q + 1);
    // for (int i = 1; i <= 2 * n + 1; ++i) {
    //     cout << char(mnc.r[i]);
    // }
    // cout << endl;
    // for (int i = 1; i <= 2 * n + 1; ++i) {
    //     cout << mnc.p[i] << " \n"[i == 2 * n + 1];
    // }
    // cout << endl;
    for (int i = 1; i <= q; ++i) {
        cin >> l[i] >> r[i];
        l[i] = 2 * l[i] - 1;
        r[i] = 2 * r[i] + 1;
        id[i] = i;
    }
    sort(id.begin() + 1, id.end(), [&](ll a, ll b) {
        return l[a] + r[a] < l[b] + r[b];
    });
    segtree.build(1, 1, 2 * n + 1);
    vector<ll> ans(q + 1);
    for (int i = 1, j = 1; i <= q; ++i) {
        while (j <= ((l[id[i]] + r[id[i]]) >> 1)) {
            segtree.Add(1, j - mnc.p[j] + 1, j, 1);
            ++j;
        }
        ans[id[i]] += segtree.Sum(1, l[id[i]], 2 * n);
    }
    segtree.build(1, 1, 2 * n + 1);
    for (int i = q, j = 2 * n + 1; i >= 1; --i)  {
```

```
        while (j > ((l[id[i]] + r[id[i]]) >> 1)) {
            segtree.Add(1, j, j + mnc.p[j] - 1, 1);
            --j;
        }
        ans[id[i]] += segtree.Sum(1, 1, r[id[i]]);
    }
    for (int i = 1; i <= q; ++i) {
        ans[i] -= (r[i] + 1) / 2 - (l[i] / 2);
        cout << ans[i] / 2 << endl;
    }
}

signed main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    //cin >> t;
    while(t--) solve();
    return 0;
}
```

最长双回文串

```
#include <bits/stdc++.h>
#define endl '\n'

using ll = long long;

constexpr int N = 2e5 + 10;
constexpr int mod = 998244353;

using namespace std;

int L[N], R[N];

struct Manacher {
    int r[N], p[N], n;
    void clear() {
        memset(r, 0, sizeof r);
        memset(p, 0, sizeof p);
    }

    void calc(string s, int tn){
        n = tn;
        int mx(0), center;
        r[0] = -2;
        for (int i = 1; i <= n; i++)
            r[2 * i] = s[i];
        for (int i = 1; i <= n; i++)
            r[2 * i - 1] = -1;
        r[2 * n + 1] = -1;
        for (int i = 1; i <= 2 * n + 1; i++){
            if (mx >= i)
                p[i] = min(p[2 * center - i], mx - i + 1);
            else
                p[i] = 1;
            while (r[i - p[i]] == r[i + p[i]])
                p[i]++;
            if (i + p[i] - 1 > mx){
                mx = i + p[i] - 1;
                center = i;
            }
            int ll = i - p[i] + 1;
            int rr = i + p[i] - 1;
            R[rr] = max(R[rr], p[i] - 1); // 以 rr 为回文右端点的最长回文串
            L[ll] = max(L[ll], p[i] - 1); // 以 ll 为回文左端点的最长回文串
        }
    }
}mnc;

void solve(){
```

```
    string s;
    cin >> s;
    int n = s.length();
    s = " " + s;
    mnc.calc(s, n);
    for (int i = 2 * n - 1; i >= 1; i -= 2) {
        R[i] = max(R[i], R[i + 2] - 2);
    }
    for (int i = 3; i <= 2 * n + 1; i += 2) {
        L[i] = max(L[i], L[i - 2] - 2);
    }
    ll ans = 0;
    for (int i = 1; i <= 2 * n + 1; i += 2) if (L[i] && R[i]) ans = max(ans, 1ll * (L[i] + R[i]));
    cout << ans << endl;
}

signed main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    //cin >> t;
    while(t--) solve();
    return 0;
}
```

每个回文串有不同字符累和输出

```
#include <bits/stdc++.h>
#define endl '\n'

using ll = long long;

constexpr int N = 3e5 + 10;
constexpr int ALP = 26;
constexpr int mod = 998244353;

using namespace std;

struct PAM {
    int nxt[N][ALP]; // nxt 指针, 参照 Trie 树
    int fail[N]; // fail 失配后缀链接
    int cnt[N]; // 此回文串出现个数
    int num[N];
    int len[N]; // 回文串长度
    int s[N]; // 存放添加的字符
    int lst; //指向上一个字符所在的节点, 方便下一次 add
    int n; // 已添加字符个数
    int p; // 节点个数

    int newNode(int w) {
        for (int i = 0; i < ALP; ++i) {
            nxt[p][i] = 0;
        }
        cnt[p] = 0;
        num[p] = 0;
        len[p] = w;
        return p++;
    }

    void init() {
        p = 0;
        newNode(0);
        newNode(-1);
        lst = 0;
        n = 0;
        s[n] = -1;
        fail[0] = 1;
    }

    int getFail(int x) {
        while (s[n - (len[x] + 1)] != s[n]) x = fail[x];
```

```
        return x;
    }

    void add(int c) {
        c -= 'a';
        s[++n] = c;
        int cur = getFail(lst);
        if (!nxt[cur][c]) {
            int now = newNode(len[cur] + 2);
            fail[now] = nxt[getFail(fail[cur])][c];
            nxt[cur][c] = now;
            num[now] = num[fail[now]] + 1;
        }
        lst = nxt[cur][c];
        cnt[lst]++;
    }

    void count() {
        for (int i = p - 1; i >= 0; i--)
            cnt[fail[i]] += cnt[i];
    }


    ll dfs(int u, int vis) {
        int tot = 0;
        ll res = 0;
        for (int i = 0; i < ALP; ++i) if (vis >> i & 1) tot++;
        if (u >= 2) res += tot * cnt[u];
        for (int i = 0; i < ALP; ++i) {
            if (nxt[u][i]) res += dfs(nxt[u][i], vis | (1 << i));
        }
        return res;
    }

    ll getAns() {
        ll res = 0;
        res = dfs(0, 0);
        res += dfs(1, 0);
        return res;
    }
}pam;

void solve(){
    string s;
    cin >> s;
    int n = s.length();
    pam.init();
    for (int i = 0; i < n; ++i) pam.add(s[i]);
    pam.count();
    cout << pam.getAns() << endl;
}

signed main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    //cin >> t;
    while(t--) solve();
    return 0;
}
```

划分为若干个偶数回文串方案数

```
#include <bits/stdc++.h>
#define endl '\n'

using ll = long long;

constexpr int N = 1e6 + 10;
constexpr int mod = 1e9 + 7;
constexpr int ALP = 26;
```

```cpp

using namespace std;

/*
    1. 一个回文串 S 的后缀 T 如果是回文串等价于 T 是 S 的 border
    2. 对于若干个回文后缀，如果长度都大于 len/2，那么它们构成了一个等差数列
    3. 将一个串 S 的所有 border 按长度从小到大排序后，能形成 log 个等差数列
*/

ll f[N], g[N];

struct PAM {
    int nxt[N][ALP]; // nxt 指针，参照 Trie 树
    int fail[N]; // fail 失配后缀链接
    int cnt[N]; // 此回文串出现个数
    int num[N];
    int len[N]; // 回文串长度
    int s[N]; // 存放添加的字符
    int lst; //指向上一个字符所在的节点，方便下一次 add
    int n; // 已添加字符个数
    int p; // 节点个数
    int lft[N]; // 除去回文后缀后剩余的长度
    int top[N]; // 等差数列失配链接
    int id[N]; // 位于回文树的下标


    int newNode(int w) {
        for (int i = 0; i < ALP; ++i) {
            nxt[p][i] = 0;
        }
        cnt[p] = 0;
        num[p] = 0;
        len[p] = w;
        return p++;
    }

    void init() {
        p = 0;
        newNode(0);
        newNode(-1);
        lst = 0;
        n = 0;
        s[n] = -1;
        fail[0] = 1;
    }

    int getFail(int x) {
        while (s[n - (len[x] + 1)] != s[n]) x = fail[x];
        return x;
    }

    void add(int c) {
        c -= 'a';
        s[++n] = c;
        int cur = getFail(lst);
        if (!nxt[cur][c]) {
            int now = newNode(len[cur] + 2);
            fail[now] = nxt[getFail(fail[cur])][c];
            nxt[cur][c] = now;
            num[now] = num[fail[now]] + 1;
            lft[now] = len[now] - len[fail[now]];
            top[now] = lft[now] == lft[fail[now]] ? top[fail[now]] : fail[now];
        }
        id[n] = lst = nxt[cur][c];
        cnt[lst]++;
    }

    void count() {
        for (int i = p - 1; i >= 0; i--)
            cnt[fail[i]] += cnt[i];
    }
```

```
    // f[i] 记录 [1...i] 时的方案数, g[i] 记录以回文串 i 为最大长度的等差数列的当前 f 值和
    void solve() {
        for (int i = 1; i <= n; ++i) {
            int idx = id[i];
            int tot = 0;
            for (int j = idx; j; j = top[j]) {
                tot++;
                g[j] = f[i - len[top[j]] - lft[j]];
                if (lft[j] == lft[fail[j]]) g[j] = (g[j] + g[fail[j]]) % mod;
                if (i % 2 == 0) f[i] = (f[i] + g[j]) % mod;
            }
        }
    }
}pam;

void solve(){
    string s;
    cin >> s;
    int n = s.length();
    s = " " + s;
    string str = " ";
    for (int i = 1; i <= n / 2; ++i) {
        str.push_back(s[i]);
        str.push_back(s[n - i + 1]);
    }
    pam.init();
    for (int i = 1; i <= n; ++i) pam.add(str[i]);
    f[0] = 1;
    pam.solve();
    cout << f[n] << endl;
}

signed main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    //cin >> t;
    while(t--) solve();
    return 0;
}
```

CDQ 解决高维偏序

HDU2024 暑期多校 1007

$i < j$
$f[i] < f[j]$
$g[i] < g[j]$
满足上面三个条件则 i 向 j 建边，计算每个点的出度

```
#include <algorithm>
#include <bits/stdc++.h>
#include <vector>
#define endl '\n'
using namespace std;
typedef long double db;
typedef long long ll;

const ll N = 2e5 + 10;
const ll mod = 998244353;
const ll inf32 = 0x3f3f3f3f;
const ll inf64 = 5e18;

int tr[N];
int lowbit(int x) {return x & -x;}
void add(int p, int val){
    for (; p < N; p += lowbit(p)){
        tr[p] += val;
    }
```

```
}
int query(int p){
    int res = 0;
    for (; p; p -= lowbit(p)){
        res += tr[p];
    }
    return res;
}

int query(int p1, int p2){
    return query(p2) - query(p1 - 1);
}

struct Query{
    int f, g, idx;
}node[N];

ll f_[N], g_[N];
ll n;
ll sum[N];

Query tmp[N];

void cdq1(int l, int r){
    int mid = (l + r) >> 1;
    if (l == r) {
        node[l].f = n - f_[l] + 1;
        node[l].g = n - g_[l] + 1;
        node[l].idx = l;
        return;
    }
    cdq1(l, mid);
    cdq1(mid + 1, r);
    int cnt = l, f1 = l, f2 = mid + 1;
    while (f1 <= mid && f2 <= r){
        if (node[f1].f <= node[f2].f) {
            sum[node[f1].idx] += query(node[f1].g - 1);
            tmp[cnt++] = node[f1++];
        }else{
            add(node[f2].g, 1);
            tmp[cnt++] = node[f2++];
        }
    }
    while (f1 <= mid){
        sum[node[f1].idx] += query(node[f1].g - 1);
        tmp[cnt++] = node[f1++];
    }
    for (int i = mid + 1; i < f2; ++i) add(node[i].g, -1);
    while (f2 <= r){
        tmp[cnt++] = node[f2++];
    }
    for (int i = l; i <= r; ++i){
        node[i] = tmp[i];
    }
}

void cdq2(int l, int r){
    int mid = (l + r) >> 1;
    if (l == r) {
        node[l].f = f_[l];
        node[l].g = g_[l];
        node[l].idx = l;
        return;
    }
    cdq2(l, mid);
    cdq2(mid + 1, r);
    int cnt = l, f1 = l, f2 = mid + 1;
    while (f1 <= mid && f2 <= r){
        if (node[f1].f < node[f2].f) {
            add(node[f1].g, 1);
            tmp[cnt++] = node[f1++];
```

```
        }else{
            sum[node[f2].idx] -= query(node[f2].g - 1);
            tmp[cnt++] = node[f2++];
        }
    }
    while(f2 <= r) {
        sum[node[f2].idx] -= query(node[f2].g - 1);
        tmp[cnt++] = node[f2++];
    }
    for(int i = l; i < f1; i++) add(node[i].g, -1);
    while(f1 <= mid){
        tmp[cnt++] = node[f1++];
    }
    for (int i = l; i <= r; ++i){
        node[i] = tmp[i];
    }
}

void solve(){
    cin >> n;
    for (int i = 1; i <= n; ++i) cin >> f_[i];
    for (int i = 1; i <= n; ++i) cin >> g_[i];
    ll ans = 1ll * n * (n - 1) * (n - 2) / 6;
    for (int i = 1; i <= n; ++i) sum[i] = i - 1;
    cdq1(1, n);
    cdq2(1, n);
    // for (int i = 1; i <= n; ++i){
    //     cout << sum[i] << " \n"[i == n];
    // }
    for (int i = 1; i <= n; ++i) ans = ans - sum[i] * (sum[i] - 1) / 2;
    cout << ans << endl;
}

signed main(){
    ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
    int t = 1;
    //cin >> t;
    while(t--) solve();
    return 0;
}
```

CF1045G

火星上有 N 个机器人排成一行，第 i 个机器人的位置为 x_i，视野为 r_i，智商为 q_i。我们认为第 i 个机器人可以看到的位置是 $[x_i - r_i, x_i + r_i]$。
如果一对机器人相互可以看到，且它们的智商 q_i 的差距**不大于**K，那么它们会开始聊天。为了防止它们吵起来，请计算有多少对机器人可能会聊天。

```
#include <algorithm>
#include <bits/stdc++.h>
#define endl '\n'
#define int ll
using namespace std;
typedef long double db;
typedef long long ll;

const ll N = 1e5 + 10;
const ll mod = 998244353;
const ll inf32 = 0x3f3f3f3f;
const ll inf64 = 5e18;

struct Query{
    int pos, R, l, r;
    int q;
}qry[N];

int tr[N << 2], mx, n, k, ans = 0;

int lowbit(int x) {return x & -x;}

```

```cpp
void add(int p, int val){
    for (int i = p; i <= mx; i += lowbit(i)) tr[i] += val;
}

int query(int p){
    int res = 0;
    for (int i = p; i; i -= lowbit(i)) res += tr[i];
    return res;
}

int query(int p1, int p2){
    return query(p2) - query(p1 - 1);
}

bool cmplen(const Query& x, const Query& y){
    return x.R > y.R;
}

unordered_map<int, int> mp;
set<int> t;

Query tmp[N];

void merge(int l, int mid, int r){
    int cnt = l;
    int i = l, j = mid + 1;
    while (i <= mid && j <= r) tmp[cnt++] = qry[i].q <= qry[j].q ? qry[i++] : qry[j++];
    while (i <= mid) tmp[cnt++] = qry[i++];
    while (j <= r) tmp[cnt++] = qry[j++];
    for (int i = l; i <= r; ++i) qry[i] = tmp[i];
}

void cdq(int l, int r){
    int mid = (l + r) >> 1;
    if (l == r) return;
    cdq(l, mid);
    cdq(mid + 1 ,r);
    int cl = l, cr = l - 1;
    for (int i = mid + 1; i <= r; ++i){
        auto [pos, R, ql, qr, q] = qry[i];
        while (cl <= mid && q - k > qry[cl].q) add(qry[cl++].pos, -1);
        while (cr < mid && qry[cr + 1].q <= q + k) add(qry[++cr].pos, 1);
        ans += query(ql, qr);
    }
    for (int i = cl; i <= cr; ++i) add(qry[i].pos, -1);
    merge(l, mid, r);// 对 q 排序满足第二个偏序条件
}

void solve(){
    cin >> n >> k;
    for (int i = 1; i <= n; ++i){
        auto &[pos, R, _1, _2, q] = qry[i];
        cin >> pos >> R >> q;
        t.insert(pos);
        t.insert(pos - R);
        t.insert(pos + R);
    }
    for (auto v : t) mp[v] = ++mx;
    for (int i = 1; i <= n; ++i){
        qry[i].l = mp[qry[i].pos - qry[i].R];
        qry[i].r = mp[qry[i].pos + qry[i].R];
        qry[i].pos = mp[qry[i].pos];
    }
    // 对 R 排序满足第一个偏序条件
    sort(qry + 1, qry + n + 1, cmplen);
    cdq(1, n);
    cout << ans << endl;
}

signed main(){
    ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
```

```
    int t = 1;
    //cin >> t;
    while(t--) solve();
    return 0;
}
```

陌上开花（‘纯板子’）

有 $ n $ 个元素，第 $ i $ 个元素有 $ a_i,b_i,c_i $ 三个属性，设 $ f(i) $ 表示满足 $ a_j ⩽a_i $ 且 $ b_j ⩽b_i $ 且 $ c_j ⩽c_i $ 且 $ j ≠i $ 的 $ j $ 的数量。

```
#include <bits/stdc++.h>
#include <vector>
#define endl '\n'
using namespace std;
typedef long double db;
typedef long long ll;

const ll N = 2e5 + 10;
const ll mod = 998244353;
const ll inf32 = 0x3f3f3f3f;
const ll inf64 = 5e18;

struct Query{
    int a, b, c;
    int w, ans;
}node[N], qry[N];

struct BIT{
    vector<int> tr;
    void init(int n){
        vector<int>(n + 1).swap(tr);
    }
    int lowbit(int x) {return x & -x;}
    void add(int x, int val){
        for (; x < tr.size(); x += lowbit(x))
            tr[x] += val;
    }
    int query(int x){
        int res = 0;
        for (; x; x -= lowbit(x)){
            res += tr[x];
        }
        return res;
    }
}T;

bool cmpa(const Query& u, const Query& v){
    if (u.a == v.a){
        if (u.b == v.b)
            return u.c < v.c;
        return u.b < v.b;
    }
    return u.a < v.a;
}

bool cmpb(const Query& u, const Query& v){
    if (u.b == v.b){
        return u.c < v.c;
    }
    return u.b < v.b;
}

Query tmp[N];
ll cnt[N];

void cdq(int l, int r){
    if (l == r) return;
    int mid = (l + r) >> 1;
    cdq(l, mid);
    cdq(mid + 1, r);
```

```
    sort(qry + l, qry + mid + 1, cmpb);
    sort(qry + mid + 1, qry + r + 1, cmpb);
    int f2 = mid + 1, f1 = l;
    for (; f2 <= r; ++f2){
        while (qry[f1].b <= qry[f2].b && f1 <= mid){
            T.add(qry[f1].c, qry[f1].w);
            f1++;
        }
        qry[f2].ans += T.query(qry[f2].c);
    }
    for (int i = l; i < f1; ++i) T.add(qry[i].c, -qry[i].w);
}

void solve(){
    int n_, k, n = 0;
    cin >> n_ >> k;
    T.init(k);
    for (int i = 1; i <= n_; ++i){
        auto &[a, b, c, w, ans] = node[i];
        cin >> a >> b >> c;
    }
    sort(node + 1, node + n_ + 1, cmpa);
    int c = 0;
    for (int i = 1; i <= n_; ++i){
        c++;
        if (node[i].a != node[i + 1].a || node[i].b != node[i + 1].b || node[i].c != node[i + 1].c){
            qry[++n] = node[i];
            qry[n].w = c;
            c = 0;
        }
    }
    cdq(1, n);
    for (int i = 1; i <= n; ++i){
        cnt[qry[i].ans + qry[i].w - 1] += qry[i].w;
    }
    for (int i = 0; i < n_; ++i) cout << cnt[i] << endl;
}

signed main(){
    ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
    int t = 1;
    //cin >> t;
    while(t--) solve();
    return 0;
}
```

dsu on tree

点分树模版 (距离为 k 是否存在)

```
#include <bits/stdc++.h>
#define endl '\n'

using ll = long long;

constexpr int N = 2e5 + 10;
constexpr int mod = 998244353;

using namespace std;

void solve() {
    int n, m;
    cin >> n >> m;
    vector<vector<pair<int, int>>> G(n + 1);
    for(int i = 1; i < n; i++) {
        ll u, v, w;
        cin >> u >> v >> w;
        G[u].emplace_back(v, w);
        G[v].emplace_back(u, w);
    }

```

```
    vector <int> q(m + 1), ans(m + 1);

    for(int i = 1; i <= m; i++) {
        cin >> q[i];
    }

    set <int> s;
    for(int i = 1; i <= m; i++) {
        s.insert(i);
    }

    vector <int> l(n + 1), r(n + 1), siz(n + 1), son(n + 1), id(n + 1);
    vector <ll> dis(n + 1);
    int idx = 0;

    auto dfs1 = [&](auto self, int now, int p) -> void {
        l[now] = ++idx, id[idx] = now;
        for(auto [u, w] : G[now]) {
            if(u == p) continue;
            dis[u] = dis[now] +  w;
            self(self, u, now);
            siz[now] += siz[u];
            if(siz[u] > siz[son[now]]) {
                son[now] = u;
            }
        }
        siz[now]++;
        r[now] = idx;
    };

    dfs1(dfs1, 1, 0);

    unordered_map<ll, int> cnt;

    auto upd = [&](int x, int k) {
        cnt[dis[x]] += k;
    };

    auto dfs2 = [&](auto self, int now, int p, int keep) -> void {
        for(auto [u, w] : G[now]) {
            if(u == p || u == son[now]) continue;
            self(self, u, now, 0);
        }

        if(son[now]) self(self, son[now], now, 1);

        vector <int> ww;
        for(auto i : s) {
            if(cnt[dis[now] + q[i]]) {
                ans[i] = 1;
                ww.push_back(i);
            }
        }

        while(!ww.empty()) {
            s.erase(ww.back());
            ww.pop_back();
        }

        upd(now, 1);

        for(auto [u, w] : G[now]) {
            if(u == p || u == son[now]) continue;
            for(int i = l[u]; i <= r[u]; i++) {
                for(auto j : s) {
                    ll res = dis[now] + dis[now] + q[j] - dis[id[i]];
                    if(cnt[res]) {
                        ans[j] = 1;
                        ww.push_back(j);
                    }
                }
```

```
            }

            while(!ww.empty()) {
                s.erase(ww.back());
                ww.pop_back();
            }

            for(int i = l[u]; i <= r[u]; i++) {
                upd(id[i], 1);
            }
        }

        if(!keep) {
            for(int i = l[now]; i <= r[now]; i++) {
                upd(id[i], -1);
            }
        }
    };

    dfs2(dfs2, 1, 0, 1);

    for(int i = 1; i <= m; i++) {
        cout << (ans[i] ? "AYE" : "NAY") << endl;
    }

}

signed main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    //cin >> t;
    while(t--) solve();
    return 0;
}
```

CF600E(纯板子)

```
#include <bits/stdc++.h>
#define endl '\n'

using ll = long long;

constexpr int N = 2e5 + 10;
constexpr int mod = 998244353;

using namespace std;

int cnt[N];

void solve(){
    int n;
    cin >> n;
    vector<int> col(n + 1);
    for (int i = 1; i <= n; ++i){
        cin >> col[i];
    }
    vector<vector<int>> G(n + 1);
    for (int i = 1; i < n; ++i){
        int u, v;
        cin >> u >> v;
        G[u].push_back(v);
        G[v].push_back(u);
    }

    vector<int> l(n + 1), r(n + 1), rnk(n + 1), sz(n + 1), son(n + 1);
    int tot = 0;

    function<void(int, int)> dfs1 = [&](int u, int fa){
        sz[u] = 1;
        l[u] = ++tot;
```

```
        rnk[tot] = u;
        for (auto v : G[u]){
            if (v == fa) continue;
            dfs1(v, u);
            sz[u] += sz[v];
            if (sz[v] > sz[son[u]]) son[u] = v;
        }
        r[u] = tot;
    };

    dfs1(1, 0);

    vector<ll> ans(n + 1);
    ll res = 0, mx = -1;

    auto modify = [&] (int x){
        cnt[col[x]]++;
        if (cnt[col[x]] == mx) res += col[x];
        else if (cnt[col[x]] > mx) {
            mx = cnt[col[x]];
            res = col[x];
        }
    };

    function<void(int, int, int)> dfs2 = [&](int u, int fa, int keep){
        for (auto v : G[u]){
            if (v == fa || v == son[u]) continue;
            dfs2(v, u, 0);
        }
        if (son[u]) dfs2(son[u], u, 1);

        modify(u);

        for (auto v : G[u]){
            if (v == fa || v == son[u]) continue;
            for (int i = l[v]; i <= r[v]; ++i){
                modify(rnk[i]);
            }
        }

        ans[u] = res;

        if (!keep){
            for (int i = l[u]; i <= r[u]; ++i){
                cnt[col[rnk[i]]]--;
            }
            mx = -1;
            res = 0;
        }
    };

    dfs2(1, 0, 1);

    for (int i = 1; i <= n; ++i) cout << ans[i] << " \n"[i == n];
}

signed main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    //cin >> t;
    while(t--) solve();
    return 0;
}
```

LCT

LCA(询问以 root 为根 x, y 的 LCA, 询问格式 root, x, y)

```
#include <algorithm>
#include <bits/stdc++.h>
```

```cpp
#include <cassert>
#include <iostream>
#include <queue>
#include <vector>
#define endl '\n'
#define pll pair<i64, i64>
#define tll tuple<i64, i64, i64>
#define all(a) a.begin() + 1, a.end()
using namespace std;
using db = long double;
using i64 = long long;
const i64 N = 1e5 + 10;
const i64 mod = 998244353;
const i64 inf32 = 1e9;
const i64 inf64 = 5e18;

int n, m;
int st[N];
struct node{
    int son[2];
    int fa, tag;
    int sz;
}tr[N];

inline int is_root(int rt){
    return tr[tr[rt].fa].son[0] != rt && tr[tr[rt].fa].son[1] != rt;
}

inline void push_down(int rt){
    if (!rt || !tr[rt].tag) return;
    tr[tr[rt].son[0]].tag ^= 1;
    tr[tr[rt].son[1]].tag ^= 1;
    tr[rt].tag ^= 1;
    swap(tr[rt].son[0], tr[rt].son[1]);
}


inline void rotate(int rt){
    int f = tr[rt].fa, gf = tr[f].fa, k = tr[f].son[0] == rt ? 0 : 1;
    if (!is_root(f)){
        if (tr[gf].son[0] == f) tr[gf].son[0] = rt;
        else tr[gf].son[1] = rt;
    }
    tr[rt].fa = gf; tr[f].fa = rt;
    tr[tr[rt].son[!k]].fa = f;
    tr[f].son[k] = tr[rt].son[!k];
    tr[rt].son[!k] = f;
}

void splay(int rt){
    int top = 0;
    st[++top] = rt;
    for (int i = rt; !is_root(i); i = tr[i].fa){
        st[++top] = tr[i].fa;
    }
    while (top){
        push_down(st[top--]);
    }
    while (!is_root(rt)){
        int f = tr[rt].fa, gf = tr[f].fa;
        if (!is_root(f)){
            if ((tr[f].son[0] == rt) ^ (tr[gf].son[0] == f)) rotate(rt);
            else rotate(f);
        }
        rotate(rt);
    }
}


int access(int rt){
    int tmp = 0;
```

```
    while (rt){
        splay(rt);
        tr[rt].son[1] = tmp;
        tmp = rt;
        rt = tr[rt].fa;
    }
    return tmp;
}

void makeroot(int rt){
    access(rt);
    splay(rt);
    tr[rt].tag ^= 1;
}

int findroot(int rt){
    access(rt);
    splay(rt);
    int tmp = rt;
    while (tr[tmp].son[0]) tmp = tr[tmp].son[0];
    splay(tmp);
    return tmp;
}

void cut(int x, int y){
    makeroot(x);
    access(y);
    splay(y);
    tr[y].son[0] = tr[x].fa = 0;
}

void link(int x, int y){
    makeroot(x);
    tr[x].fa = y;
}

int lca(int rt, int x, int y){
    makeroot(rt);
    access(x);
    return access(y);
}

void solve(){
    cin >> n;
    for (int i = 1, u, v; i < n; ++i){
        cin >> u >> v;
        link(u, v);
    }
    cin >> m;
    while (m--){
        int rt, x, y;
        cin >> rt >> x >> y;
        cout << lca(rt, x, y) << endl;
    }
}

int main(){
    ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
    int t = 1;
    //cin >> t;
    while(t--) solve();
    return 0;
}
```

HNOI 弹飞绵羊 (树上两点间距离，连/断)

```
#include <algorithm>
#include <bits/stdc++.h>
#include <cassert>
#include <iostream>
#include <queue>
```

```cpp
#include <vector>
#define endl '\n'
#define pll pair<i64, i64>
#define tll tuple<i64, i64, i64>
#define all(a) a.begin() + 1, a.end()
using namespace std;
using db = long double;
using i64 = long long;
const i64 N = 2e5 + 10;
const i64 mod = 998244353;
const i64 inf32 = 1e9;
const i64 inf64 = 5e18;

int n, m;
int st[N];
struct node{
    int son[2];
    int fa, tag;
    i64 sz;
}tr[N];

inline int is_root(int rt){
    return tr[tr[rt].fa].son[0] != rt && tr[tr[rt].fa].son[1] != rt;
}

inline void push_down(int rt){
    if (!rt || !tr[rt].tag) return;
    tr[tr[rt].son[0]].tag ^= 1;
    tr[tr[rt].son[1]].tag ^= 1;
    tr[rt].tag ^= 1;
    swap(tr[rt].son[0], tr[rt].son[1]);
}

inline void push_up(int rt){
    tr[rt].sz = tr[tr[rt].son[0]].sz + tr[tr[rt].son[1]].sz + 1;
}

inline void rotate(int rt){
    int f = tr[rt].fa, gf = tr[f].fa, k = tr[f].son[0] == rt ? 0 : 1;
    if (!is_root(f)){
        if (tr[gf].son[0] == f) tr[gf].son[0] = rt;
        else tr[gf].son[1] = rt;
    }
    tr[rt].fa = gf; tr[f].fa = rt;
    tr[tr[rt].son[!k]].fa = f;
    tr[f].son[k] = tr[rt].son[!k];
    tr[rt].son[!k] = f;
    push_up(f);
}

void splay(int rt){
    int top = 0;
    st[++top] = rt;
    for (int i = rt; !is_root(i); i = tr[i].fa){
        st[++top] = tr[i].fa;
    }
    while (top){
        push_down(st[top--]);
    }
    while (!is_root(rt)){
        int f = tr[rt].fa, gf = tr[f].fa;
        if (!is_root(f)){
            if ((tr[f].son[0] == rt) ^ (tr[gf].son[0] == f)) rotate(rt);
            else rotate(f);
        }
        rotate(rt);
        push_up(rt);
    }
}


```

```cpp
int access(int rt){
    int tmp = 0;
    while (rt){
        splay(rt);
        tr[rt].son[1] = tmp;
        push_up(rt);
        tmp = rt;
        rt = tr[rt].fa;
    }
    return tmp;
}

void makeroot(int rt){
    access(rt);
    splay(rt);
    tr[rt].tag ^= 1;
}

int findroot(int rt){
    access(rt);
    splay(rt);
    int tmp = rt;
    while (tr[tmp].son[0]) tmp = tr[tmp].son[0];
    splay(tmp);
    return tmp;
}

void cut(int x, int y){
    makeroot(x);
    access(y);
    splay(y);
    tr[y].son[0] = tr[x].fa = 0;
}

void link(int x, int y){
    makeroot(x);
    tr[x].fa = y;
}

int lca(int rt, int x, int y){
    makeroot(rt);
    access(x);
    return access(y);
}

void solve(){
    cin >> n;
    vector<int> p(n + 10);
    for (int i = 1; i <= n; ++i){
        tr[i].sz = 1;
    }
    for (int i = 1; i <= n; ++i){
        cin >> p[i];
        link(i, min(p[i] + i, n + 1));
        p[i] = min(p[i] + i, n + 1);
    }
    cin >> m;
    while(m--){
        int op;
        cin >> op;
        if (op == 1){
            int x;
            cin >> x;
            x++;
            makeroot(n + 1);
            access(x), splay(x);
            cout << tr[tr[x].son[0]].sz << endl;
        }else{
            int x, y;
            cin >> x >> y;
            x++;
```

```
            int t = min(n + 1, x + y);
            cut(x, p[x]);
            link(x, t);
            p[x] = t;
        }
    }
}

int main(){
    ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
    int t = 1;
    //cin >> t;
    while(t--) solve();
    return 0;
}
```

动态并查集

```
#include <algorithm>
#include <bits/stdc++.h>
#include <cassert>
#include <queue>
#include <vector>
#define endl '\n'
#define pll pair<i64, i64>
#define tll tuple<i64, i64, i64>
#define all(a) a.begin() + 1, a.end()
using namespace std;
using db = long double;
using i64 = long long;
const i64 N = 1e4 + 10;
const i64 mod = 998244353;
const i64 inf32 = 1e9;
const i64 inf64 = 5e18;

int n, m;
int st[N];
struct node{
    int son[2];
    int fa, tag;
}tr[N];

inline int is_root(int rt){
    return tr[tr[rt].fa].son[0] != rt && tr[tr[rt].fa].son[1] != rt;
}

inline void push_down(int rt){
    if (!rt || !tr[rt].tag) return;
    tr[tr[rt].son[0]].tag ^= 1;
    tr[tr[rt].son[1]].tag ^= 1;
    tr[rt].tag ^= 1;
    swap(tr[rt].son[0], tr[rt].son[1]);
}


inline void rotate(int rt){
    int f = tr[rt].fa, gf = tr[f].fa, k = tr[f].son[0] == rt ? 0 : 1;
    if (!is_root(f)){
        if (tr[gf].son[0] == f) tr[gf].son[0] = rt;
        else tr[gf].son[1] = rt;
    }
    tr[rt].fa = gf; tr[f].fa = rt;
    tr[tr[rt].son[!k]].fa = f;
    tr[f].son[k] = tr[rt].son[!k];
    tr[rt].son[!k] = f;
}

void splay(int rt){
    int top = 0;
    st[++top] = rt;
    for (int i = rt; !is_root(i); i = tr[i].fa){
```

```
        st[++top] = tr[i].fa;
    }
    while (top){
        push_down(st[top--]);
    }
    while (!is_root(rt)){
        int f = tr[rt].fa, gf = tr[f].fa;
        if (!is_root(f)){
            if ((tr[f].son[0] == rt) ^ (tr[gf].son[0] == f)) rotate(rt);
            else rotate(f);
        }
        rotate(rt);
    }
}


int access(int rt){
    int tmp = 0;
    while (rt){
        splay(rt);
        tr[rt].son[1] = tmp;
        tmp = rt;
        rt = tr[rt].fa;
    }
    return tmp;
}

void makeroot(int rt){
    access(rt);
    splay(rt);
    tr[rt].tag ^= 1;
}

int findroot(int rt){
    access(rt);
    splay(rt);
    int tmp = rt;
    while (tr[tmp].son[0]) tmp = tr[tmp].son[0];
    splay(tmp);
    return tmp;
}

void cut(int x, int y){
    makeroot(x);
    access(y);
    splay(y);
    tr[y].son[0] = tr[x].fa = 0;
}

void link(int x, int y){
    makeroot(x);
    tr[x].fa = y;
}

void solve(){
    cin >> n >> m;
    while (m--){
        int x, y;
        string s;
        cin >> s;
        cin >> x >> y;
        if (s[0] == 'C') link(x, y);
        if (s[0] == 'D') cut(x, y);
        if (s[0] == 'Q'){
            if (findroot(x) == findroot(y)){
                cout << "Yes" << endl;
            }else{
                cout << "No" << endl;
            }
        }
    }
```

```
}

int main(){
    ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
    int t = 1;
    //cin >> t;
    while(t--) solve();
    return 0;
}
```

LCT(最小生成树)

```
#include <bits/stdc++.h>
#define endl '\n'
using namespace std;
typedef long double db;
typedef long long ll;

const ll N = 4e5 + 10;
const ll mod = 998244353;
const ll inf32 = 0x3f3f3f3f;
const ll inf64 = 5e18;

int n, m, idx, ans, cnt;
namespace LCT
{
    struct node
    {
        int ch[2], w, id, fa, sz, mx, tag;
    } t[N];
#define isroot(x) ((t[t[x].fa].ch[0] != x) && (t[t[x].fa].ch[1] != x))
    void pushr(int x)
    {
        swap(t[x].ch[0], t[x].ch[1]);
        t[x].tag ^= 1;
    }
    void push_up(int x)
    {
        t[x].id = x;
        t[x].mx = t[x].w;
        if (t[t[x].ch[0]].mx > t[x].mx)
            t[x].mx = t[t[x].ch[0]].mx, t[x].id = t[t[x].ch[0]].id;
        if (t[t[x].ch[1]].mx > t[x].mx)
            t[x].mx = t[t[x].ch[1]].mx, t[x].id = t[t[x].ch[1]].id;
    }
    void push_down(int x)
    {
        if (t[x].tag)
        {
            if (t[x].ch[0])
                pushr(t[x].ch[0]);
            if (t[x].ch[1])
                pushr(t[x].ch[1]);
            t[x].tag = 0;
        }
    }
    void push_all(int x)
    {
        if (!isroot(x))
            push_all(t[x].fa);
        push_down(x);
    }
    void rotate(int x)
    {
        int y = t[x].fa;
        int z = t[y].fa;
        int k = t[y].ch[1] == x;
        if (!isroot(y))
            t[z].ch[t[z].ch[1] == y] = x;
        t[x].fa = z;
        t[y].ch[k] = t[x].ch[k ^ 1];
```

```
60          t[t[x].ch[k ^ 1]].fa = y;
61          t[x].ch[k ^ 1] = y;
62          t[y].fa = x;
63          push_up(y);
64          push_up(x);
65      }
66      void splay(int x)
67      {
68          push_all(x);
69          while (!isroot(x))
70          {
71              int y = t[x].fa;
72              int z = t[y].fa;
73              if (!isroot(y))
74                  (t[z].ch[1] == y) ^ (t[y].ch[1] == x) ? rotate(x) : rotate(y);
75              rotate(x);
76          }
77      }
78      void access(int x)
79      {
80          for (int y = 0; x; y = x, x = t[x].fa)
81              splay(x), t[x].ch[1] = y, push_up(x);
82      }
83      void make_root(int x)
84      {
85          access(x);
86          splay(x);
87          pushr(x);
88      }
89      int find_root(int x)
90      {
91          access(x);
92          splay(x);
93          while (t[x].ch[0])
94              push_down(x), x = t[x].ch[0];
95          splay(x);
96          return x;
97      }
98      void split(int x, int y)
99      {
100         make_root(x);
101         access(y);
102         splay(y);
103     }
104     void link(int x, int y)
105     {
106         make_root(x);
107         if (find_root(y) != x)
108             t[x].fa = y;
109     }
110     int ck(int x, int y)
111     {
112         make_root(x);
113         return find_root(y) != x;
114     }
115     void cut(int x, int y)
116     {
117         make_root(x);
118         if (find_root(y) == x && t[y].fa == x && !t[y].ch[0])
119         {
120             t[x].ch[1] = t[y].fa = 0;
121             push_up(x);
122         }
123     }
124 }
125
126 void solve(){
127     using namespace LCT;
128     cin >> n >> m;
129     idx = n;
130     for (int i = 1, x, y, z; i <= m; i++)
```

```
    {
        cin >> x >> y >> z;
        t[++idx].w = z;
        if (x != y && ck(x, y))
            link(x, idx), link(idx, y), ans += z, ++cnt;
        else
        {
            split(x, y);
            int now = t[y].id;
            if (t[now].mx <= z)
                continue;
            ans -= t[now].mx - z;
            splay(now);
            t[t[now].ch[0]].fa = t[t[now].ch[1]].fa = 0;
            link(x, idx);
            link(idx, y);
        }
    }
    if (cnt < n - 1) cout << "orz" << endl;
    else cout << ans << endl;
}

signed main(){
    ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
    int t = 1;
    //cin >> t;
    while(t--) solve();
    return 0;
}
```

最小生成树的运用 (树上构建最小生成树，每个边有权值 a，b 满足 1 到 n 的 max(a) + max(b) 最小)

```
#include <bits/stdc++.h>
#define endl '\n'
using namespace std;
typedef long double db;
typedef long long ll;

const ll N = 2e5 + 10;
const ll mod = 998244353;
const ll inf32 = 0x3f3f3f3f;
const ll inf64 = 5e18;

int n, m, ans = inf32;
namespace LCT
{
    struct Edge{
        int u, v, a, b;
    }e[N];
    int cmp(Edge a, Edge b){return a.a == b.a ? a.b < b.b : a.a < b.a;}
    struct node
    {
        int ch[2], w, id, fa, sz, mx, tag;
    } t[N];
#define isroot(x) ((t[t[x].fa].ch[0] != x) && (t[t[x].fa].ch[1] != x))
    void pushr(int x)
    {
        swap(t[x].ch[0], t[x].ch[1]);
        t[x].tag ^= 1;
    }
    void push_up(int x)
    {
        t[x].id = x;
        t[x].mx = t[x].w;
        if (t[t[x].ch[0]].mx > t[x].mx)
            t[x].mx = t[t[x].ch[0]].mx, t[x].id = t[t[x].ch[0]].id;
        if (t[t[x].ch[1]].mx > t[x].mx)
            t[x].mx = t[t[x].ch[1]].mx, t[x].id = t[t[x].ch[1]].id;
    }
    void push_down(int x)
    {
```

```
        if (t[x].tag)
        {
            if (t[x].ch[0])
                pushr(t[x].ch[0]);
            if (t[x].ch[1])
                pushr(t[x].ch[1]);
            t[x].tag = 0;
        }
    }
    void push_all(int x)
    {
        if (!isroot(x))
            push_all(t[x].fa);
        push_down(x);
    }
    void rotate(int x)
    {
        int y = t[x].fa;
        int z = t[y].fa;
        int k = t[y].ch[1] == x;
        if (!isroot(y))
            t[z].ch[t[z].ch[1] == y] = x;
        t[x].fa = z;
        t[y].ch[k] = t[x].ch[k ^ 1];
        t[t[x].ch[k ^ 1]].fa = y;
        t[x].ch[k ^ 1] = y;
        t[y].fa = x;
        push_up(y);
        push_up(x);
    }
    void splay(int x)
    {
        push_all(x);
        while (!isroot(x))
        {
            int y = t[x].fa;
            int z = t[y].fa;
            if (!isroot(y))
                (t[z].ch[1] == y) ^ (t[y].ch[1] == x) ? rotate(x) : rotate(y);
            rotate(x);
        }
    }
    void access(int x)
    {
        for (int y = 0; x; y = x, x = t[x].fa)
            splay(x), t[x].ch[1] = y, push_up(x);
    }
    void make_root(int x)
    {
        access(x);
        splay(x);
        pushr(x);
    }
    int find_root(int x)
    {
        access(x);
        splay(x);
        while (t[x].ch[0])
            push_down(x), x = t[x].ch[0];
        splay(x);
        return x;
    }
    void split(int x, int y)
    {
        make_root(x);
        access(y);
        splay(y);
    }
    void link(int x, int y)
    {
        make_root(x);
```

```
        if (find_root(y) != x)
            t[x].fa = y;
    }
    int ck(int x, int y)
    {
        make_root(x);
        return find_root(y) != x;
    }
    void cut(int x, int y)
    {
        make_root(x);
        if (find_root(y) == x && t[y].fa == x && !t[y].ch[0])
        {
            t[x].ch[1] = t[y].fa = 0;
            push_up(x);
        }
    }
}

void solve(){
    using namespace LCT;
    cin >> n >> m;
    for (int i = 1; i <= m; ++i){
        auto &[u, v, a, b] = e[i];
        cin >> u >> v >> a >> b;
    }
    sort(e + 1, e + 1 + m, cmp);
    for (int i = 1; i <= m; ++i){
        int idx = n + i;
        auto [x, y, a, b] = e[i];
        t[idx].w = b;
        if (x == y) continue;
        if (ck(x, y)){
            link(x, idx);
            link(idx, y);
        }else{
            split(x, y);
            int now = t[y].id;
            if (t[now].mx <= b) continue;
            splay(now);
            t[t[now].ch[0]].fa = t[t[now].ch[1]].fa = 0;
            link(x, idx);
            link(idx, y);
        }
        if (!ck(1, n)){
            split(1, n);
            ans = min(ans, t[n].mx + a);
        }
    }
    if (ans == inf32) cout << -1 << endl;
    else cout << ans << endl;
}

signed main(){
    ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
    int t = 1;
    //cin >> t;
    while(t--) solve();
    return 0;
}
```

LCT 树上路径信息维护

一棵 n 个点的树，每个点的初始权值为 1。对于这棵树有 q 个操作，每个操作为以下四种操作之一：

+ u v c：将 u 到 v 的路径上的点的权值都加上自然数 c；

- u1 v1 u2 v2：将树中原有的边 (u1,v1) 删除，加入一条新边 (u2,v2)，保证操作完之后仍然是一棵树；

* u v c：将 u 到 v 的路径上的点的权值都乘上自然数 c；

/ u v：询问 u 到 v 的路径上的点的权值和，求出答案对于 51061 的余数。

数据范围：1 <= n，q <= 10^5，0 <= c <= 10^4。

```cpp
#include <bits/stdc++.h>
#define endl '\n'
using namespace std;
typedef long double db;
typedef long long ll;

const ll N = 2e5 + 10;
const ll mod = 51061;
const ll inf32 = 0x3f3f3f3f;
const ll inf64 = 5e18;

int n, q;
namespace LCT
{
    struct node
    {
        int ch[2], id, fa, sz, tag;
        ll taga, tagc;//加法，乘法 tag
        ll sum, val;
    } t[N];
#define isroot(x) ((t[t[x].fa].ch[0] != x) && (t[t[x].fa].ch[1] != x))
    void push_up(int x)
    {
        t[x].sum = (t[x].val + t[t[x].ch[0]].sum % mod + t[t[x].ch[1]].sum) % mod;
        t[x].sz = 1 + t[t[x].ch[0]].sz + t[t[x].ch[1]].sz;
    }
    void pushr(int x)
    {
        swap(t[x].ch[0], t[x].ch[1]);
        t[x].tag ^= 1;
    }
    void pusha(int x, int c){
        t[x].taga = t[x].taga + c % mod;
        t[x].val = t[x].val + c % mod;
        t[x].sum = t[x].sum + (c * t[x].sz % mod) % mod;
    }
    void pushc(int x, int c){
        t[x].taga = t[x].taga * c % mod;
        t[x].val = t[x].val * c % mod;
        t[x].sum = t[x].sum * c % mod;
        t[x].tagc = t[x].tagc * c % mod;
    }
    void push_down(int x)
    {
        if (t[x].tag)
        {
            if (t[x].ch[0])
                pushr(t[x].ch[0]);
            if (t[x].ch[1])
                pushr(t[x].ch[1]);
            t[x].tag = 0;
        }
        if (t[x].tagc != 1){
            pushc(t[x].ch[0], t[x].tagc);
            pushc(t[x].ch[1], t[x].tagc);
            t[x].tagc = 1;
        }
        if (t[x].taga){
            pusha(t[x].ch[0], t[x].taga);
            pusha(t[x].ch[1], t[x].taga);
            t[x].taga = 0;
        }
    }
    void push_all(int x)
    {
        if (!isroot(x))
            push_all(t[x].fa);
        push_down(x);
```

```
    }
    void rotate(int x)
    {
        int y = t[x].fa;
        int z = t[y].fa;
        int k = t[y].ch[1] == x;
        if (!isroot(y))
            t[z].ch[t[z].ch[1] == y] = x;
        t[x].fa = z;
        t[y].ch[k] = t[x].ch[k ^ 1];
        t[t[x].ch[k ^ 1]].fa = y;
        t[x].ch[k ^ 1] = y;
        t[y].fa = x;
        push_up(y);
        push_up(x);
    }
    void splay(int x)
    {
        push_all(x);
        while (!isroot(x))
        {
            int y = t[x].fa;
            int z = t[y].fa;
            if (!isroot(y))
                (t[z].ch[1] == y) ^ (t[y].ch[1] == x) ? rotate(x) : rotate(y);
            rotate(x);
        }
    }
    void access(int x)
    {
        for (int y = 0; x; y = x, x = t[x].fa)
            splay(x), t[x].ch[1] = y, push_up(x);
    }
    void make_root(int x)
    {
        access(x);
        splay(x);
        pushr(x);
    }
    int find_root(int x)
    {
        access(x);
        splay(x);
        while (t[x].ch[0])
            push_down(x), x = t[x].ch[0];
        splay(x);
        return x;
    }
    void split(int x, int y)
    {
        make_root(x);
        access(y);
        splay(y);
    }
    void link(int x, int y)
    {
        make_root(x);
        if (find_root(y) != x)
            t[x].fa = y;
    }
    int ck(int x, int y)
    {
        make_root(x);
        return find_root(y) != x;
    }
    void cut(int x, int y)
    {
        make_root(x);
        if (find_root(y) == x && t[y].fa == x && !t[y].ch[0])
        {
            t[x].ch[1] = t[y].fa = 0;
```

```
            push_up(x);
        }
    }
}

void solve(){
    using namespace LCT;
    cin >> n >> q;
    for (int i = 1; i <= n; ++i) {
        t[i].sz = t[i].tagc = 1;
        t[i].val = t[i].sum = 1;
        t[i].taga = 0;
    }
    for (int i = 1; i < n; ++i){
        int u, v;
        cin >> u >> v;
        link(u, v);
    }
    while (q--){
        char op;
        int x, y, z;
        cin >> op;
        if (op == '+'){
            cin >> x >> y >> z;
            split(x, y);
            pusha(y, z);
        }else if (op == '-'){
            cin >> x >> y;
            cut(x, y);
            cin >> x >> y;
            link(x, y);
        }else if (op == '*'){
            cin >> x >> y >> z;
            split(x, y);
            pushc(y, z);
        }else{
            cin >> x >> y;
            split(x, y);
            cout << t[y].sum % mod << endl;
        }
    }
}

signed main(){
    ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
    int t = 1;
    //cin >> t;
    while(t--) solve();
    return 0;
}
```

大融合 (子树大小查询)

给定 n 个点，进行 Q 次操作，操作有两种:

A x y： 在 x,y 之间连一条边，保证 x,y 不连通

Q x y： 求通过边 (x,y) 的简单路径数量，保证存在边 (x,y)

N,Q <= 10^5

```
#include <bits/stdc++.h>
#define endl '\n'
using namespace std;
typedef long double db;
typedef long long ll;

const ll N = 2e5 + 10;
const ll mod = 998244353;
const ll inf32 = 0x3f3f3f3f;
const ll inf64 = 5e18;
```

```cpp

namespace LCT
{
    struct node
    {
        int ch[2], fa, tag;
        int sz, szx; //sz 实子树, szx 虚子树
    } t[N];
#define isroot(x) ((t[t[x].fa].ch[0] != x) && (t[t[x].fa].ch[1] != x))
    void pushr(int x)
    {
        swap(t[x].ch[0], t[x].ch[1]);
        t[x].tag ^= 1;
    }
    void push_up(int x)
    {
        t[x].sz = 1 + t[t[x].ch[0]].sz + t[t[x].ch[0]].szx + t[t[x].ch[1]].sz + t[t[x].ch[1]].szx;
    }
    void push_down(int x)
    {
        if (t[x].tag)
        {
            if (t[x].ch[0])
                pushr(t[x].ch[0]);
            if (t[x].ch[1])
                pushr(t[x].ch[1]);
            t[x].tag = 0;
        }
    }
    void push_all(int x)
    {
        if (!isroot(x))
            push_all(t[x].fa);
        push_down(x);
    }
    void rotate(int x)
    {
        int y = t[x].fa;
        int z = t[y].fa;
        int k = t[y].ch[1] == x;
        if (!isroot(y))
            t[z].ch[t[z].ch[1] == y] = x;
        t[x].fa = z;
        t[y].ch[k] = t[x].ch[k ^ 1];
        t[t[x].ch[k ^ 1]].fa = y;
        t[x].ch[k ^ 1] = y;
        t[y].fa = x;
        push_up(y);
        push_up(x);
    }
    void splay(int x)
    {
        push_all(x);
        while (!isroot(x))
        {
            int y = t[x].fa;
            int z = t[y].fa;
            if (!isroot(y))
                (t[z].ch[1] == y) ^ (t[y].ch[1] == x) ? rotate(x) : rotate(y);
            rotate(x);
        }
    }
    void access(int x)
    {
        for (int y = 0; x; y = x, x = t[x].fa){
            splay(x);
            if (t[x].ch[1])
                t[x].szx += t[t[x].ch[1]].sz + t[t[x].ch[1]].szx;
            t[x].ch[1] = y;
            if (y)
                t[x].szx -= t[y].sz + t[y].szx;
```

```
            push_up(x);
        }

    }
    void make_root(int x)
    {
        access(x);
        splay(x);
        pushr(x);
    }
    int find_root(int x)
    {
        access(x);
        splay(x);
        while (t[x].ch[0])
            push_down(x), x = t[x].ch[0];
        splay(x);
        return x;
    }
    void split(int x, int y)
    {
        make_root(x);
        access(y);
        splay(y);
    }
    void link(int x, int y)
    {
        split(x, y);
        t[x].fa = y;
        t[y].szx += t[x].sz + t[x].szx;
    }
    int ck(int x, int y)
    {
        make_root(x);
        return find_root(y) != x;
    }
    void cut(int x, int y)
    {
        make_root(x);
        if (find_root(y) == x && t[y].fa == x && !t[y].ch[0])
        {
            t[x].ch[1] = t[y].fa = 0;
            push_up(x);
        }
    }
}

void solve(){
    using namespace LCT;
    int n, q, x, y;
    cin >> n >> q;
    for (int i = 1; i <= n; ++i) t[i].sz = 1;
    while (q--){
        char op;
        cin >> op;
        if (op == 'A'){
            cin >> x >> y;
            link(x, y);
        }else{
            cin >> x >> y;
            split(x, y);
            cout << 1ll * (t[x].sz + t[x].szx) * (t[y].sz + t[y].szx - t[x].sz - t[x].szx) << endl;
        }
    }
}

signed main(){
    ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
    int t = 1;
    //cin >> t;
    while(t--) solve();
```

```
    return 0;
}
```

分块

分块 I(区间修改, 区间查询)

```
#include <bits/stdc++.h>
#define endl '\n'
using namespace std;
typedef long double db;
typedef long long ll;

const ll N = 2e5 + 10;
const ll mod = 998244353;
const ll inf32 = 0x3f3f3f3f;
const ll inf64 = 5e18;

int n, m, sq;
int v[N], bl[N], tag[N], ans[N];

void update(int a, int b){
    for (int i = a; i <= min(bl[a] * sq, b); ++i) {
        ans[bl[a]] -= (v[i] ^ tag[bl[a]]);
        v[i] ^= 1;
        ans[bl[a]] += (v[i] ^ tag[bl[a]]);
    }
    if (bl[a] != bl[b]){
        for (int i = (bl[b] - 1) * sq + 1; i <= b; ++i) {
            ans[bl[b]] -= (v[i] ^ tag[bl[b]]);
            v[i] ^= 1;
            ans[bl[b]] += (v[i] ^ tag[bl[b]]);
        }
    }
    for (int i = bl[a] + 1; i <= bl[b] - 1; ++i){
        tag[i] ^= 1;
        ans[i] = sq - ans[i];
    }
}

ll query(int a, int b){
    ll res = 0;
    for (int i = a; i <= min(bl[a] * sq, b); ++i) {
        res += (v[i] ^ tag[bl[a]]);
    }
    if (bl[a] != bl[b]){
        for (int i = (bl[b] - 1) * sq + 1; i <= b; ++i) {
            res += (v[i] ^ tag[bl[b]]);
        }
    }
    for (int i = bl[a] + 1; i <= bl[b] - 1; ++i) res += ans[i];
    return res;
}

void solve(){
    cin >> n >> m;
    sq = sqrt(n);
    for (int i = 1; i <= n; ++i) bl[i] = (i - 1) / sq + 1;
    for (int i = 1; i <= m; ++i){
        int op, l, r;
        cin >> op >> l >> r;
        if (op == 0) update(l, r);
        else cout << query(l, r) << endl;
    }
}

signed main(){
    ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
    int t = 1;
    //cin >> t;
    while(t--) solve();
```

```
    return 0;
}
```

分块 II

给出一个长为 n 的数列，以及 n 个操作，操作涉及区间加法，询问区间内小于某个值 x 的元素个数。

```
#include <bits/stdc++.h>
#define endl '\n'
using namespace std;
typedef long double db;
typedef long long ll;

const ll N = 2e5 + 10;
const ll mod = 998244353;
const ll inf32 = 0x3f3f3f3f;
const ll inf64 = 5e18;

int n, blo;
int v[N], bl[N], atag[N];
vector<int> ve[2005];

void reset(int x)
{
    ve[x].clear();
    for (int i = (x - 1) * blo + 1; i <= min(x * blo, n); i++)
        ve[x].push_back(v[i]);
    sort(ve[x].begin(), ve[x].end());
}

void add(int a, int b, int c)
{
    for (int i = a; i <= min(bl[a] * blo, b); i++)
        v[i] += c;
    reset(bl[a]);
    if (bl[a] != bl[b])
    {
        for (int i = (bl[b] - 1) * blo + 1; i <= b; i++)
            v[i] += c;
        reset(bl[b]);
    }
    for (int i = bl[a] + 1; i <= bl[b] - 1; i++)
        atag[i] += c;
}

int query(int a, int b, int c)
{
    int ans = 0;
    for (int i = a; i <= min(bl[a] * blo, b); i++)
        if (v[i] + atag[bl[a]] < c)
            ans++;
    if (bl[a] != bl[b])
        for (int i = (bl[b] - 1) * blo + 1; i <= b; i++)
            if (v[i] + atag[bl[b]] < c)
                ans++;
    for (int i = bl[a] + 1; i <= bl[b] - 1; i++)
    {
        int x = c - atag[i];
        ans += lower_bound(ve[i].begin(), ve[i].end(), x) - ve[i].begin();
    }
    return ans;
}

int main()
{
    cin >> n;
    blo = sqrt(n);
    for (int i = 1; i <= n; i++)
        cin >> v[i];
    for (int i = 1; i <= n; i++)
    {
```

```
        bl[i] = (i - 1) / blo + 1;
        ve[bl[i]].push_back(v[i]);
    }
    for (int i = 1; i <= bl[n]; i++)
        sort(ve[i].begin(), ve[i].end());
    for (int i = 1; i <= n; i++)
    {
        int f, a, b, c;
        cin >> f >> a >> b >> c;
        if (f == 0)
            add(a, b, c);
        if (f == 1)
            printf("%d\n", query(a, b, c * c));
    }
    return 0;
}
```

线性 RMQ

```
/**
    线性 RMQ
 */

#include <bits/stdc++.h>
#include <limits>
#define endl '\n'
using namespace std;
typedef long long ll;

const int N = 5e4 + 10;
const int M = 2e4 + 10;
const int L = 80;
const int mod = 998244353;
const int inf32 = 0x3f3f3f3f;
const ll inf64 = 4e18;

int a[N + M];
int highbit[M];
int stmax[M][L], stmin[M][L];
int premax[M][L], premin[M][L];
int sufmax[M][L], sufmin[M][L];
int quemax[M][L], quemin[M][L];
int stackmax[L], stackmin[L];

void solve(){
    int n, q;
    cin >> n >> q;
    int B = int(log2(n)); // 块的大小
    int S = (n - 1) / B + 1; // 块的个数
    for (int b = 0; b < S; ++b) stmin[b][0] = inf32;
    for (int i = 0; i < n; ++i){
        cin >> a[i];
        stmin[i / B][0] = min(stmin[i / B][0], a[i]);
        stmax[i / B][0] = max(stmax[i / B][0], a[i]);
    }
    for (int b = S - 1; b >= 0; b--){
        for (int k = 1; b + (1 << k) - 1 < S; ++k){
            stmin[b][k] = min(stmin[b][k - 1], stmin[b + (1 << (k - 1))][k - 1]);
            stmax[b][k] = max(stmax[b][k - 1], stmax[b + (1 << (k - 1))][k - 1]);
        }
    }
    for (int b = 0; b < S; ++b){
        int be = b * B;
        premin[b][0] = premax[b][0] = a[be];
        for (int k = 1; k < B; ++k){
            premin[b][k] = min(premin[b][k - 1], a[be + k]);
            premax[b][k] = max(premax[b][k - 1], a[be + k]);
        }
        sufmin[b][B - 1] = sufmax[b][B - 1] = a[be + B - 1];
        for (int k = B - 2; k >= 0; --k){
            sufmin[b][k] = min(sufmin[b][k + 1], a[be + k]);
```

```
            sufmax[b][k] = max(sufmax[b][k + 1], a[be + k]);
        }
    }
    for (int b = 0; b < S; ++b){
        int be = b * B;
        int spmin = 0, nowmin = 0;
        int spmax = 0, nowmax = 0;
        for (int i = 0; i < B; ++i){
            while (spmin && a[be + stackmin[spmin]] > a[be + i]) nowmin ^= 1 << stackmin[spmin--];
            while (spmax && a[be + stackmax[spmax]] < a[be + i]) nowmax ^= 1 << stackmax[spmax--];
            quemin[b][i] = (nowmin ^= 1 << (stackmin[++spmin] = i));
            quemax[b][i] = (nowmax ^= 1 << (stackmax[++spmax] = i));
        }
    }
    for (int i = 2; i <= S; ++i) highbit[i] = highbit[i >> 1] + 1;

    while(q --) {
        int l, r;
        cin >> l >> r;
        l--, r--;
        int L = l / B, R = r / B;
        int li = l % B, ri = r % B;

        int mn = inf32, mx = 0;
        if(L == R) {
            mn = min(mn, a[l + __builtin_ctz(quemin[R][ri] >> li)]);
            mx = max(mx, a[l + __builtin_ctz(quemax[R][ri] >> li)]);
        }
        else {
            mn = min(mn, sufmin[L][li]);
            mn = min(mn, premin[R][ri]);
            mx = max(mx, sufmax[L][li]);
            mx = max(mx, premax[R][ri]);
            int len = R - L - 1;
            int k = highbit[len];
            if(len) {
                mn = min(mn, stmin[L + 1][k]);
                mn = min(mn, stmin[R - (1 << k)][k]);
                mx = max(mx, stmax[L + 1][k]);
                mx = max(mx, stmax[R - (1 << k)][k]);
            }
        }
        cout << mx - mn << endl;
    }
}

signed main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    //cin >> t;
    while(t--) solve();
    return 0;
}
```

区间 [l, r] 有多少数字出现正偶数次

```
// https://www.luogu.com.cn/problem/P4135
#include <bits/stdc++.h>
#define endl '\n'

using ll = long long;

constexpr int N = 1e5 + 10;
constexpr int M = 320;
constexpr int mod = 998244353;

using namespace std;

int a[N], block[N], cnt[M][N];
int L[M], f[M][M], num[N];
```

```cpp

int n, c, q, B;

void solve(){
    cin >> n >> c >> q;
    B = sqrt(n);
    for (int i = 1; i <= n; ++i) {
        cin >> a[i];
        block[i] = (i - 1) / B + 1;
        if (block[i] != block[i - 1]) L[block[i]] = i;
    }
    block[n + 1] = block[n] + 1;
    L[block[n + 1]] = n + 1;
    for (int i = 1; i <= block[n]; ++i){
        int t = 0;
        for (int j = L[i]; j <= n; ++j) {
            cnt[i][a[j]]++;
            if ((cnt[i][a[j]] & 1) && cnt[i][a[j]] > 1) t--;
            else if (cnt[i][a[j]] % 2 == 0) t++;
            if (block[j] != block[j + 1]) {
                f[i][block[j]] = t;
            }
        }
    }
    int ans = 0;
    stack<int> st;
    while(q--) {
        int l, r;
        cin >> l >> r;
        l = (l + ans) % n + 1, r = (r + ans) % n + 1;
        if (l > r) swap(l, r);
        ans = 0;
        if (block[l] == block[r]) {
            for (int i = l; i <= r; ++i) {
                num[a[i]]++;
                st.push(a[i]);
            }
            while (!st.empty()) {
                int t = st.top();
                st.pop();
                if (num[t]){
                    ans += (num[t] & 1) ^ 1;
                    num[t] = 0;
                }
            }
            cout << ans << endl;
            continue;
        }
        if (block[l] + 1 <= block[r] - 1) {
            ans = f[block[l] + 1][block[r] - 1];
        }
        for (int i = l; i < L[block[l] + 1]; ++i) {
            num[a[i]]++;
            st.push(a[i]);
        }
        for (int i = L[block[r]]; i <= r; ++i) {
            num[a[i]]++;
            st.push(a[i]);
        }
        while (!st.empty()) {
            int t = st.top();
            st.pop();
            if (!num[t]) continue;
            if (cnt[block[l] + 1][t] - cnt[block[r]][t] > 0 && (cnt[block[l] + 1][t] - cnt[block[r]][t]) % 2 == 0 &&
↪  num[t] % 2) ans--;
            else if (cnt[block[l] + 1][t] - cnt[block[r]][t] > 0 && (cnt[block[l] + 1][t] - cnt[block[r]][t]) % 2 &&
↪  num[t] % 2) ans++;
            else if (cnt[block[l] + 1][t] - cnt[block[r]][t] == 0 && num[t] % 2 == 0) ans++;
            num[t] = 0;
        }
        cout << ans << endl;
```

```
    }
}

signed main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    //cin >> t;
    while(t--) solve();
    return 0;
}
```

区间众数查询 1

```
#include <bits/stdc++.h>
#define endl '\n'

using ll = long long;

constexpr int N = 1e5 + 10;
constexpr int M = 320;
constexpr int mod = 998244353;

using namespace std;

int n, m, q, B;
int a[N], b[N], block[N], cnt[M][N], L[M];
int c[N];
pair<int, int> f[M][M];

void solve(){
    int n, q;
    cin >> n >> q;
    B = sqrt(n);
    for (int i = 1; i <= n; ++i) {
        cin >> a[i];
        b[i] = a[i];
        block[i] = (i - 1) / B + 1;
        if (block[i] != block[i - 1]) L[block[i]] = i;
    }
    block[n + 1] = block[n] + 1;
    L[block[n + 1]] = n + 1;
    sort(b + 1, b + n + 1);
    for (int i = 1; i <= n; ++i) a[i] = lower_bound(b + 1, b + n + 1, a[i]) - b;
    for (int i = 1; i <= block[n]; ++i){
        int mxcnt = 0, num = n;
        for (int j = L[i]; j <= n; ++j) {
            cnt[i][a[j]]++;
            if (cnt[i][a[j]] > mxcnt) {
                mxcnt = cnt[i][a[j]];
                num = a[j];
            }else if (cnt[i][a[j]] == mxcnt) {
                num = min(num, a[j]);
            }
            if (block[j] != block[j + 1]) {
                f[i][block[j]] = {mxcnt, num};
            }
        }
    }

    int ans = 0;
    stack<int> st;
    while (q--) {
        int l, r;
        cin >> l >> r;
        l = (l + ans - 1) % n + 1, r = (r + ans - 1) % n + 1;
        if (l > r) swap(l, r);
        ans = 0;
        int mxcnt = 0, num = n;
        if (block[l] == block[r]) {
            for (int i = l; i <= r; ++i) {
```

```
                c[a[i]]++;
                st.push(a[i]);
            }
            while (!st.empty()) {
                int t = st.top();
                st.pop();
                if (c[t]) {
                    if (c[t] > mxcnt) {
                        mxcnt = c[t];
                        num = t;
                    }else if (c[t] == mxcnt) {
                        num = min(num, t);
                    }
                    c[t] = 0;
                }
            }
            ans = b[num];
            cout << ans << endl;
            continue;
        }
        if (block[l] + 1 <= block[r] - 1) {
            mxcnt = f[block[l] + 1][block[r] - 1].first;
            num = f[block[l] + 1][block[r] - 1].second;
        }
        for (int i = l; i < L[block[l] + 1]; ++i) {
            c[a[i]]++;
            st.push(a[i]);
        }
        for (int i = L[block[r]]; i <= r; ++i) {
            c[a[i]]++;
            st.push(a[i]);
        }
        while (!st.empty()) {
            int t = st.top();
            st.pop();
            if (!c[t]) continue;
            if (c[t] + cnt[block[l] + 1][t] - cnt[block[r]][t] > mxcnt) {
                mxcnt = c[t] + cnt[block[l] + 1][t] - cnt[block[r]][t];
                num = t;
            }else if (c[t] + cnt[block[l] + 1][t] - cnt[block[r]][t] == mxcnt) {
                num = min(num, t);
            }
            c[t] = 0;
        }
        ans = b[num];
        cout << ans << endl;
    }
}

signed main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    //cin >> t;
    while(t--) solve();
    return 0;
}
```

区间众数查询 2

```
#include <bits/stdc++.h>
#define endl '\n'

using ll = long long;

constexpr int N = 5e5 + 10;
constexpr int M = 710;
constexpr int mod = 998244353;

using namespace std;

```

```cpp
int a[N], b[N], block[N], L[M], R[M], f[M][M], tot[N], pos[N];
vector<int> v[N];
int B;

void solve(){
    int n, m;
    cin >> n >> m;
    for (int i = 1; i <= n; ++i) {
        cin >> a[i];
        b[i] = a[i];
    }
    sort(b + 1, b + n + 1);
    int len = unique(b + 1, b + n + 1) - b - 1;
    for (int i = 1; i <= n; ++i) {
        a[i] = lower_bound(b + 1, b + len + 1, a[i]) - b;
    }
    for (int i = 1; i <= n; ++i) {
        v[a[i]].push_back(i);
        pos[i] = v[a[i]].size();
        pos[i]--;
    }
    int B = sqrt(n);
    for (int i = 1; i <= n; ++i) {
        block[i] = (i - 1) / B + 1;
    }
    for (int i = 1; i <= block[n]; ++i) {
        L[i] = (i - 1) * B + 1;
        R[i] = i * B;
    }
    R[block[n]] = n;
    for (int i = 1; i <= block[n]; ++i) {
        memset(tot, 0, sizeof tot);
        for (int j = i; j <= block[n]; ++j) {
            f[i][j] = f[i][j - 1];
            for (int k = L[j]; k <= R[j]; ++k) {
                f[i][j] = max(f[i][j], ++tot[a[k]]);
            }
        }
    }

    auto query = [&](int l, int r) -> int {
        int ans = 0;
        if (block[l] == block[r]) {
            for (int i = l; i <= r; ++i) tot[a[i]] = 0;
            for (int i = l; i <= r; ++i) ans = max(ans, ++tot[a[i]]);
            return ans;
        }
        ans = f[block[l] + 1][block[r] - 1];
        for (int i = l; i <= R[block[l]]; ++i) {
            auto it = pos[i];
            while (it + ans < v[a[i]].size() && v[a[i]][it + ans] <= r) {
                ans++;
            }
        }
        for (int i = L[block[r]]; i <= r; ++i) {
            auto it = pos[i];
            while (it - ans >= 0 && v[a[i]][it - ans] >= l) ++ans;
        }
        return ans;
    };
    int lstans = 0;
    while (m--) {
        int l, r;
        cin >> l >> r;
        l ^= lstans, r ^= lstans;
        if (l > r) swap(l, r);
        cout << (lstans = query(l, r)) << endl;
    }
}

signed main(){
```

```
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    //cin >> t;
    while(t--) solve();
    return 0;
}
```

Persistent Data Structure

区间第 K 小

```
//HH 的项链
#include <bits/stdc++.h>
#define endl '\n'
#define pll pair<ll, ll>
#define tll tuple<ll, ll, ll>
#define all(a) a.begin() + 1, a.end()
using namespace std;
using i64 = long long;
const i64 N = 1e6 + 10;
const i64 mod = 998244353;
const int inf32 = 1e9;
const i64 inf64 = 5e18;

int lst[N];
int root[N];
struct PersistentTree{
    int ls[N << 5], rs[N << 5], sum[N << 5], tot = 0;
    //单点修改
    inline void update(int & rt, int old, int l, int r, int p, int k){
        rt = ++tot;
        ls[rt] = ls[old], rs[rt] = rs[old], sum[rt] = sum[old] + k;
        if (l == r) return;
        int mid = l + r >> 1;
        if (p <= mid) update(ls[rt], ls[old], l, mid, p, k);
        else update(rs[rt], rs[old], mid + 1, r, p, k);
    }
    //区间查询
    inline int query(int rt, int l, int r, int p){
        if (l == r) return sum[rt];
        int mid = l + r >> 1;
        if (p <= mid) return query(ls[rt], l, mid, p) + sum[rs[rt]];
        else return query(rs[rt], mid + 1, r, p);
    }
}T;

void solve(){
    int n;
    cin >> n;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; ++i) cin >> a[i];
    for (int i = 1, tmp; i <= n; ++i){
        if (!lst[a[i]]) T.update(root[i], root[i - 1], 1, n, i, 1);
        else{
            T.update(tmp, root[i - 1], 1, n, lst[a[i]], -1);
            T.update(root[i], tmp, 1, n, i, 1);
        }
        lst[a[i]] = i;
    }
    int q;
    cin >> q;
    while (q--){
        int l, r;
        cin >> l >> r;
        cout << T.query(root[r], 1, n, l) << endl;
    }

}

int main(){
```

```
    ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
    int t = 1;
    //cin >> t;
    while(t--){
        solve();
    }
    return 0;
}
```

最大异或和

```
#include <bits/stdc++.h>
#define endl '\n'
using namespace std;
typedef long double db;
typedef long long ll;

const ll N = 6e5 + 10;
const ll mod = 998244353;
const ll inf32 = 0x3f3f3f3f;
const ll inf64 = 5e18;

int a[N], s[N];
struct Trie
{
    int cnt, rt[N], ch[N * 33][2], val[N * 33];
    void insert(int o, int lst, int v)
    {
        for (int i = 28; i >= 0; i--)
        {
            val[o] = val[lst] + 1; // 在原版本的基础上更新
            if ((v & (1 << i)) == 0)
            {
                if (!ch[o][0])
                    ch[o][0] = ++cnt;
                ch[o][1] = ch[lst][1];
                o = ch[o][0];
                lst = ch[lst][0];
            }
            else
            {
                if (!ch[o][1])
                    ch[o][1] = ++cnt;
                ch[o][0] = ch[lst][0];
                o = ch[o][1];
                lst = ch[lst][1];
            }
        }
        val[o] = val[lst] + 1;
    }

    int query(int o1, int o2, int v)
    {
        int ret = 0;
        for (int i = 28; i >= 0; i--)
        {
            int t = ((v & (1 << i)) ? 1 : 0);
            if (val[ch[o1][!t]] - val[ch[o2][!t]])
                ret += (1 << i), o1 = ch[o1][!t],
                                 o2 = ch[o2][!t]; // 尽量向不同的地方跳
            else
                o1 = ch[o1][t], o2 = ch[o2][t];
        }
        return ret;
    }
} st;

void solve()
{
    int n, q;
    cin >> n >> q;
```

```
    for (int i = 1; i <= n; ++i) cin >> a[i], s[i] = s[i - 1] ^ a[i];
    for (int i = 1; i <= n; ++i){
        st.rt[i] = ++st.cnt;
        st.insert(st.rt[i], st.rt[i - 1], s[i]);
    }
    while (q--){
        char op;
        cin >> op;
        if (op == 'A'){
            ++n;
            cin >> a[n];
            s[n] = s[n - 1] ^ a[n];
            st.rt[n] = ++st.cnt;
            st.insert(st.rt[n], st.rt[n - 1], s[n]);
        }else{
            int l, r, x;
            cin >> l >> r >> x;
            l--, r--;
            if (l == 0) cout << max(s[n] ^ x, st.query(st.rt[r], st.rt[0], s[n] ^ x)) << endl;
            else cout << st.query(st.rt[r], st.rt[l - 1], s[n] ^ x) << endl;
        }
    }
}

signed main()
{
    ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

区间 MEX

```
#include <bits/stdc++.h>
#define endl '\n'
#define pll pair<i64, i64>
#define tll tuple<i64, i64, i64>
#define all(a) a.begin() + 1, a.end()
using namespace std;
using db = long double;
using i64 = long long;
const i64 N = 3e5 + 10;
const i64 mod = 998244353;
const i64 inf32 = 1e9;
const i64 inf64 = 5e18;

int rt[N];
struct PersistentTree{
    int ls[N << 5], rs[N << 5], mn[N << 5], sum[N << 5], tot = 0;
    inline void build(int & rt, int l, int r){
        rt = ++tot;
        if (l == r) return;
        int mid = l + r >> 1;
        build(ls[rt], l, mid);
        build(rs[rt], mid + 1, r);
    }
    inline void update(int & rt, int old, int l, int r, int p, int k){
        rt = ++tot;
        ls[rt] = ls[old], rs[rt] = rs[old], mn[rt] = mn[old], sum[rt] = sum[old];
        if (l == r){
            mn[rt] = k;
            sum[rt]++;
            return;
        }
        int mid = l + r >> 1;
        if (p <= mid) update(ls[rt], ls[old], l, mid, p, k);
        else update(rs[rt], rs[old], mid + 1, r, p, k);
        mn[rt] = min(mn[ls[rt]], mn[rs[rt]]);
```

```
        sum[rt] = sum[ls[rt]] + sum[rs[rt]];
    }
    inline int query(int rt, int old, int p){
        if (mn[rt] >= p) return sum[rt] - sum[old];
        if (ls[rt] == 0) return 0;
        int res = 0;
        if (mn[ls[rt]] >= p) res = sum[ls[rt]] - sum[ls[old]] + query(rs[rt], rs[old], p);
        else res = query(ls[rt], ls[old], p);
        return res;
    }
}T;

void solve(){
    int n, q;
    cin >> n;
    T.build(rt[0], 1, n);
    vector<int> a(n + 1);
    for (int i = 1; i <= n; ++i) {
        cin >> a[i];
        if (a[i] <= n) T.update(rt[i], rt[i - 1], 1, n, a[i], i);
        else rt[i] = rt[i - 1];
    }
    cin >> q;
    while (q--){
        int l, r;
        cin >> l >> r;
        cout << (r - l + 1) - T.query(rt[r], rt[l - 1], l) << endl;
    }
}

int main(){
    ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
    int t = 1;
    //cin >> t;
    while(t--) solve();
    return 0;
}
```

区间多少数出现多次

```
//采花 60Pts
#include <bits/stdc++.h>
#define endl '\n'
#define pll pair<ll, ll>
#define tll tuple<ll, ll, ll>
#define all(a) a.begin() + 1, a.end()
using namespace std;
using i64 = long long;
using db = long double;
const i64 N = 2e6 + 10;
const i64 mod = 998244353;
const i64 inf32 = 1e9;
const i64 inf64 = 5e18;

int lst[N], pre[N];
int root[N];
struct PersistentTree{
    int ls[N << 5], rs[N << 5], sum[N << 5], tot = 0;
    //单点修改
    inline void update(int & rt, int old, int l, int r, int p, int k){
        rt = ++tot;
        ls[rt] = ls[old], rs[rt] = rs[old], sum[rt] = sum[old] + k;
        if (l == r) return;
        int mid = l + r >> 1;
        if (p <= mid) update(ls[rt], ls[old], l, mid, p, k);
        else update(rs[rt], rs[old], mid + 1, r, p, k);
    }
    //区间查询
    inline int query(int rt, int l, int r, int p){
        if (l == r) return sum[rt];
        int mid = l + r >> 1;
```

```
        if (p <= mid) return query(ls[rt], l, mid, p) + sum[rs[rt]];
        else return query(rs[rt], mid + 1, r, p);
    }
}T;

void solve(){
    int n, c, q;
    cin >> n >> c >> q;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; ++i) cin >> a[i];
    for (int i = 1; i <= n; ++i){
        root[i] = root[i - 1];//防止第二个 if 无继承
        if (lst[a[i]]) T.update(root[i], root[i - 1], 1, n, lst[a[i]], 1);
        if (pre[lst[a[i]]]) T.update(root[i], root[i], 1, n, pre[lst[a[i]]], -1);
        pre[i] = lst[a[i]];
        lst[a[i]] = i;
    }
    for (int i = 1; i <= q; ++i){
        int l, r;
        cin >> l >> r;
        cout << T.query(root[r], 1, n, l) << endl;
    }
}

int main(){
    ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
    int t = 1;
    //cin >> t;
    while(t--) solve();
    return 0;
}
```

区间种类数

```
//HH 的项链
#include <bits/stdc++.h>
#define endl '\n'
#define pll pair<ll, ll>
#define tll tuple<ll, ll, ll>
#define all(a) a.begin() + 1, a.end()
using namespace std;
using i64 = long long;
const i64 N = 1e6 + 10;
const i64 mod = 998244353;
const int inf32 = 1e9;
const i64 inf64 = 5e18;

int lst[N];
int root[N];
struct PersistentTree{
    int ls[N << 5], rs[N << 5], sum[N << 5], tot = 0;
    //单点修改
    inline void update(int & rt, int old, int l, int r, int p, int k){
        rt = ++tot;
        ls[rt] = ls[old], rs[rt] = rs[old], sum[rt] = sum[old] + k;
        if (l == r) return;
        int mid = l + r >> 1;
        if (p <= mid) update(ls[rt], ls[old], l, mid, p, k);
        else update(rs[rt], rs[old], mid + 1, r, p, k);
    }
    //区间查询
    inline int query(int rt, int l, int r, int p){
        if (l == r) return sum[rt];
        int mid = l + r >> 1;
        if (p <= mid) return query(ls[rt], l, mid, p) + sum[rs[rt]];
        else return query(rs[rt], mid + 1, r, p);
    }
}T;

void solve(){
    int n;
```

```
    cin >> n;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; ++i) cin >> a[i];
    for (int i = 1, tmp; i <= n; ++i){
        if (!lst[a[i]]) T.update(root[i], root[i - 1], 1, n, i, 1);
        else{
            T.update(tmp, root[i - 1], 1, n, lst[a[i]], -1);
            T.update(root[i], tmp, 1, n, i, 1);
        }
        lst[a[i]] = i;
    }
    int q;
    cin >> q;
    while (q--){
        int l, r;
        cin >> l >> r;
        cout << T.query(root[r], 1, n, l) << endl;
    }

}

int main(){
    ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
    int t = 1;
    //cin >> t;
    while(t--){
        solve();
    }
    return 0;
}
```

可持久化并查集

```
#include <bits/stdc++.h>
#define endl '\n'
#define pll pair<i64, i64>
#define tll tuple<i64, i64, i64>
#define all(a) a.begin() + 1, a.end()
using namespace std;
using db = long double;
using i64 = long long;
const i64 N = 2e5 + 10;
const i64 mod = 998244353;
const i64 inf32 = 1e9;
const i64 inf64 = 5e18;

int n, m;
int rootfa[N], rootdep[N];
struct PersistentDSU{
    int ls[N << 5], rs[N << 5], val[N << 5], tot = 0, idx = 0;
    inline void build(int & rt, int l, int r){
        rt = ++tot;
        if (l == r){
            val[rt] = ++idx;
            return;
        }
        int mid = (l + r) >> 1;
        build(ls[rt], l, mid);
        build(rs[rt], mid + 1, r);
    }
    inline void update(int & rt, int old, int l, int r, int p, int k){
        rt = ++tot;
        ls[rt] = ls[old], rs[rt] = rs[old], val[rt] = val[old];
        if (l == r){
            val[rt] = k;
            return;
        }
        int mid = (l + r) >> 1;
        if (p <= mid) update(ls[rt], ls[old], l, mid, p, k);
        else update(rs[rt], rs[old], mid + 1, r, p, k);
    }
```

```
39      inline int query(int rt, int l, int r, int p){
40          if (l == r) return val[rt];
41          int mid = (l + r) >> 1;
42          if (p <= mid) return query(ls[rt], l, mid, p);
43          else return query(rs[rt], mid + 1, r, p);
44      }
45      //找到第 ver 个版本中 x 的父亲
46      inline int find(int ver, int x){
47          //去 ver 版本线段树 rootfa[ver] 中查询 x 上的值
48          int fx = query(rootfa[ver], 1, n, x);
49          if (x != fx) x = find(ver, fx); //不能进行路径压缩
50          return x;
51      }
52      //在 ver 个版本中合并集合 x 和集合 y
53      inline void merge(int ver, int x, int y){
54          //千万要注意 rootdep 和 rootfa!!!
55          x = find(ver - 1, x);
56          y = find(ver - 1, y);
57          if (x == y){
58              rootfa[ver] = rootfa[ver - 1];
59              rootdep[ver] = rootdep[ver - 1];
60          }else{
61              int depx = query(rootdep[ver - 1], 1, n, x);
62              int depy = query(rootdep[ver - 1], 1, n, y);
63              if (depx < depy){
64                  //fa[x] = y
65                  update(rootfa[ver], rootfa[ver - 1], 1, n, x, y);
66                  rootdep[ver] = rootdep[ver - 1];
67              }else if (depy < depx){
68                  //fa[y] = x
69                  update(rootfa[ver], rootfa[ver - 1], 1, n, y, x);
70                  rootdep[ver] = rootdep[ver - 1];
71              }else{
72                  //fa[x] = y
73                  update(rootfa[ver], rootfa[ver - 1], 1, n, x, y);
74                  update(rootdep[ver], rootdep[ver - 1], 1, n, y, depy + 1);
75              }
76          }
77      }
78  }D;
79
80  void solve(){
81      cin >> n >> m;
82      D.build(rootfa[0], 1, n);
83      for (int i = 1; i <= m; ++i){
84          int op, a, b;
85          cin >> op;
86          if (op == 1){ //合并
87              cin >> a >> b;
88              D.merge(i, a, b);
89          }else if (op == 2){ //回滚
90              int k;
91              cin >> k;
92              rootfa[i] = rootfa[k];
93              rootdep[i] = rootdep[k];
94          }else if (op == 3){ //查询
95              cin >> a >> b;
96              rootfa[i] = rootfa[i - 1];
97              rootdep[i] = rootdep[i - 1];
98              a = D.find(i, a);
99              b = D.find(i, b);
100             if (a == b) cout << 1 << endl;
101             else cout << 0 << endl;
102         }
103     }
104 }
105
106 int main(){
107     ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
108     int t = 1;
109     //cin >> t;
```

```
    while(t--) solve();
    return 0;
}
```

子序列自动机

```
#include <bits/stdc++.h>
#define endl '\n'
#define pll pair<i64, i64>
#define tll tuple<i64, i64, i64>
#define all(a) a.begin() + 1, a.end()
using namespace std;
using i64 = long long;
using db = long double;
const i64 N = 1e5 + 10;
const i64 mod = 998244353;
const i64 inf32 = 1e9;
const i64 inf64 = 5e18;

int typ, n, q, m;
int root[N];
struct PersistentTree{
    int ls[N << 5], rs[N << 5], nxt[N << 5], tot = 0;
    //单点修改
    inline void update(int & rt, int old, int l, int r, int p, int k){
        rt = ++tot;
        ls[rt] = ls[old], rs[rt] = rs[old], nxt[rt] = nxt[old];
        if (l == r){
            nxt[rt] = k;
            return;
        }
        int mid = l + r >> 1;
        if (p <= mid) update(ls[rt], ls[old], l, mid, p, k);
        else update(rs[rt], rs[old], mid + 1, r, p, k);
    }
    //单点查询
    inline int query(int rt, int l, int r, int p){
        if (l == r) return nxt[rt];
        int mid = l + r >> 1;
        if (p <= mid) return query(ls[rt], l, mid, p);
        else return query(rs[rt], mid + 1, r, p);
    }
    //建树
    inline void build(vector<int> a){
        int n = a.size() - 1;
        for (int i = n; i >= 1; --i){
            update(root[i], root[i + 1], 1, m, a[i], i + 1);
        }
    }
}T;

void solve(){
    cin >> typ >> n >> q >> m;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; ++i) cin >> a[i];
    T.build(a);
    while (q--){
        int k;
        cin >> k;
        bool ok = true;
        int rt = 1;
        while (k--){
            int p;
            cin >> p;
            int to = T.query(root[rt], 1, m, p);
            if (!to) ok = false;
            if (to) rt = to;
        }
        if (ok) cout << "Yes" << endl;
        else cout << "No" << endl;
    }
```

```
}

int main(){
    ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
    int t = 1;
    //cin >> t;
    while(t--) solve();
    return 0;
}
```

Tree Dp

树的平衡点

```
function<void(int, int)> dfs = [&](int u, int fa){
    sz[u] = 1;
    for (auto v : G[u]){
        if (v == fa) continue;
        dfs(v, u);
        sz[u] += sz[v];
        int mx = max(sz[u], n - sz[u]);
        if (mx <= mn){
            mn = mx;
            id = min(u, id);
        }
    }
};
```

树的最小点覆盖 (最少的点覆盖所有边)

```
void dp(int u) {
    bool fg = 0;
    for(int i = h[u]; ~i; i = nex[i]) {
        int j = v[i];
        fg = 1;
        dp(j);
        f[u][0] += f[j][1];
        f[u][1] += min(f[j][0], f[j][1]);
    }
    f[u][1] += 1;
    if(!fg) {
        f[u][0] = 0; f[u][1] = 1;
    }
}
```

树的最小支配集 (最少的点覆盖所有点)

f[i][0] 选 i 且 i 及 i 的子树都被覆盖了 f[i][1] 不选 i 且 i 被其儿子覆盖 f[i][2] 不选 i 且 i 被其父亲覆盖 (儿子可选可不选)

```
void dfs(int u, int fa){
    f[u][0] = 1; f[u][1] = f[u][2] = 0;
    bool ok = false;
    int tmp = inf32;
    for (auto v : G[u]){
        if (v == fa) continue;
        dfs(v, u);
        f[u][2] += min(f[v][1], f[v][0]);
        f[u][0] += min({f[v][0], f[v][1], f[v][2]});
        if (f[v][0] <= f[v][1]){
            ok = true;
            f[u][1] += f[v][0];
        }else{
            f[u][1] += f[v][1];
            tmp = min(tmp, f[v][0] - f[v][1]);
        }
    }
    if (!ok) f[u][1] += tmp;
}
```

树的最大独立集 (选定的任意两点之间无边)

```
function<void(int, int)> dfs = [&](int u, int fa)
{
    dp[u][1] = h[u];
    for (auto v : G[u])
    {
        if (v == fa)
            continue;
        dfs(v, u);
        dp[u][0] += max(dp[v][1], dp[v][0]);
        dp[u][1] += dp[v][0];
    }
};
```

基环树最大独立集

```
#include <bits/stdc++.h>
#define endl '\n'

using ll = long long;
using db = long double;

constexpr int N = 2e5 + 10;
constexpr int mod = 998244353;

using namespace std;

void solve(){
    int n;
    cin >> n;
    vector<int> f(n + 1);
    auto find = [&](auto self, int x) -> int {
        if (x == f[x]) return x;
        return f[x] = self(self, f[x]);
    };
    vector<int> p(n + 1);
    int S, T;
    for (int i = 1; i <= n; ++i) cin >> p[i], f[i] = i;
    vector<vector<int>> G(n + 1);
    for (int i = 1; i <= n; ++i) {
        int u, v;
        cin >> u >> v;
        u++, v++;
        int fu = find(find, u);
        int fv = find(find, v);
        if (fu == fv) {
            S = u, T = v;
            continue;
        }
        G[u].push_back(v);
        G[v].push_back(u);
        f[fu] = fv;
    }
    double k;
    cin >> k;
    ll ans = 0;

    vector dp(n + 1, vector<ll>(2));
    auto dfs = [&](auto self, int u, int fa) -> void {
        dp[u][1] = p[u];
        dp[u][0] = 0;
        //cerr << u << endl;
        for (auto v : G[u]) {
            if (v == fa) continue;
            self(self, v, u);
            dp[u][1] += dp[v][0];
            dp[u][0] += max(dp[v][1], dp[v][0]);
        }
    };

```

```
    dfs(dfs, S, 0);
    ans = max(ans, dp[S][0]);
    dfs(dfs, T, 0);
    ans = max(ans, dp[T][0]);
    cout << fixed << setprecision(1) << 1.0 * ans * k << endl;
}

signed main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    //cin >> t;
    while(t--) solve();
    return 0;
}
```

树上背包 (最多不超过 m 条边)

```
#include <bits/stdc++.h>
#include <cstring>
#include <vector>
#define endl '\n'
using namespace std;
typedef long double db;
typedef long long ll;

const ll N = 1e2 + 10;
const ll mod = 998244353;
const ll inf32 = 0x3f3f3f3f;
const ll inf64 = 5e18;

vector<pair<int, int>> G[N];
int n, m, dp[N][N], sz[N], tmp[N];

void dfs(int u, int fa){
    sz[u] = 1;
    for (auto [v, w] : G[u]){
        if (v == fa) continue;
        dfs(v, u);
        memset(tmp, 0, sizeof tmp);
        for (int i = sz[u] - 1; i >= 0; --i){
            for (int j = sz[v] - 1; j >= 0; --j){
                int nxt = i + j + 1;
                if (nxt <= m){
                    tmp[nxt] = max(tmp[nxt], dp[u][i] + dp[v][j] + w);
                }
            }
        }
        sz[u] += sz[v];
        for (int i = 0; i <= sz[u] - 1; ++i)
            dp[u][i] = max(dp[u][i], tmp[i]);
    }
}

void solve(){
    cin >> n >> m;
    for (int i = 1; i < n; ++i){
        int u, v, w;
        cin >> u >> v >> w;
        G[u].emplace_back(v, w);
        G[v].emplace_back(u, w);
    }
    dfs(1, 0);
    int ans = 0;
    for (int i = 1; i <= m; ++i){
        ans = max(ans, dp[1][i]);
    }
    cout << ans << endl;
}

signed main(){
```

```
    ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
    int t = 1;
    //cin >> t;
    while(t--) solve();
    return 0;
}
```

2022CCPC-A(树上背包)

爱丽丝想在公园里找到她丢失的猫。

爱丽丝想在公园里找到她丢失的猫。

公园是一棵有根的树，由 n 个顶点组成。顶点的编号从 1 到 n ，根顶点为 1 。

爱丽丝现在位于顶点 1 。她知道猫已经从顶点 1 跑到了树的某片叶子上，而且没有顶点被访问超过一次。叶子是没有子顶点的顶点。

每个顶点上都有一个监视器。顶点 i 上的监视器可以观察到猫是否访问了顶点 i ，以及猫去了哪个顶点 (如果顶点 i 不是叶子)。爱丽丝需要花费 a_i 秒来检查 i_{th} 监视器的数据。

爱丽丝也可以自己搜索一些叶子。搜索 i 个叶子需要 t_i 秒。请注意，i 是顶点的计数，而不是顶点的标签。

帮助爱丽丝确定要检查哪些监视器和搜索哪些树叶，以便唯一确定猫的位置，并尽可能减少所需的总时间。请注意，要检查的监视器和要搜索的树叶应该在一开始就决定好，之后不得更改。

找出最短的时间。

```
#include <bits/stdc++.h>
#include <vector>
#define endl '\n'
using namespace std;
typedef long double db;
typedef long long ll;

const ll N = 2e3 + 10;
const ll mod = 998244353;
const ll inf32 = 0x3f3f3f3f;
const ll inf64 = 5e18;

vector<int> G[N];
ll sz[N], a[N], t[N];
ll dp[N][N][2], g[N][N];
ll tmp1[N][2], tmp2[N];


void dfs(int u, int fa){
    if (G[u].size() == 1 && fa){
        sz[u] = 1;
        dp[u][0][0] = dp[u][1][1] = 0;
        dp[u][1][0] = a[u];
        return;
    }
    dp[u][0][0] = 0;
    g[u][0] = 0;
    for (auto v : G[u]){
        if (v == fa) continue;
        dfs(v, u);
        memset(tmp1, 0x3f, sizeof tmp1);
        memset(tmp2, 0x3f, sizeof tmp2);
        for (int i = sz[u]; i >= 0; --i){
            for (int j = sz[v]; j >= 0; --j){
                int nxt = i + j;
                tmp1[nxt][0] = min(tmp1[nxt][0], dp[u][i][0] + dp[v][j][0]);
                tmp1[nxt][1] = min({tmp1[nxt][1], dp[u][i][0] + dp[v][j][1], dp[u][i][1] + dp[v][j][0]});
                tmp2[nxt] = min(tmp2[nxt], g[u][i] + min(dp[v][j][0], dp[v][j][1]));
            }
        }
        sz[u] += sz[v];
        memcpy(dp[u], tmp1, sizeof tmp1);
        memcpy(g[u], tmp2, sizeof tmp2);
    }
```

```
    for (int i = 1; i <= sz[u]; ++i)
        dp[u][i][0] = min(dp[u][i][0], g[u][i] + a[u]);
}

void solve(){
    int n;
    cin >> n;
    for (int i = 1; i <= n; ++i) cin >> a[i];
    for (int i = 1; i <= n; ++i) cin >> t[i];
    for (int i = 1; i <= n; ++i) sz[i] = 0;
    for (int i = 1; i <= n; ++i) G[i].clear();
    for (int i = 1, u, v; i < n; ++i){
        cin >> u >> v;
        G[u].push_back(v);
        G[v].push_back(u);
    }
    memset(dp, 0x3f, sizeof dp);
    memset(g, 0x3f, sizeof g);
    if (n == 1) {
        cout << 0 << endl;
        return;
    }
    ll ans = inf64;
    dfs(1, 0);
    for (int i = 1; i <= sz[1]; ++i){
        ans = min(ans, t[sz[1] - i] + min(dp[1][i][0], dp[1][i][1]));
    }
    cout << ans << endl;
}

signed main(){
    ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
    int t = 1;
    cin >> t;
    while(t--) solve();
    return 0;
}
```

```
#include <bits/stdc++.h>
#include <vector>
#define endl '\n'
using namespace std;
typedef long double db;
typedef long long ll;

const ll N = 2e3 + 10;
const ll mod = 998244353;
const ll inf32 = 0x3f3f3f3f;
const ll inf64 = 5e18;

vector<int> G[N];
ll sz[N], a[N], t[N];
ll dp[N][N][2], g[N][N];

void dfs(int u, int fa){
    if (G[u].size() == 1 && fa){
        sz[u] = 1;
        dp[u][0][0] = dp[u][1][1] = 0;
        dp[u][1][0] = a[u];
        return;
    }
    dp[u][0][0] = 0;
    g[u][0] = 0;
    for (auto v : G[u]){
        if (v == fa) continue;
        dfs(v, u);
        for (int i = sz[u]; i >= 0; --i){
            for (int j = sz[v]; j >= 0; --j){
                int nxt = i + j;
                dp[u][nxt][0] = min(dp[u][nxt][0], dp[u][i][0] + dp[v][j][0]);
                dp[u][nxt][1] = min({dp[u][nxt][1], dp[u][i][0] + dp[v][j][1], dp[u][i][1] + dp[v][j][0]});
                g[u][nxt] = min(g[u][nxt], g[u][i] + min(dp[v][j][0], dp[v][j][1]));
```

```
            }
        }
        sz[u] += sz[v];
    }
    for (int i = 1; i <= sz[u]; ++i)
        dp[u][i][0] = min(dp[u][i][0], g[u][i] + a[u]);
}

void solve(){
    int n;
    cin >> n;
    for (int i = 1; i <= n; ++i) cin >> a[i];
    for (int i = 1; i <= n; ++i) cin >> t[i];
    for (int i = 1; i <= n; ++i) sz[i] = 0;
    for (int i = 1; i <= n; ++i) G[i].clear();
    for (int i = 1, u, v; i < n; ++i){
        cin >> u >> v;
        G[u].push_back(v);
        G[v].push_back(u);
    }
    memset(dp, 0x3f, sizeof dp);
    memset(g, 0x3f, sizeof g);
    if (n == 1) {
        cout << 0 << endl;
        return;
    }
    ll ans = inf64;
    dfs(1, 0);
    for (int i = 1; i <= sz[1]; ++i){
        ans = min(ans, t[sz[1] - i] + min(dp[1][i][0], dp[1][i][1]));
    }
    cout << ans << endl;
}

signed main(){
    ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
    int t = 1;
    cin >> t;
    while(t--) solve();
    return 0;
}
```

树联通点集 (换根)

```
#include <bits/stdc++.h>
#include <vector>
#define endl '\n'
using namespace std;
typedef long double db;
typedef long long ll;

const ll N = 1e6 + 10;
const ll mod = 1e9 + 7;
const ll inf32 = 0x3f3f3f3f;
const ll inf64 = 5e18;

int n;
ll f[N], ans[N];
vector<int> G[N];

ll qmi(ll a, ll b = mod - 2){
    ll res = 1;
    while (b){
        if (b & 1) res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}

void dfs1(int u, int fa){
```

```
    f[u] = 1;
    for (auto v : G[u]){
        if (v == fa) continue;
        dfs1(v, u);
        f[u] = f[u] * (f[v] + 1) % mod;
    }
}

void dfs2(int u, int fa){
    if (!fa) ans[u] = f[u];
    else if ((f[u] + 1) % mod == 0){
        dfs1(u, u);
        ans[u] = f[u];
    }else ans[u] = (ans[fa] % mod * qmi(f[u] + 1) + 1) % mod * f[u] % mod;
    for (auto v : G[u]){
        if (v == fa) continue;
        dfs2(v, u);
    }
}

void solve(){
    cin >> n;
    for (int i = 1; i < n; ++i){
        int u, v;
        cin >> u >> v;
        G[u].push_back(v);
        G[v].push_back(u);
    }
    dfs1(1, 1);
    dfs2(1, 0);
    for (int i = 1; i <= n; ++i) cout << ans[i] << endl;
}

signed main(){
    ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
    int t = 1;
    //cin >> t;
    while(t--) solve();
    return 0;
}
```

树划分联通块大小小于等于 k(树上背包)

另一种背包写法

```
#include <bits/stdc++.h>
#include <cstring>
#include <vector>
#define endl '\n'
using namespace std;
typedef long double db;
typedef long long ll;

const ll N = 2e3 + 10;
const ll mod = 998244353;
const ll inf32 = 0x3f3f3f3f;
const ll inf64 = 5e18;

int n, k, sz[N];
vector<int> G[N];
ll dp[N][N], ans = 0; //dp[i][j] i 所在的联通块大小为 j

void dfs(int u, int fa){
    sz[u] = dp[u][1] = 1;
    for (auto v : G[u]){
        if (v == fa) continue;
        dfs(v, u);
        ll sum = 0;
        for (int i = 1; i <= sz[v] && i <= k; ++i) sum = (sum + dp[v][i]) % mod;
        for (int i = min(sz[u], k); i >= 1; --i){
            for (int j = min(sz[v], k); j >= 1; --j){
```

```
                int nxt = i + j;
                if (nxt <= k){
                    dp[u][nxt] = (dp[u][nxt] + dp[u][i] * dp[v][j]) % mod;
                }
            }
            dp[u][i] = (dp[u][i] * sum) % mod;
        }
        sz[u] += sz[v];
    }
}

void solve(){
    cin >> n >> k;
    for (int i = 1; i < n; ++i){
        int u, v;
        cin >> u >> v;
        G[u].push_back(v);
        G[v].push_back(u);
    }
    dfs(1, 0);
    for (int i = 1; i <= k && i <= sz[1]; ++i) ans = (ans + dp[1][i]) % mod;
    cout << ans << endl;
}

signed main(){
    ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
    int t = 1;
    //cin >> t;
    while(t--) solve();
    return 0;
}
```

树上子链 (点权和最大)

给定一棵树 T，树 T 上每个点都有一个权值。定义一颗树的子链的大小为：这个子链上所有结点的权值和。请在树 T 中找出一条最大的子链并输出。

```
#include <bits/stdc++.h>
#define endl '\n'
#define pll pair<ll, ll>
#define tll tuple<ll, ll, ll>
#define ios ios::sync_with_stdio(false), cin.tie(0), cout.tie(0)
using namespace std;
typedef long long ll;
const ll maxn = 2e5 + 10;
const ll mod = 998244353;
vector<ll> G[maxn];
ll w[maxn], dis[maxn], ans = -1e18;

void solve(){
    int n;
    cin >> n;
    for (int i = 1; i <= n; ++i){
        cin >> w[i];
    }
    for (int i = 1; i <= n - 1; ++i){
        int u, v;
        cin >> u >> v;
        G[u].push_back(v);
        G[v].push_back(u);
    }
    function<void(int, int)> dfs = [&](int u, int fa){
        ll tmp = 0, mx1 = 0, mx2 = 0;
        for (auto v: G[u]){
            if (v == fa) continue;
            dfs(v, u);
            tmp = dis[v];
            if (tmp >= mx1){
                mx2 = mx1;
                mx1 = tmp;
            }else if (tmp >= mx2){
```

```
                mx2 = tmp;
            }
        }
        ans = max(ans, mx1 + mx2 + w[u]);
        dis[u] = mx1 + w[u];
    };
    dfs(1, 0);
    cout << ans << endl;
}

int main(){
    ios;
    int t = 1;
    //cin >> t;
    while(t--){
        solve();
    }
    return 0;
}
```

Nim Cheater(树剖优化 dp)

```
#include <bits/stdc++.h>
#define endl '\n'

using ll = long long;

constexpr int N = 4e4 + 10;
constexpr int mod = 998244353;

using namespace std;

int dp[16390];
int f[32][16390];

void solve(){
    int n;
    cin >> n;
    vector<int> a(n + 1), b(n + 1), fa(n + 1), ask(n + 1), ans(n + 1);
    vector<vector<int>> G(n + 1);
    int lst = 0, rt = 0;
    for (int i = 1; i <= n; ++i) {
        string s;
        cin >> s;
        if (s[0] == 'A'){
            cin >> a[i] >> b[i];
            if (rt == 0) rt = i, lst = i;
            else{
                G[lst].push_back(i);
                fa[i] = lst;
                lst = i;
            }
            ask[i] = i;
        }else{
            lst = fa[lst];
            ask[i] = lst;
        }
    }
    vector<int> sz(n + 1), son(n + 1);

    function<void(int)> dfs1 = [&](int u) {
        sz[u] = 1;
        for (auto v : G[u]){
            dfs1(v);
            sz[u] += sz[v];
            if (sz[v] > sz[son[u]]) son[u] = v;
        }
    };

    dfs1(rt);

```

```
    function<void(int, int, int)> dfs2 = [&](int u, int num, int sum){
        memcpy(dp, f[num], sizeof(dp));
        for (int i = 0; i < 16384; ++i){
            dp[i] = max(dp[i], f[num][i ^ a[u]] + b[u]);
        }
        ans[u] = sum - dp[0];
        memcpy(f[num], dp, sizeof(f[num]));
        for (auto v : G[u]){
            if (v == son[u]) continue;
            memcpy(f[num + 1], f[num], sizeof(f[num]));
            dfs2(v, num + 1, sum + b[v]);
        }
        if (son[u])
            dfs2(son[u], num, sum + b[son[u]]);
    };

    memset(f, -0x3f, sizeof(f));
    f[0][0] = 0;
    dfs2(rt, 0, b[rt]);
    for (int i = 1; i <= n; ++i) cout << ans[ask[i]] << endl;
}

signed main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    //cin >> t;
    while(t--) solve();
    return 0;
}
```

树状数组

二维偏序

```
#include <bits/stdc++.h>
using namespace std;

constexpr int MAXN = 1e7 + 5;
int sum[MAXN], ans[MAXN];
vector<pair<int, int>> vec;
vector<tuple<int, int, int, int>> q;

inline int lowbit(int x) { return x & (-x); }

void add(int pos, int x)
{
    for (; pos < MAXN; pos += lowbit(pos))
        sum[pos] += x;
}

int query_presum(int pos)
{
    int ans = 0;
    for (; pos > 0; pos -= lowbit(pos))
        ans += sum[pos];
    return ans;
}

int main()
{
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    int n, m, x1, x2, y1, y2;
    cin >> n >> m;
    for (int i = 0; i < n; i++)
    {
        cin >> x1 >> y1, ++x1, ++y1;
        vec.emplace_back(x1, y1);
    }
    sort(vec.begin(), vec.end());
    for (int i = 0; i < m; i++)
```

```
    {
        cin >> x1 >> y1 >> x2 >> y2;
        ++x1, ++y1, ++x2, ++y2;
        q.emplace_back(x1 - 1, y1 - 1, 1, i);
        q.emplace_back(x1 - 1, y2, -1, i);
        q.emplace_back(x2, y1 - 1, -1, i);
        q.emplace_back(x2, y2, 1, i);
    }
    sort(q.begin(), q.end());
    int cur = 0;
    for (auto [x, y, c, id] : q)
    {
        while (cur < n && vec[cur].first <= x)
            add(vec[cur].second, 1), ++cur;
        ans[id] += c * query_presum(y);
    }
    for (int i = 0; i < m; i++)
        cout << ans[i] << "\n";

    return 0;
}
```

数位 DP

XOR SUM

路易斯喜欢整数。他将非负整数序列 $A = [a_1, a_2, \dots, a_k]$ 的值定义为以下公式（$\oplus$ 表示位排他性-或）：

$$\sum_{i=1}^{k} \sum_{j=1}^{i-1} a_i \oplus a_j$$

他想知道有多少个不同的序列 A 满足以下条件：

- A 的长度是 k。
- A 的值是 n。
- $0 \le a_i \le m$ ($1 \le i \le k$).

当且仅当存在一个索引 i 时，$[a_1, \dots, a_k]$、$[b_1, \dots, b_k]$ 这两个序列被认为是不同的，即 $a_i \neq b_i$。告诉路易斯答案模块 $10^9 + 7$。

输入一行包含三个整数 n, m, k, ($0 \le n \le 10^{15}$, $0 \le m \le 10^{12}$, $1 \le k \le 18$)。

```
#include <bits/stdc++.h>
#define endl '\n'
using namespace std;
typedef long double db;
typedef long long ll;

const ll N = 21;
const ll mod = 1e9 + 7;
const ll inf32 = 0x3f3f3f3f;
const ll inf64 = 5e18;
int C[N][N];

void init(int n){
    int i, j;
    for (C[0][0] = i = 1; i <= n; ++i)
        for (C[i][0] = j = 1; j <= i; ++j)
            C[i][j] = (C[i - 1][j] + C[i - 1][j - 1]) % mod;
}

ll n, m, k;
unordered_map<ll, ll> f[N << 1][N];
ll dp(int now, int lim, ll sum){
    if (sum > n) return 0;
    if (sum + (k >> 1ll) * (k + 1 >> 1ll) * ((1ll << now + 1) - 1) < n) return 0ll;
    if (now < 0) return sum == n;
    if (f[now][lim].count(sum)) return f[now][lim][sum];
    ll res = 0;
```

```
    if (m >> now & 1){
        for (int i = 0; i <= lim; ++i) //之前卡上界的个数
            for (int j = 0; j <= k - lim; ++j)
               (res += C[lim][i] * C[k - lim][j] % mod * dp(now - 1, i ,sum + (i + j) * (k - i - j) * (1ll << now)) %
↪  mod) %= mod;
    }else{
        for (int j = 0; j <= k - lim; ++j){
           (res += C[lim][0] * C[k - lim][j] % mod * dp(now - 1, lim, sum + j * (k - j) * (1ll << now)) % mod) %= mod;
        }
    }
    return f[now][lim][sum] = res;
}

void solve(){
    cin >> n >> m >> k;
    init(k);
    ll ans = dp(39, k, 0);
    cout << ans << endl;
}

signed main(){
    ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
    int t = 1;
    //cin >> t;
    while(t--) solve();
    return 0;
}
```

math

区间筛法

```
#include <bits/stdc++.h>
#define endl '\n'
#define int ll

using ll = long long;

constexpr int N = 1.2e6 + 10;
constexpr int P = 1e5 + 10;
constexpr int M = 108;
constexpr int mod = 998244353;

using namespace std;

bool vis[N];
int prime[P], pcnt, ans;

void init(){
    for (int i = 2; i < N; ++i) if (!vis[i]){
        prime[++pcnt] = i;
        for (int j = 2; j * i < N; ++j){
            vis[i * j] = 1;
        }
    }
}

inline int S(ll x){
    int res = 0;
    while(x) res += x % 10, x /= 10;
    return res;
}

int val[110];

void solve(){
    ll n;
    ans = 0;
    cin >> n;
    if (n <= M){
        for (int i = 1; i <= n; ++i)
```

```
            if (n % i == S(i)) ans++;
        cout << ans << endl;
        return;
    }
    vector<pair<ll, int>> a[M + 1];
    for (int i = 0; i <= M; ++i) val[i] = n - i;
    for (int i = 1; 1ll * prime[i] * prime[i] <= n; ++i){
        if (prime[i] <= M){
            for (int j = 0; j <= M; ++j){
                int cnt = 0;
                while (val[j] % prime[i] == 0){
                    val[j] /=prime[i];
                    cnt++;
                }
                if (cnt) a[j].push_back({prime[i], cnt});
            }
        }else if (n % prime[i] <= M){
            int j = n % prime[i], cnt = 0;
            while (val[j] % prime[i] == 0){
                val[j] /= prime[i];
                cnt++;
            }
            if (cnt) a[j].push_back({prime[i], cnt});
        }
    }
    for (int i = 0; i <= M; ++i){
        if (val[i] > 1) a[i].push_back({val[i], 1});
    }

    using pt = vector<pair<ll, int>>::iterator;
    auto dfs = [&](auto self, ll x, pt s, pt t, int res) -> void {
        if (s == t) {
            if (S(x) == res) ans += (x > res);
            return;
        }
        auto [y, z] = *s;
        ++s, self(self, x, s, t, res);
        for (int i = 1; i <= z; ++i){
            x *= y;
            self(self, x, s, t, res);
        }
        return;
    };

    for (int i = 0; i <= M; ++i){
        dfs(dfs, 1, a[i].begin(), a[i].end(), i);
    }
    cout << ans << endl;
}

signed main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    init();
    int t = 1;
    cin >> t;
    while(t--) solve();
    return 0;
}
```

矩阵四则运算

```
#include <bits/stdc++.h>
#define endl '\n'

using ll = long long;

constexpr int N = 2e5 + 10;
constexpr int mod = 1e9 + 7;

using namespace std;
```

```cpp

constexpr int SIZE = 100;

struct Matrix {
    ll M[SIZE + 5][SIZE + 5];

    void clear() {memset(M, 0, sizeof(M));}

    void reset() {
        clear();
        for (int i = 1; i <= SIZE; i++) M[i][i] = 1;
    }

    Matrix friend operator * (const Matrix & A, const Matrix & B) {
        Matrix C;
        C.clear();
        for (int i = 1; i <= SIZE; i++) {
            for (int j = 1; j <= SIZE; j++) {
                for (int k = 1; k <= SIZE; k++) {
                    C.M[i][j] = (C.M[i][j] + A.M[i][k] * B.M[k][j]) % mod;
                }
            }
        }
        return C;
    }

    Matrix friend operator + (const Matrix & A, const Matrix & B) {
        Matrix C;
        C.clear();
        for (int i = 1; i <= SIZE; i++) {
            for (int j = 1; j <= SIZE; j++) {
                C.M[i][j] = (A.M[i][j] + B.M[i][j]) % mod;
            }
        }
        return C;
    }

    Matrix friend operator - (const Matrix & A, const Matrix & B) {
        Matrix C;
        C.clear();
        for (int i = 1; i <= SIZE; i++){
            for (int j = 1; j <= SIZE; j++){
                C.M[i][j] = (A.M[i][j] - B.M[i][j] + mod) % mod;
            }
        }
        return C;
    }

    Matrix friend operator ^ (const Matrix & A, ll p) {
        Matrix C;
        C.reset();
        auto tmp = A;
        while (p) {
            if (p & 1) C = C * tmp;
            tmp = tmp * tmp;
            p >>= 1;
        }
        return C;
    }
};

Matrix t3, t4, t5;

Matrix MatrixSum(const Matrix & A, ll c) {
    if (c == 1){
        t3 = A;
        return A;
    }
    Matrix t2 = MatrixSum(A, c / 2ll);
    if (c % 2) {
        t4 = t3 * t3 * A;
```

```
        t5 = t3;
        t3 = t4;
        return (t2 + (t2 * t5)) + t3;
    }else{
        t4 = t3 * t3;
        t5 = t3;
        t3 = t4;
        return t2 + (t2 * t5);
    }
}

Matrix f1, f2;

void solve(){
    ll n, m, L, R;
    cin >> n >> m >> L >> R;
    f1.clear(), f2.clear();
    vector<vector<ll>> e(2 * n + 1, vector<ll>(2 * n + 1, 0ll));
    for (int i = 1; i <= n * 2; ++i){
        for (int j = 1; j <= n * 2; ++j) {
            e[i][j] = 0;
        }
    }
    for (int i = 1; i <= m; ++i){
        ll u, v, w;
        cin >> u >> v >> w;
        e[u][v] = (e[u][v] + w) % mod;
    }
    f1.M[1][1] = 1;
    for (int i = 2; i <= n; ++i){
        for (int j = 1; j < i; ++j){
            f1.M[1][i] = (f1.M[1][i] + f1.M[1][j] * e[j][i]) % mod;
        }
    }

    for (int i = 1; i <= n; ++i){
        for (int j = 1; j < n + i; ++j){
            if (j <= n) {
                f2.M[j][i] = (f2.M[j][i] + e[j][n + i]) % mod;
            }else {
                for (int k = 1; k <= n; ++k){
                    f2.M[k][i] = (f2.M[k][i] + f2.M[k][j - n] * e[j - n][i]) % mod;
                }
            }
        }
    }
    ll st = L / n; if (L % n == 0) st--;
    ll ed = R / n; if (R % n == 0) ed--;
    Matrix f3, f4;
    f3.clear(), f4.clear();
    if (!st) {
        f3 = f1;
    }else {
        auto p = f2 ^ st;
        for (int i = 1; i <= n; ++i){
            for (int j = 1; j <= n; ++j){
                f3.M[1][i] = (f3.M[1][i] + f1.M[1][j] * p.M[j][i])   % mod;
            }
        }
    }
    ll ans = 0;
    ll t1 = L % n;
    if (!t1) t1 = n;
    if (st == ed){
        ll t2 = R % n;
        if (!t2) t2 = n;
        for (int i = t1; i <= t2; ++i){
            ans = (ans + f3.M[1][i]) % mod;
        }
    }else{
        for (int i = t1; i <= n; ++i) ans = (ans + f3.M[1][i]) % mod;
```

```
        f4 = f3;
        f3.clear();
        auto p = f2 ^ ed;
        for (int i = 1; i <= n; ++i){
            for (int j = 1; j <= n; ++j){
                f3.M[1][i] = (f3.M[1][i] + f1.M[1][j] * p.M[j][i]) % mod;
            }
        }
        ll t2 = R % n;
        if (!t2) t2 = n;
        for (int i = 1; i <= t2; ++i){
            ans = (ans + f3.M[1][i]) % mod;
        }
        if (st != ed - 1){
            auto p = MatrixSum(f2, ed - st - 1);
            f3.clear();
            for (int i = 1; i <= n; ++i) {
                for (int j = 1; j <= n; ++j){
                    f3.M[1][i] = (f3.M[1][i] + f4.M[1][j] * p.M[j][i]) % mod;
                }
            }
            for (int i = 1; i <= n; ++i){
                ans = (ans + f3.M[1][i]) % mod;
            }
        }
    }
    cout << ans << endl;
}

signed main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    cin >> t;
    while(t--) solve();
    return 0;
}
```

构造异或方程组非 0 解

```
#include <bits/stdc++.h>
#define endl '\n'

using ll = long long;

constexpr int N = 2e5 + 10;
constexpr int mod = 998244353;

using namespace std;

array<bitset<60>, 60> matrix;
array<ll, 60> ans = {0};

void solve(){
    int n, m;
    cin >> n >> m;
    for (int i = 1; i <= m; ++i){
        int u, v;
        cin >> u >> v; u--, v--;
        matrix[u].set(v);
        matrix[v].set(u);
    }

    for (int i = 0, j = 0; i < n; ++i){
        for (int k = j; k < n; ++k){
            if (matrix[k][i]){
                swap(matrix[j], matrix[k]);
                for (int l = 0; l < n; ++l){
                    if (l != j && matrix[l][i]){
                        matrix[l] ^= matrix[j];
                    }
```

```
                }
                ++j;
                break;
            }
        }
    }

    for (int i = 0; i < n; ++i){
        if (matrix[i].count() == 1){
            cout << "No" << endl;
            return;
        }
        if (!matrix[i].count()) break;
        int a = 0;
        while (!matrix[i][a]) ++a;
        for (int j = a + 1; j < n; ++j){
            if (matrix[i][j]){
                ans[a] |= 1ll << n - j - 1;
                ans[j] |= 1ll << n - j - 1;
            }
        }
    }

    cout << "Yes" << endl;
    for (int i = 0; i < n; ++i){
        cout << ans[i] + !ans[i] << " \n"[i == n - 1];
    }
}

signed main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    //cin >> t;
    while(t--) solve();
    return 0;
}
```

4. exgcd

```
ll exgcd(ll a, ll b, ll &x, ll &y){
    if (b == 0)
        return x = 1, y = 0, a;
    int r = exgcd(b, a % b, x, y);
    tie(x, y) = make_tuple(y, x - (a / b) * y);
    return r;
}
```

$ax+by=(a,b)$ 有无数多组解，假设 x_0,y_0 是其中一解，则 $(x_0+k\frac{b}{(a,b)},y_0-k\frac{a}{(a,b)})$ 为其所有解，其中 $(x_0\%\frac{b}{(a,b)}+\frac{b}{(a,b)})\%\frac{b}{(a,b)}$ 为其最小正整数解。

线性筛

```
void getPrime(){
    for (int i = 2; i < maxn; ++i){
        if (!vis[i]) prime[cnt++] = i;
        for (int j = 0; prime[j] < maxn / i; ++j){
            vis[prime[j] * i] = 1;
            if (i % prime[j] == 0) break;
        }
    }
}
```

埃氏筛求区间质数

```
int prime[maxn], cnt;
bool vis[maxn];
int p[N];

void getPrime()
```

```
{
    for (int i = 2; i < maxn; ++i)
    {
        if (!vis[i])
            prime[++cnt] = i;
        for (int j = 1; prime[j] <= (maxn - 10) / i; ++j)
        {
            vis[prime[j] * i] = 1;
            if (i % prime[j] == 0)
                break;
        }
    }
}

void solve()
{
    int l, r;
    cin >> l >> r;
    for (int i = 1; i <= cnt && prime[i] <= r; ++i)
    {
        int w = (l + prime[i] - 1) / prime[i];
        for (int j = max(2ll, w); prime[i] * j <= r; ++j)
        {
            p[prime[i] * j - l] = 1;
        }
    }
    vi vec;
    for (int i = max(2ll, l); i <= r; ++i)
    {
        if (!p[i - l])
            vec.push_back(i);
        p[i - l] = 0;
    }
}

signed main()
{
    ios;
    getPrime();
    int t = 1;
    cin >> t;
    while (t--)
    {
        solve();
    }
    return 0;
}
```

质因数分解

```
int p[maxn], a[maxn], cnt;
void div(int x)
{
    memset(a, 0, sizeof a);
    cnt = 0;
    for (int i = 2; i * i <= x; ++i)
    {
        if (x % i == 0)
        {
            p[++cnt] = i;
            while (x % i == 0)
            {
                x /= i;
                a[cnt]++;
            }
        }
    }
    if (x != 1)
    {
        p[++cnt] = x;
        a[cnt] = 1;
```

```
    }
}
```

同余方程（模版）

$ax \equiv 1 \pmod b$ 等价于 $ax + by = 1$

```
int exgcd(int a, int b, int & x, int & y){
    if (b == 0){
        x = 1, y = 0;
        return a;
    }
    int d = exgcd(b, a % b, y, x);
    y -= a / b * x;
    return d;
}

void solve(){
    int a, b;
    cin >> a >> b;
    if (gcd(a, b) != 1){
        cout << -1 << endl;
        return;
    }
    int x, y;
    int d = exgcd(a, b, x, y);
    b /= d;
    cout << (x % b + b) % b << endl;
}
```

中国剩余定理（任意两个模数需要互质）

1）计算所有模数的积 M

2）对于第 i 个方程计算 $M_i = M/m_i$ 在模上 m_i 上逆元，并计算出 $M_i * inv(M_i, m_i)$

3）$ans \equiv \sum_{i=1}^{k} r_i * M_i * inv(M_i, m_i) \pmod M$

```
#include <bits/stdc++.h>
#define endl '\n'
#define pll pair<ll, ll>
#define tll tuple<ll, ll, ll>
#define vi vector<int>
#define vl vector<ll>
#define x first
#define y second
#define all(a) a.begin() + 1, a.end()
#define int ll
#define rep(i, j, k) for (int i = (j); i <= (k); i++)
#define per(i, j, k) for (int i = (j); i >= (k); i--)
#define ios ios::sync_with_stdio(false), cin.tie(0), cout.tie(0)
using namespace std;
using i64 = long long;
typedef long long ll;
const ll maxn = 2e5 + 10;
const ll mod = 998244353;
const ll inf32 = 1e9;
const ll inf64 = 1e18;

typedef __int128 i128;

i128 read()
{
    i128 x = 0;
    bool f = 0;
    char c = getchar();
    while (c < '0' || c > '9')
    {
        if (c == '-')
            f = 1;
```

```
        c = getchar();
    }
    while (c >= '0' && c <= '9')
    {
        x = (x << 1) + (x << 3) + (c ^ 48);
        c = getchar();
    }
    return f ? -x : x;
}

inline void write(i128 x)
{
    if (x < 0)
        putchar('-'), x = -x;
    if (x > 9)
        write(x / 10);
    putchar(x % 10 + '0');
}

i64 exgcd(i64 a, i64 b, i64 &x, i64 &y)
{
    if (b == 0)
    {
        x = 1, y = 0;
        return a;
    }
    i64 d = exgcd(b, a % b, y, x);
    y -= a / b * x;
    return d;
}

void solve()
{
    i64 p1, r1;  // 定义两个 64 位整数 p1 和 r1，分别表示第一个同余方程的模数和余数
    int n;  // 定义整数 n，表示同余方程的数量
    cin >> n;  // 从输入中读取 n
    cin >> p1 >> r1;  // 从输入中读取第一个同余方程的模数和余数
    bool flag = false;  // 定义一个标志变量 flag，用于标记是否存在无解的情况

    for (int i = 0; i < n - 1; i++)  // 对于剩下的 n-1 个同余方程
    {
        i64 p2, r2;  // 定义两个 64 位整数 p2 和 r2，分别表示当前同余方程的模数和余数
        cin >> p2 >> r2;  // 从输入中读取当前同余方程的模数和余数
        if (flag == true)  // 如果已经发现存在无解的情况，则跳过剩下的处理
            continue;
        i64 y1, y2;  // 定义两个 64 位整数 y1 和 y2，用于存储扩展欧几里得算法的结果
        i64 d = exgcd(p1, p2, y1, y2);  // 调用扩展欧几里得算法，计算 p1 和 p2 的最大公约数 d，并得到 y1 和 y2 的值
        if ((r2 - r1) % d != 0)  // 如果 (r2 - r1) 不能被 d 整除，则说明这两个同余方程无解
            flag = true;  // 设置 flag 为 true，表示存在无解的情况
        y1 = (i128)y1 * (r2 - r1) / d % (p2 / d);  // 计算新的 y1 的值
        i64 lc = (i128)p1 * p2 / d;  // 计算 p1 和 p2 的最小公倍数 lc
        r1 = (((i128)p1 * y1 % lc + r1) % lc + lc) % lc;  // 计算新的 r1 的值
        p1 = lc;  // 更新 p1 的值为 lc
    }
    if (flag)  // 如果存在无解的情况
        cout << -1 << endl;  // 输出-1
    else  // 否则
        cout << r1 << endl;  // 输出最终的结果 r1
}

signed main()
{
    ios;
    int t = 1;
    //cin >> t;
    while (t--)
    {
        solve();
    }
    return 0;
}
```

第一个大于 u 的特解

```
i64 exgcd(i64 a, i64 b, i64 &x, i64 &y)
{
    if (b == 0)
    {
        x = 1, y = 0;
        return a;
    }
    i64 d = exgcd(b, a % b, y, x);
    y -= a / b * x;
    return d;
}

int p, e, k, u;
int a[3], m[] = {23, 28, 33};
void solve()
{
    cin >> p >> e >> k >> u;
    a[0] = p, a[1] = e, a[2] = k;
    int a1 = a[0], m1 = m[0];
    i64 lc;
    for (int i = 1; i <= 2; i++)
    {
        int a2 = a[i], m2 = m[i];
        i64 y1, y2;
        i64 d = exgcd(m1, m2, y1, y2);
        y1 = (i128)y1 * (a2 - a1) / d % (m2 / d);
        lc = (i128)m1 * m2 / d;
        a1 = (((i128)m1 * y1 % lc + a1) % lc + lc) % lc;
        m1 = lc;
    }
    if (a1 <= u) a1 += lc;
    cout << a1 - u << endl;
}
```

欧拉降幂

$$a^b \equiv \begin{cases} a^{b \bmod \varphi(p)}, & \gcd(a,p)=1 \\ a^b, & \gcd(a,p)\neq 1, b<\varphi(p) \\ a^{b \bmod \varphi(p)+\varphi(p)}, & \gcd(a,p)\neq 1, b\geq \varphi(p) \end{cases} \pmod p$$

$m \to \varphi(m) \to \varphi(\varphi(m)) \to ... \to 1$ 这个过程最多进行 $log(m)$ 次

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

const int N = 1e5 + 10;

int n, m, w[N], q;
int phi_m[N], total;
bool k;

int power(int a, int b, int mod) {
    int res = 1;
    while (b) {
        if (b & 1) {
            res = res * a;
            if (res >= mod) {
                k = true;
                res %= mod;
            }
        }
        a = a * a;
        if (a >= mod) {
            k = true;
            a %= mod;
        }
        b >>= 1;
    }
```

```
    return res;
}

int f(int n) {
    int res = n;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            res = res / i * (i - 1);
            while (n % i == 0) n /= i;
        }
    }
    if (n > 1) res = res / n * (n - 1);
    return res;
}

void init() {
    int tmp = m;
    total = 0;
    phi_m[++total] = tmp;
    while (tmp != 1) {
        tmp = f(tmp);
        phi_m[++total] = tmp;
    }
}

void solve(int l, int r) {
    r = min(r, l - 1 + total);
    int res = 1;
    for (int i = r; i >= l; i--) {
        k = false;
        res = power(w[i], res, phi_m[i - l + 1]);
        if (k) res += phi_m[i - l + 1];
    }
    cout << res % phi_m[1] << "\n";
}

signed main() {
    cin >> n >> m;
    init();  // 预处理欧拉函数的值
    for (int i = 1; i <= n; i++) cin >> w[i];
    cin >> q;
    while (q--) {
        int l, r;
        cin >> l >> r;
        solve(l, r);
    }
    return 0;
}
```

容斥的一点点小结论

错排问题

$d(n) = \sum_{i=0}^{n}(-1)^i\binom{n}{i}(n-i)! = n!\sum_{i=0}^{n}(-1)^i\frac{1}{i!}$

$\lim_{n\to\infty}\frac{d(n)}{n!} = \frac{1}{e}$

$N\left((1-a_1)(1-a_2)\dots(1-a_n)\right) = \sum_{i=0}^{n}(-1)^i\sum_{1<j_1<j_2<\dots<j_i<n}N\left(a_{j_1}a_{j_2}\dots a_{j_i}\right)$

$N\left(a_1\dots a_x(1-a_{x+1})\cdots(1-a_{x+n})\right) = \sum_{i=0}^{n}(-1)^i\sum_{x\le j_1<j_2<\dots<j_i\le n}N(a_1\dots a_x a_{j1}\dots a_{ji})$

图论

(有向图) 强联通分量缩点

强连通分量缩点后的图称为 SCC。以 O(N + M) 的复杂度完成上述全部操作。

缩点后的图拥有拓扑序

```cpp
#include <bits/stdc++.h>
#define endl '\n'

using ll = long long;

constexpr int N = 2e5 + 10;
constexpr int mod = 998244353;

using namespace std;

struct SCC {
    int n, now, cnt;
    vector<vector<int>> ver;
    vector<int> dfn, low, col, id, S;

    SCC(int n) : n(n), ver(n + 1), low(n + 1), id(n + 1) {
        dfn.resize(n + 1, - 1);
        col.resize(n + 1, - 1);
        now = cnt = 0;
    }

    void add(int x, int y) {
        ver[x].push_back(y);
    }

    void tarjan(int x) {
        dfn[x] = low[x] = now++;
        S.push_back(x);
        for (auto y : ver[x]) {
            if (dfn[y] == -1) {
                tarjan(y);
                low[x] = min(low[x], low[y]);
            } else if (col[y] == -1) {
                low[x] = min(low[x], dfn[y]);
            }
        }
        if (dfn[x] == low[x]) {
            int pre;
            cnt++;
            id[cnt] = x;
            do {
                pre = S.back();
                col[pre] = cnt;
                S.pop_back();
            } while (pre != x);
        }
    }

    auto work () {
        for (int i = 1; i <= n; ++i) {
            if (dfn[i] == -1) tarjan(i);
        }

        vector<int> siz(cnt + 1);
        vector<vector<int>> adj(cnt + 1);

        for (int i = 1; i <= n; ++i) {
            siz[col[i]]++;
            for (auto j : ver[i]) {
                int x = col[i], y = col[j];
                if (x != y) {
                    adj[x].push_back(y);
                }
            }
        }

        //return tuple{cnt, adj, col, sz};
        return tuple{cnt, col, siz, id};
    }
};

```

```cpp
void solve() {
    int n, m;
    cin >> n >> m;
    SCC scc(n);
    vector<pair<int, int>> e(n + 1);
    for (int i = 1; i <= m; ++i) {
        int u, v;
        cin >> u >> v;
        e[i] = {u, v};
        scc.add(u, v);
    }

    auto [cnt, col, sz, id] = scc.work();

    vector<int> in(n + 1);
    for (int i = 1; i <= m; ++i) {
        auto [u, v] = e[i];
        if (col[u] != col[v]) {
            in[col[v]]++;
        }
    }
    vector<int> ans;
    // for (int i = 1; i <= cnt; ++i){
    //     cout << id[i] << " \n"[i == cnt];
    // }
    for (int i = 1; i <= cnt; ++i) {
        if (in[i] == 0) ans.push_back(id[i]);
    }
    sort(ans.begin(), ans.end());
    cout << ans.size() << endl;
    for (auto x : ans) {
        cout << x << " ";
    }
    cout << endl;
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    //cin >> t;
    while (t--) solve();
    return 0;
}
```

(‘无向图’) 割点缩点

割点缩点后的图称为点双连通图 (V - DCC)，该模板可以在 O(N + M) 复杂度内求解图中全部割点、划分点双（颜色相同的点位于同一个点双连通分量中）。

```cpp
VDCC(int n) : n(n) {
    ver.resize(n + 1);
    dfn.resize(n + 1);
    low.resize(n + 1);
    col.resize(2 * n + 1);
    point.resize(n + 1);
    S.clear();
    cnt = now = 0;
}

void add(int u, int v) {
    if (u == v) return;
    ver[u].push_back(v);
    ver[v].push_back(u);
}


void tarjan(int u, int rt) {
    low[u] = dfn[u] = ++now;
    S.push_back(u);
    if (u == rt && !ver[u].size()) {
```

```
        ++cnt;
        col[cnt].push_back(u);
        return;
    }
    int flag = 0;
    for (auto v : ver[u]) {
        if (!dfn[v]) {
            tarjan(v, rt);
            low[u] = min(low[u], low[v]);
            if (dfn[u] <= low[v]) {
                flag++;
                if (u != rt || flag > 1) {
                    point[u] = 1;
                }
                int pre = 0;
                cnt++;
                do {
                    pre = S.back();
                    col[cnt].push_back(pre);
                    S.pop_back();
                } while (pre != v);
                col[cnt].push_back(u);
            }
        } else {
            low[u] = min(low[u], dfn[v]);
        }
    }
}
pair<int, vector<vector<int>>> rebuild() { // 新图的顶点数量，新图
    work();
    vector<vector<int>> adj(cnt + 1);
    for (int i = 1; i <= cnt; ++i) {
        if (!col[i].size()) {
            continue;
        }
        for (auto j : col[i]) {
            if (point[j]) {
                adj[i].push_back(point[j]);
                adj[point[j]].push_back(i);
            }
        }
    }
    return {cnt, adj};
}

void work() {
    for (int i = 1; i <= n; ++i) {
        if (!dfn[i]) tarjan(i, i);
    }
}
};
```

(‘无向图’) 边双，不求桥

割边缩点后的图称为边双连通图 (E - DCC)，该模板可以在 O(N + M) 复杂度内求解图中全部割边、划分边双（颜色相同的点位于同一个边双连通分量中）。

性质补充：对于一个边双，删去任意边后依旧联通；对于边双中的任意两点，一定存在两条不相交的路径连接这两个点（路径上可以有公共点，但是没有公共边）。

```
struct EDCC {
    int n, now, cnt;
    vector<vector<pair<int, int>>> ver;
    vector<int> col, dfn, low, vis, ind, S;

    EDCC(int n) : n(n), ver(n + 1), col(n + 1), dfn(n + 1), low(n + 1), vis(n + 1), ind(n + 1) {
        now = cnt = 0;
    }

    void add(int u, int v, int id) {
        if (u == v) return;
```

```
        ver[u].emplace_back(v, id << 1);
        ver[v].emplace_back(u, id << 1 | 1);
    }

    void tarjan(int u, int fa) {
        low[u] = dfn[u] = ++now;
        S.push_back(u);
        vis[u] = 1;
        for (auto [v, t] : ver[u]) {
            if (t == (fa ^ 1)) continue;
            if (!dfn[v]) {
                tarjan(v, t);
                low[u] = min(low[u], low[v]);
            } else if (vis[v]) {
                low[u] = min(low[u], dfn[v]);
            }
        }
        if (low[u] == dfn[u]) {
            int pre;
            cnt++;
            do {
                pre = S.back();
                vis[pre] = 0;
                col[pre] = cnt;
                S.pop_back();
            } while (pre != u);
        }
    }

    auto work() {
        for (int i = 1; i <= n; ++i) if (!dfn[i]) tarjan(i, 0);
        vector<int> sz(cnt + 1);
        vector<vector<int>> adj(cnt + 1);
        for (int i = 1; i <= n; ++i) {
            int x = col[i];
            sz[x]++;
            for (auto [j, t] : ver[i]) {
                int y = col[j];
                if (x != y) {
                    adj[x].push_back(y);
                    ind[y]++;
                }
            }
        }

        return tuple{cnt, adj, col, sz, ind};
    }
};
```

求桥

下面代码实现了求割边，其中，当 isbridge[x] 为真时，(father[x],x) 为一条割边。(下面代码无法处理重边)

```
int low[MAXN], dfn[MAXN], dfs_clock;
bool isbridge[MAXN];
vector<int> G[MAXN];
int cnt_bridge;
int father[MAXN];

void tarjan(int u, int fa) {
    father[u] = fa;
    low[u] = dfn[u] = ++dfs_clock;
    for (int i = 0; i < G[u].size(); i++) {
        int v = G[u][i];
        if (!dfn[v]) {
            tarjan(v, u);
            low[u] = min(low[u], low[v]);
            if (low[v] > dfn[u]) {
                isbridge[v] = true;
                ++cnt_bridge;
```

```
            }
        } else if (dfn[v] < dfn[u] && v != fa) {
            low[u] = min(low[u], dfn[v]);
        }
    }
}
```

求无向图中不在奇圈上的点

1. 如果一个双连通分量内的某些顶点在一个奇圈中（即双连通分量含有奇圈），那么这个双连通分量的其他顶点也在某个奇圈中；2. 如果一个双连通分量含有奇圈，则他必定不是一个二分图。反过来也成立，这是一个充要条件。基于以上俩点，Tarjan 求出每个点双连通图分量后，用交叉染色法判断是否存在奇圈！

```
#include<bits/stdc++.h>
using namespace std;
const int maxn = 2e6 + 50;
const int N = 1050;
struct node{
    int to, next;
} edge[maxn];
int top, stack1[maxn];
int tot, cnt, dccn;
int dfn[N], low[N];
int head[N];
vector<int> dcc[N];
void add(int from, int to){
    edge[++cnt].to = to;edge[cnt].next = head[from];head[from] = cnt;
}
void ins(int u, int v){add(u, v); add(v, u);}
void Tarjan(int u, int rt){
    dfn[u] = low[u] = ++tot;
    if(rt == u && head[u] == -1){
        dccn++;
        dcc[dccn].clear();
        dcc[dccn].push_back(u);
        return;
    }
    stack1[++top] = u;
    for(int i = head[u];i != -1;i = edge[i].next){
        int v = edge[i].to;
        if(!dfn[v]){
            Tarjan(v, rt);
            low[u] = min(low[u], low[v]);
            if(low[v] >= dfn[u]){
                dccn++;
                dcc[dccn].clear();
                dcc[dccn].push_back(u);
                while (1){
                    int w = stack1[top];
                    dcc[dccn].push_back(w);
                    top--;
                    if (w == v){ // 做到子树全部弹出为止，不是 u，不然 v 的兄弟也会被弹出
                        break;
                    }
                }
            }
        }
        else low[u] = min(low[u], dfn[v]);
    }
}
int now[N], col[N], vis[N];
bool dfs(int u, int c){
    col[u] = c;
    for(int i = head[u];i != -1;i = edge[i].next){
        int v = edge[i].to;
        if(!now[v]) continue;
        if(col[v] == c) return false;
        if(col[v] == 0 && !dfs(v, 3 - c)) return false;
    }
    return true;
```

```
}
int mp[N][N];
void init(){
    cnt = 0; tot = 0, dccn = 0, top = 0;
    memset(head, -1, sizeof(head));
    memset(mp, 0, sizeof(mp));
    memset(low, 0, sizeof(low));
    memset(dfn, 0, sizeof(dfn));
    memset(vis, 0, sizeof(vis));
}
int main()
{
    std::ios::sync_with_stdio(false);
    int n, m;
    while(cin >> n >> m){
        if(n == 0 && m == 0) break;
        init();
        for(int i = 0;i < m;i++){
            int u, v;
            cin >> u >> v;
            mp[u][v] = mp[v][u] = 1;
        }
        for(int i = 1;i <= n;i++){
            for(int j = i + 1;j <= n;j++){
                if(mp[i][j] == 0){
                    ins(i, j);
                }
            }
        }
        for(int i = 1;i <= n;i++) if(!dfn[i]) Tarjan(i, i);
        for(int i = 1;i <= dccn;i++){
            memset(now, 0, sizeof(now));
            memset(col, 0, sizeof(col));
            for(int j = 0;j < dcc[i].size();j++){
                now[dcc[i][j]] = 1;
            }
            if(!dfs(dcc[i][0], 1)){
                for(int j = 0;j < dcc[i].size();j++){
                    vis[dcc[i][j]] = 1;
                }
            }
        }
        int ans = 0;
        for(int i = 1;i <= n;i++) if(!vis[i]) ans++;
        cout << ans << endl;
    }
    return 0;
}
```

DAG 最小路径覆盖

```
#include <bits/stdc++.h>
#define endl '\n'

using ll = long long;

constexpr int N = 2e5 + 10;
constexpr int mod = 998244353;

using namespace std;

struct SCC {
    int n, now, cnt;
    vector<vector<int>> ver;
    vector<int> dfn, low, col, id, S;

    SCC(int n) : n(n), ver(n + 1), low(n + 1), id(n + 1) {
        dfn.resize(n + 1, - 1);
        col.resize(n + 1, - 1);
        now = cnt = 0;
    }
```

```
21
22      void add(int x, int y) {
23          ver[x].push_back(y);
24      }
25
26      void tarjan(int x) {
27          dfn[x] = low[x] = now++;
28          S.push_back(x);
29          for (auto y : ver[x]) {
30              if (dfn[y] == -1) {
31                  tarjan(y);
32                  low[x] = min(low[x], low[y]);
33              } else if (col[y] == -1) {
34                  low[x] = min(low[x], dfn[y]);
35              }
36          }
37          if (dfn[x] == low[x]) {
38              int pre;
39              cnt++;
40              id[cnt] = x;
41              do {
42                  pre = S.back();
43                  col[pre] = cnt;
44                  S.pop_back();
45              } while (pre != x);
46          }
47      }
48  };
49
50  void solve(){
51      int n;
52      cin >> n;
53      vector<string> s(n + 1);
54      for (int i = 1; i <= n; ++i) {
55          cin >> s[i];
56      }
57      SCC scc(n);
58      for (int i = 1; i <= n; ++i) {
59          for (int j = 1; j <= n; ++j) {
60              if (s[i].back() == s[j].front()) {
61                  scc.add(i, j);
62              }
63          }
64      }
65
66      for (int i = 1; i <= n; ++i) {
67          if (scc.dfn[i] == -1) scc.tarjan(i);
68      }
69
70      int cnt = scc.cnt;
71
72      vector<int> siz(cnt + 1);
73      vector<vector<int>> adj(cnt + 1, vector<int>(cnt + 1));
74
75      for (int i = 1; i <= n; ++i) {
76          siz[scc.col[i]]++;
77          for (auto j : scc.ver[i]) {
78              int x = scc.col[i], y = scc.col[j];
79              if (x != y) {
80                  adj[x][y] = 1;
81              }
82          }
83      }
84
85      for (int k = 1; k <= cnt; ++k) {
86          for (int i = 1; i <= cnt; ++i) {
87              for (int j = 1; j <= cnt; ++j) {
88                  if (i == j) continue;
89                  if (adj[i][k] && adj[k][j]) adj[i][j] = 1;
90              }
91          }
```

```
    }

    vector<int> vis(cnt + 1), pr(cnt + 1, -1);
    int tot = 0;
    auto dfs = [&](auto self, int u) -> bool {
        vis[u] = tot;
        for (int v = 1; v <= cnt; ++v) {
            if (adj[u][v] && (pr[v] == -1 || (vis[pr[v]] != tot && self(self, pr[v])))) {
                pr[v] = u;
                return true;
            }
        }
        return false;
    };
    int ans = 0;
    for (int i = 1; i <= cnt; ++i) {
        tot++;
        if (dfs(dfs, i)) ans++;
    }
    cout << cnt - ans << endl;
}

signed main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    //cin >> t;
    while(t--) solve();
    return 0;
}
```

Segment Tree

线段树 I

```
#include <bits/stdc++.h>
#define endl '\n'
#define int ll
using namespace std;
typedef long double db;
typedef long long ll;

const ll N = 4e5 + 10;
const ll mod = 998244353;
const ll inf32 = 0x3f3f3f3f;
const ll inf64 = 5e18;

ll a[N], sum[N << 2], tag[N << 2];

void push_up(int x) {sum[x] = sum[x << 1] + sum[x << 1 | 1];}

void build(int u, int l, int r){
    tag[u] = 0;
    if (l == r) {sum[u] = a[l]; return;}
    int mid = (l + r) >> 1;
    build(u << 1, l, mid);
    build(u << 1 | 1, mid + 1, r);
    push_up(u);
}

inline void f(int u, int l, int r, int k){
    tag[u] += k;
    sum[u] += k * (r - l + 1);
}

inline void push_down(int u, int l, int r){
    int mid = (l + r) >> 1;
    f(u << 1, l, mid, tag[u]);
    f(u << 1 | 1, mid + 1, r, tag[u]);
    tag[u] = 0;
}
```

```

inline void update(int u, int nl, int nr, int l, int r, int k){
    if (nl <= l && r <= nr) {f(u, l, r, k); return;}
    push_down(u, l, r);
    int mid = (l + r) >> 1;
    if (nl <= mid) update(u << 1, nl, nr, l, mid, k);
    if (nr > mid) update(u << 1 | 1, nl, nr, mid + 1, r, k);
    push_up(u);
}

inline int query(int u, int qx, int qy, int l, int r){
    ll res = 0;
    if (qx <= l && r <= qy) return sum[u];
    int mid = l + r >> 1;
    push_down(u, l, r);
    if (qx <= mid) res += query(u << 1, qx, qy, l, mid);
    if (qy > mid) res += query(u << 1 | 1, qx, qy, mid + 1, r);
    return res;
}

void solve(){
    int n, m;
    cin >> n >> m;
    for (int i = 1; i <= n; ++i) cin >> a[i];
    build(1, 1, n);
    while(m--){
        int op, x, y, k;
        cin >> op >> x >> y;
        if (op == 1) {
            cin >> k;
            update(1, x, y, 1, n, k);
        } else {
            cout << query(1, x, y, 1, n) << endl;
        }
    }
}

signed main(){
    ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
    int t = 1;
    //cin >> t;
    while(t--){
        solve();
    }
    return 0;
}
```

线段树 II

```

#include <bits/stdc++.h>
#define endl '\n'
using namespace std;
typedef long double db;
typedef long long ll;

const ll N = 4e5 + 10;
const ll mod = 998244353;
const ll inf32 = 0x3f3f3f3f;
const ll inf64 = 5e18;

ll a[N], sum[N << 2], taga[N << 2], tagm[N << 2], p;

void push_up(int x) {sum[x] = (sum[x << 1] + sum[x << 1 | 1]) % p;}

void build(int u, int l, int r){
    taga[u] = 0;
    tagm[u] = 1;
    if (l == r) {sum[u] = a[l]; return;}
    int mid = (l + r) >> 1;
    build(u << 1, l, mid);
    build(u << 1 | 1, mid + 1, r);
```

```
    push_up(u);
}

inline void push_down(int u, int l, int r){
    int m = l + r >> 1;
    ll &tm = tagm[u], &ta = taga[u];
    if (tm != 1){
        taga[u << 1] = (taga[u << 1] * tm) % p;
        taga[u << 1 | 1] = (taga[u << 1 | 1] * tm) % p;
        tagm[u << 1] = (tagm[u << 1] * tm) % p;
        tagm[u << 1 | 1] = (tagm[u << 1 | 1] * tm) % p;
        sum[u << 1] = (sum[u << 1] * tm) % p;
        sum[u << 1 | 1] = (sum[u << 1 | 1] * tm) % p;
        tm = 1;
    }
    if (ta){
        taga[u << 1] = (taga[u << 1] + ta) % p;
        taga[u << 1 | 1] = (taga[u << 1 | 1] + ta) % p;
        sum[u << 1] = (sum[u << 1] + ta * (m - l + 1)) % p;
        sum[u << 1 | 1] = (sum[u << 1 | 1] + ta * (r - m)) % p;
        ta = 0;
    }
}

inline void update1(int u, int nl, int nr, int l, int r, int k){
    if (nl <= l && r <= nr) {
        tagm[u] = (tagm[u] * k) % p;
        taga[u] = (taga[u] * k) % p;
        sum[u] = (sum[u] * k) % p;
        return;
    }
    int mid = (l + r) >> 1;
    push_down(u, l, r);
    if (nl <= mid) update1(u << 1, nl, nr, l, mid, k);
    if (nr > mid) update1(u << 1 | 1, nl, nr, mid + 1, r, k);
    push_up(u);
}

inline void update2(int u, int nl, int nr, int l, int r, int k){
    if (nl <= l && r <= nr) {
        taga[u] = (taga[u] + k) % p;
        sum[u] = (sum[u] + k * (r - l + 1)) % p;
        return;
    }
    int mid = (l + r) >> 1;
    push_down(u, l, r);
    if (nl <= mid) update2(u << 1, nl, nr, l, mid, k);
    if (nr > mid) update2(u << 1 | 1, nl, nr, mid + 1, r, k);
    push_up(u);
}

inline int query(int u, int qx, int qy, int l, int r){
    ll res = 0;
    if (qx <= l && r <= qy) return sum[u];
    int mid = l + r >> 1;
    push_down(u, l, r);
    if (qx <= mid) res = (res + query(u << 1, qx, qy, l, mid)) % p;
    if (qy > mid) res = (res + query(u << 1 | 1, qx, qy, mid + 1, r)) % p;
    return res;
}

void solve(){
    int n, m;
    cin >> n >> m >> p;
    for (int i = 1; i <= n; ++i) cin >> a[i];
    build(1, 1, n);
    while(m--){
        int op, x, y, k;
        cin >> op >> x >> y;
        if (op == 1) {
            cin >> k;
```

```
            update1(1, x, y, 1, n, k);
        } else if (op == 2){
            cin >> k;
            update2(1, x, y, 1, n, k);
        } else{
            cout << query(1, x, y, 1, n) << endl;
        }
    }
}

signed main(){
    ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
    int t = 1;
    //cin >> t;
    while(t--){
        solve();
    }
    return 0;
}
```

扫描线

```
#include <bits/stdc++.h>
#define endl '\n'
using namespace std;
typedef long double db;
typedef long long ll;

const ll N = 2e5 + 10;
const ll mod = 998244353;
const ll inf32 = 0x3f3f3f3f;
const ll inf64 = 5e18;

struct line{
    int x1, x2, h, tag;
}li[N];
bool cmp (line a, line b) {return a.h < b.h;}

struct Tree{
    int l, r, len, cnt;
}tr[N * 8];

int d[N], n;

void build(int p, int l, int r){
    tr[p] = {l, r};
    if (l == r) return;
    int mid = l + r >> 1;
    build(p << 1, l, mid);
    build(p << 1 | 1, mid + 1, r);
}

void push_up(int p){
    int l = tr[p].l, r = tr[p].r;
    if (tr[p].cnt) tr[p].len = d[r + 1] - d[l];
    else tr[p].len = tr[p << 1].len + tr[p << 1 | 1].len;
}

void update(int p, int l, int r, int k){
    if (l <= tr[p].l && tr[p].r <= r) {
        tr[p].cnt += k;
        push_up(p);
        return;
    }
    int mid = tr[p].l + tr[p].r >> 1;
    if (l <= mid) update(p << 1, l, r, k);
    if (r > mid) update(p << 1 | 1, l, r, k);
    push_up(p);
}

void solve(){
```

```
    cin >> n;
    for (int i = 1; i <= n; ++i){
        int a, b, c, e;
        cin >> a >> b >> c >> e;
        li[i] = {a, c, b, 1};
        li[i + n] = {a, c, e, -1};
        d[i] = a, d[i + n] = c;
    }
    n <<= 1;
    sort(li + 1, li + n + 1, cmp);
    sort(d + 1, d + n + 1);
    int len = unique(d + 1, d + n + 1) - d - 1;
    build(1, 1, len - 1); //只需要 lens - 1 个区间位置即可
    ll ans = 0;
    for (int i = 1; i < n; ++i){
        int x1 = lower_bound(d + 1, d + len + 1, li[i].x1) - d;
        int x2 = lower_bound(d + 1, d + len + 1, li[i].x2) - d;
        update(1, x1, x2 - 1, li[i].tag);
        ans += 1ll * (li[i + 1].h - li[i].h) * tr[1].len;
    }
    cout << ans << endl;
}

signed main(){
    ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
    int t = 1;
    //cin >> t;
    while(t--) solve();
    return 0;
}
```

窗口的星星 (另一个版本的扫描线)

对于每组数据:

第一行 3 个整数 n, W, H 表示有 n 颗星星，窗口宽为 W，高为 H。

接下来 n 行，每行三个整数 x_i, y_i, l_i 表示星星的坐标在 (x_i, y_i)，亮度为 l_i。

输出 T 个整数，表示每组数据中窗口星星亮度总和的最大值。

```
#include <bits/stdc++.h>
#define endl '\n'
using namespace std;
typedef long double db;
typedef long long ll;

const ll N = 2e5 + 10;
const ll mod = 998244353;
const ll inf32 = 0x3f3f3f3f;
const ll inf64 = 5e18;

ll n, w, h, d[N];
struct line{
    ll x1, x2, h, val;
    bool operator < (const line& a) const{
        return (h != a.h) ? h < a.h : val > a.val;
    }
}L[N];

struct SegmentTree{
    ll l, r, mx, add;
}T[N << 2];

void push_up(int p){
    T[p].mx = max(T[p << 1].mx, T[p << 1 | 1].mx);
}

void build(int p, int l, int r){
    T[p].l = l, T[p].r = r;
    T[p].mx = T[p].add = 0;
```

```
    if (l == r) return;
    int mid = l + r >> 1;
    build(p << 1, l, mid);
    build(p << 1 | 1, mid + 1, r);
    push_up(p);
}

void push_down(int p){
    if (T[p].add){
        T[p << 1].add += T[p].add;
        T[p << 1 | 1].add += T[p].add;
        T[p << 1].mx += T[p].add;
        T[p << 1 | 1].mx += T[p].add;
        T[p].add = 0;
    }
}

void update(int p, int ql, int qr, ll val){
    int l = T[p].l, r = T[p].r;
    if (ql <= l && r <= qr){
        T[p].add += val;
        T[p].mx += val;
        return;
    }
    push_down(p);
    int mid = l + r >> 1;
    if (ql <= mid) update(p << 1, ql, qr, val);
    if (qr > mid) update(p << 1 | 1, ql, qr, val);
    push_up(p);
}

void solve(){
    cin >> n >> w >> h;
    for (int i = 1; i <= n; ++i){
        ll x, y, l;
        cin >> x >> y >> l;
        d[(i << 1) - 1] = y;
        d[(i << 1)] = y + h - 1;
        L[(i << 1) - 1] = (line){y, y + h - 1, x, l};
        L[(i << 1)] = (line){y, y + h - 1, x + w - 1, -l};
    }
    n <<= 1;
    sort(d, d + n + 1);
    sort(L + 1, L + n + 1);
    ll cnt = unique(d, d + n + 1) - d - 1;
    for (int i = 1; i <= n; ++i){
        ll x1 = lower_bound(d + 1, d + cnt + 1, L[i].x1) - d;
        ll x2 = lower_bound(d + 1, d + cnt + 1, L[i].x2) - d;
        L[i].x1 = x1, L[i].x2 = x2;
    }
    build(1, 1, cnt);
    ll ans = 0;
    for (int i = 1; i <= n; ++i){
        update(1, L[i].x1, L[i].x2, L[i].val);
        ans = max(ans, T[1].mx);
    }
    cout << ans << endl;
}

signed main(){
    ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
    int t = 1;
    cin >> t;
    while(t--) solve();
    return 0;
}
```

二分 + 线段树

输入数据的第一行为两个整数 n 和 m，n 表示序列的长度，m 表示局部排序的次数。

第二行为 n 个整数，表示 1 到 n 的一个排列。

接下来输入 m 行，每一行有三个整数 op, l, r，op 为 0 代表升序排序，op 为 1 代表降序排序，l, r 表示排序的区间。

最后输入一个整数 q，表示排序完之后询问的位置

输出数据仅有一行，一个整数，表示按照顺序将全部的部分排序结束后第 q 位置上的数字。

```cpp
#include <bits/stdc++.h>
#define endl '\n'
using namespace std;
typedef long double db;
typedef long long ll;

const ll N = 1e5 + 10;
const ll mod = 998244353;
const ll inf32 = 0x3f3f3f3f;
const ll inf64 = 5e18;

struct Sort
{ // 记录这 m 次排序操作
    int op, l, r;
} q[N];
struct Node
{                   // 线段树
    int l, r;       // sum 记录 01 序列中 1 的个数
    int sum, lazy; // lazy 为懒标记: 1 代表将此段全部变为 1, -1 代表将此段全部变为 0
} tr[N * 4];
int n, m, k, a[N];
void pushup(int u)
{
    tr[u].sum = tr[u << 1].sum + tr[u << 1 | 1].sum;
}
void pushdown(int u)
{
    if (tr[u].lazy)
    {
        tr[u << 1].lazy = tr[u << 1 | 1].lazy = tr[u].lazy;
        if (tr[u].lazy == 1)
        {
            tr[u << 1].sum = tr[u << 1].r - tr[u << 1].l + 1;
            tr[u << 1 | 1].sum = tr[u << 1 | 1].r - tr[u << 1 | 1].l + 1;
        }
        else
            tr[u << 1].sum = tr[u << 1 | 1].sum = 0;
        tr[u].lazy = 0;
    }
}
void build(int u, int l, int r, int x)
{
    if (l == r)
        tr[u] = {l, r, a[l] >= x, 0}; // 序列中大于等于 x 的数变为 1, 小于 x 的数变为 0
    else
    {
        tr[u] = {l, r};
        int mid = l + r >> 1;
        build(u << 1, l, mid, x), build(u << 1 | 1, mid + 1, r, x);
        pushup(u);
    }
}
int query(int u, int l, int r) // 查询 [l,r] 中 1 的个数
{
    if (l <= tr[u].l && tr[u].r <= r)
        return tr[u].sum;
    pushdown(u);
    int mid = tr[u].l + tr[u].r >> 1;
    int sum = 0;
    if (mid >= l)
        sum = query(u << 1, l, r);
    if (mid < r)
        sum += query(u << 1 | 1, l, r);
    return sum;
```

```
}
void update(int u, int l, int r, int c) // 将 [l,r] 区间中的数全部变为 c
{
    if (l <= tr[u].l && tr[u].r <= r)
    {
        tr[u].sum = c * (tr[u].r - tr[u].l + 1);
        tr[u].lazy = c ? 1 : -1;
    }
    else
    {
        pushdown(u);
        int mid = tr[u].l + tr[u].r >> 1;
        if (mid >= l)
            update(u << 1, l, r, c);
        if (mid < r)
            update(u << 1 | 1, l, r, c);
        pushup(u);
    }
}
bool queryPoint(int u, int x) // 查询 x 位置上的数是否为 1
{
    if (tr[u].l == tr[u].r)
        return tr[u].sum;
    pushdown(u);
    int mid = tr[u].l + tr[u].r >> 1;
    if (x <= mid)
        return queryPoint(u << 1, x);
    else
        return queryPoint(u << 1 | 1, x);
}
bool check(int mid) // 检查此答案值是否合法
{
    build(1, 1, n, mid); // 用此答案值建树
    for (int i = 1; i <= m; i++)
    {
        int op = q[i].op, l = q[i].l, r = q[i].r; // 对 [l,r] 区间进行排序
        int cnt = query(1, l, r);                 // 查询 [l,r] 中 1 的个数
        if (cnt == 0 || cnt == r - l + 1)
            continue; // 如果区间中的数全部相同，那么不需要进行排序
        if (op)
        {
            update(1, l, cnt + l - 1, 1);
            update(1, cnt + l, r, 0);
        }
        else
        {
            update(1, l, r - cnt, 0);
            update(1, r - cnt + 1, r, 1);
        }
    }
    return queryPoint(1, k); // 所有操作完成后查看 k 位置上的数是否为 1
}

void solve()
{
    cin >> n >> m;
    for (int i = 1; i <= n; ++i)
        cin >> a[i];
    for (int i = 1; i <= m; ++i)
    {
        cin >> q[i].op >> q[i].l >> q[i].r;
    }
    cin >> k;
    int l = 1, r = n;
    while (l < r)
    {
        int mid = l + r + 1 >> 1;
        if (check(mid))
            l = mid;
        else
            r = mid - 1;
```

```
    }
    cout << l << endl;
}

signed main()
{
    ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

线段树合并 (HDU2024 暑期多校 1003)

```
#include <bits/stdc++.h>
#include <vector>
#define endl '\n'
using namespace std;
typedef long double db;
typedef long long ll;
typedef unsigned long long ull;

const ll N = 5e5 + 10;
const ll mod = 998244353;
const ll inf32 = 0x3f3f3f3f;
const ll inf64 = 5e18;

vector<int> G[N];
int p[N], rk[N];

int rt[N], ls[N * 60], rs[N * 60], cnt;
int c[N * 60];
ull t[N * 60];
ull cur = 0;
ull ans[N], S1[N], S2[N], a[N];


void push_up(int p){
    t[p] = t[ls[p]] + t[rs[p]];
    c[p] = c[ls[p]] + c[rs[p]];
}

void upd(int l, int r, int &p, int x, ull v) {
    if (!p)
        p = ++cnt;
    if (l == r) {
        c[p] = 1;
        t[p] = v;
        return;
    }
    int m = (l + r) >> 1;
    if (x <= m)
        upd(l, m, ls[p], x, v);
    else
        upd(m + 1, r, rs[p], x, v);
    push_up(p);
}

int merge(int x, int y, ull prev1, ull prev2) {
    if (!x || !y) {
        cur += prev1 * t[y];
        cur += prev2 * t[x];
        return x + y;
    }
    int z = ++cnt;
    ls[z] = merge(ls[x], ls[y],prev1, prev2);
    rs[z] = merge(rs[x], rs[y], prev1 + c[ls[x]], prev2 + c[ls[y]]);
    push_up(z);
    return z;
```

```
}

void dfs(int u, int fa){
    upd(0, N, rt[u], rk[u], a[u] * a[u]);
    S1[u] = a[u];
    S2[u] = a[u] * a[u];
    for (auto v : G[u]){
        if (v == fa) continue;
        dfs(v, u);
        S1[u] += S1[v];
        S2[u] += S2[v];
        ans[u] += ans[v];
        cur = 0;
        rt[u] = merge(rt[u], rt[v], 0, 0);
        ans[u] += cur;
    }
}

void solve(){
    int n;
    cin >> n;
    for (int i = 1; i < n; ++i){
        int u, v;
        cin >> u >> v;
        --u, --v;
        G[u].push_back(v);
        G[v].push_back(u);
    }
    for (int i = 0; i < n; ++i) cin >> a[i];
    for (int i = 0; i < n; ++i) p[i] = i;
    sort(p, p + n, [&](int i, int j){return a[i] < a[j];});
    for (int i = 0; i < n; ++i) rk[p[i]] = i;
    dfs(0, -1);
    for (int i = 0; i < n; ++i){
        ans[i] *= 2;
        ans[i] += S2[i];
        ans[i] -= S1[i] * S1[i];
    }
    ull res = 0;
    for (int i = 0; i < n; ++i) res ^= ans[i];
    cout << res << endl;
}

signed main(){
    ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
    int t = 1;
    //cin >> t;
    while(t--) solve();
    return 0;
}
```

动态开点线段树

```
#include <bits/stdc++.h>
using namespace std;

const int N = 100010;
#define int long long

struct node {
    int l, r;
    int add, sum;
} tr[N << 1];
// 正常线段树，这里不开 4 倍大小会 RE

int n, m, idx, root;
int a[N];

void pushup(int p) {
    tr[p].sum = tr[tr[p].l].sum + tr[tr[p].r].sum;
}
```

```

void pushdown(int p, int l, int r) {
    if(tr[p].add) {
        int mid = l + r >> 1;
        tr[tr[p].l].sum += (mid - l + 1) * tr[p].add, tr[tr[p].l].add += tr[p].add;
        tr[tr[p].r].sum += (r - mid) * tr[p].add, tr[tr[p].r].add += tr[p].add;
        tr[p].add = 0;
    }
}

void build(int &p, int l, int r) {
    if(!p) p = ++idx;
    if(l == r) { tr[p].sum = a[l]; return ;}
    int mid = l + r >> 1;
    build(tr[p].l, l, mid), build(tr[p].r, mid + 1, r);
    pushup(p);
}

void modify(int &p, int l, int r, int ql, int qr, int k) {
    if(!p) p = ++idx;
    if(l >= ql && r <= qr) {
        tr[p].sum += (r - l + 1) * k;
        tr[p].add += k;
        return ;
    }
    pushdown(p, l, r);
    int mid = l + r >> 1;
    if(ql <= mid) modify(tr[p].l, l, mid, ql, qr, k);
    if(qr > mid) modify(tr[p].r, mid + 1, r, ql, qr, k);
    pushup(p);
}

int query(int p, int l, int r, int ql, int qr) {
    if(l >= ql && r <= qr) { return tr[p].sum; }
    int mid = l + r >> 1;
    pushdown(p, l, r);
    int v = 0;
    if(ql <= mid) v = query(tr[p].l, l, mid, ql, qr);
    if(qr > mid) v += query(tr[p].r, mid + 1, r, ql, qr);
    return v;
}

signed main() {
    cin >> n >> m;
    for (int i = 1; i <= n; i++) cin >> a[i];

    build(root, 1, n);

    int op, x, y, k;
    while(m--) {
        cin >> op >> x >> y;
        if(op == 1) {
            cin >> k;
            modify(root, 1, n, x, y, k);
        } else {
            cout << query(root, 1, n, x, y) << endl;
        }
    }
    return 0;
}
```

线段树 (单点修改、区间最值)

```
#include <bits/stdc++.h>
#include <vector>
#define endl '\n'
using namespace std;
typedef long long ll;

const int N = 2e5 + 10;
const int mod = 998244353;
```

```cpp
const int inf32 = 0x3f3f3f3f;
const ll inf64 = 4e18;

struct Node
{
    int l, r;
    ll v1, v2;
}tr[N * 4];

ll a[N], b[N];

void pushup(int u)
{
    tr[u].v1 = max(tr[u << 1].v1, tr[u << 1 | 1].v1);
    tr[u].v2 = min(tr[u << 1].v2, tr[u << 1 | 1].v2);
}

void build(int u, int l, int r)
{
    tr[u] = {l, r};
    if (l == r) {
        tr[u].v1 = tr[u].v2 = b[l];
        return;
    }
    int mid = l + r >> 1;
    build(u << 1, l, mid), build(u << 1 | 1, mid + 1, r);
    pushup(u);
}

ll query1(int u, int l, int r)
{
    if (tr[u].l >= l && tr[u].r <= r) return tr[u].v1;
    int mid = tr[u].l + tr[u].r >> 1;
    ll v = -inf64;
    if (l <= mid) v = query1(u << 1, l, r);
    if (r > mid) v = max(v, query1(u << 1 | 1, l, r));
    return v;
}

ll query2(int u, int l, int r)
{
    if (tr[u].l >= l && tr[u].r <= r) return tr[u].v2;
    int mid = tr[u].l + tr[u].r >> 1;
    ll v = inf64;
    if (l <= mid) v = query2(u << 1, l, r);
    if (r > mid) v = min(v, query2(u << 1 | 1, l, r));
    return v;
}

void modify(int u, int x, ll v)
{
    if (tr[u].l == x && tr[u].r == x) tr[u].v1 += v, tr[u].v2 += v;
    else
    {
        int mid = tr[u].l + tr[u].r >> 1;
        if (x <= mid) modify(u << 1, x, v);
        else modify(u << 1 | 1, x, v);
        pushup(u);
    }
}


void solve(){
    int n;
    cin >> n;
    for (int i = 1; i <= n; ++i)
        cin >> a[i];
    for (int i = 2; i <= n; ++i) {
        b[i] = a[i] - a[i - 1];
    }
    build(1, 1, n);
```

```
    int q;
    cin >> q;
    while (q--){
        int op, l, r;
        ll x;
        cin >> op >> l >> r;
        if (op == 1) {
            cin >> x;
            modify(1, l, x);
            modify(1, r + 1, -x);
        }else if (op == 2){
            if (l == r){
                cout << 1 << endl;
            }else{
                ll mx = query1(1, l + 1, r);
                ll mn = query2(1, l + 1, r);
                if (mx == 0 && mn == 0){
                    cout << 1 << endl;
                }else{
                    cout << 0 << endl;
                }
            }
        }else if (op == 3){
            if (l == r){
                cout << 1 << endl;
            }else{
                ll mn = query2(1, l + 1, r);
                cout << (mn > 0) << endl;
            }
        }else if (op == 4){
            if (l == r){
                cout << 1 << endl;
            }else{
                ll mx = query1(1, l + 1, r);
                cout << (mx < 0) << endl;
            }
        }else {
            int ql = l + 1, qr = r + 1, ok = 0;
            while (ql < qr){
                int mid = (ql + qr) >> 1;
                ll mn = query2(1, l + 1, mid);
                if (mn > 0) {
                    ok = mid;
                    ql = mid + 1;
                }else{
                    qr = mid;
                }
            }
            if (!ok) {
                cout << 0 << endl;
            }else{
                if (ok >= r) {
                    cout << 0 << endl;
                }else{
                    ll mx = query1(1, ok + 1, r);
                    if (mx < 0) {
                        cout << 1 << endl;
                    }else{
                        cout << 0 << endl;
                    }
                }
            }
        }
    }
}

signed main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    //cin >> t;
```

```
    while(t--) solve();
    return 0;
}
```

单点修改，区间最大字段和

```
#include<bits/stdc++.h>
using namespace std;

#define int long long

const int maxn = 1001000;
int n, m;
int ans;

struct tree{
    int lmax; // 当前区间最大前缀和
    int rmax; // 当前区间最大后缀和
    int maxx; // 当前区间最大子段和
    int sum; // 当前区间的和
}t[4 * maxn];

void push_up(int rt){
    t[rt].sum = t[rt << 1].sum + t[rt << 1 | 1].sum;
    // 当前区间的和: 左子树的和 + 右子树的和
    t[rt].rmax = max(t[rt << 1 | 1].rmax, t[rt << 1 | 1].sum + t[rt << 1].rmax);
    // 当前区间的最大后缀和: 右子树的最大后缀和 or 右子树的和 + 左子树的最大后缀和
    t[rt].lmax = max(t[rt << 1].lmax, t[rt << 1].sum + t[rt << 1 | 1].lmax);
    // 当前区间的最大前缀和: 左子树的最大前缀和 or 左子树的和 + 右子树的最大前缀和
    t[rt].maxx = max(t[rt << 1].rmax + t[rt << 1 | 1].lmax, max(t[rt << 1].maxx, t[rt << 1 | 1].maxx));
    // 当前区间的最大子段和: 左子树的最大子段和 or 右子树的最大子段和 or 左子树的最大后缀和 + 右子树的最大前缀和
}

void build(int rt, int l, int r){
    if(l == r){
        cin >> t[rt].maxx;
        t[rt].lmax = t[rt].rmax = t[rt].sum = t[rt].maxx;
        return;
    }
    int mid = l + r >> 1;
    build(rt << 1, l, mid);
    build(rt << 1 | 1, mid + 1, r);
    push_up(rt);
}

void update(int rt, int l, int r, int x, int y){
    if(l == r){
        t[rt].lmax = t[rt].rmax = t[rt].maxx = t[rt].sum = y;
        return ;
    }
    int mid = l + r >> 1;
    if(mid >= x) update(rt << 1, l, mid, x, y);
    else update(rt << 1 | 1, mid + 1, r, x, y);
    push_up(rt);
}

tree query(int rt, int l, int r, int x, int y){
    if(l >= x && r <= y) return t[rt]; // 区间完全覆盖，直接返回该节点
    int mid = l + r >> 1;
    if(y <= mid) return query(rt << 1, l, mid, x, y); // 只在左区间，直接查询左区间
    else if(x > mid) return query(rt << 1 | 1, mid + 1, r, x, y); // 只在右区间，直接查询右区间
    else{
        tree res_l = query(rt << 1, l, mid, x, y);
        tree res_r = query(rt << 1 | 1, mid + 1, r, x, y);
        tree res;
        // res_l 记录左覆盖区间，res_r 记录右覆盖区间，合并后得到 res
        // 用 push_up 同样的方式更新 res
        res.sum = res_l.sum + res_r.sum;
        res.lmax = max(res_l.sum + res_r.lmax, res_l.lmax);
        res.rmax = max(res_r.rmax, res_r.sum + res_l.rmax);
        res.maxx = max(max(res_l.maxx, res_r.maxx), res_l.rmax + res_r.lmax);
```

```
        return res;
    }
}

signed main(){
    cin >> n >> m;
    build(1, 1, n);
    int opt, x, y;
    while(m--){
        cin >> opt >> x >> y;
        if(opt == 1){
            if(x > y) swap(x, y);
            tree ans = query(1, 1, n, x, y);
            cout << ans.maxx << '\n';
        }
        else update(1, 1, n, x, y);
    }
    return 0;
}
```

半群 (一次函数嵌套)

```
#include <bits/stdc++.h>
#define endl '\n'
#define int ll
using namespace std;
typedef long long ll;

const int N = 5e5 + 10;
const int mod = 998244353;
const int inf32 = 0x3f3f3f3f;
const ll inf64 = 4e18;

int n, m, K[N],b[N];

struct node{
    int l, r;
    ll mul, sum;
}t[N << 2];

node unite(node x, node y){
    node ans;
    ans.l = x.l, ans.r = y.r;
    ans.mul = x.mul * y.mul % mod;
    ans.sum = (x.sum * y.mul % mod + y.sum) % mod;
    return ans;
}

void push_up(int u){
    t[u] = unite(t[u << 1], t[u << 1 | 1]);
}

void build(int u, int l, int r){
    if (l == r){
        t[u].l = t[u].r = l;
        t[u].sum = b[l];
        t[u].mul = K[l];
        return;
    }
    int mid = l + r >> 1;
    build(u << 1, l, mid);
    build(u << 1 | 1, mid + 1, r);
    push_up(u);
}

void modify(int u, int x, int K, int B){
    if (t[u].l == t[u].r)
    {
        t[u].sum = B;
        t[u].mul = K;
        return;
```

```
    }
    int mid = t[u].l + t[u].r >> 1;
    if (x <= mid) modify(u << 1, x, K, B);
    else modify(u << 1 | 1, x, K, B);
    push_up(u);
}

node query(int u, int l, int r){
    if (t[u].l == l && t[u].r == r) return t[u];
    int mid = t[u].l + t[u].r >> 1;
    if (r <= mid) return query(u << 1, l, r);
    else if (l > mid) return query(u << 1 | 1, l, r);
    else return unite(query(u << 1, l, mid), query(u << 1 | 1, mid + 1, r));
}

void solve(){
    cin >> n >> m;
    for (int i = 1; i <= n; ++i) cin >> K[i] >> b[i];
    build(1, 1, n);
    for (int i = 1; i <= m; ++i){
        int op;
        cin >> op;
        if (op == 0){
            int p, c, d;
            cin >> p >> c >> d;
            p++;
            modify(1, p, c, d);
        }else{
            int l, r, x;
            cin >> l >> r >> x;
            l++;
            auto ans = query(1, l, r);
            cout << (ans.mul * x % mod + ans.sum) % mod << endl;
        }
    }
}

signed main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    //cin >> t;
    while(t--) solve();
    return 0;
}
```

树上半群修改，路径查询

```
#include <bits/stdc++.h>
#define endl '\n'
using namespace std;
typedef long long ll;

const int N = 2e5 + 10;
const int mod = 998244353;
const int inf32 = 0x3f3f3f3f;
const ll inf64 = 4e18;

struct Info {
    int a1, b1, a2, b2;
};

Info operator + (const Info &l , const Info &r) {
    Info ret;
    //信息合并
    ret.a1 = (1LL * l.a1 * r.a1 % mod); //向下的
    ret.b1 = (1LL * r.a1 * l.b1 % mod + r.b1) % mod;
    ret.a2 = (1LL * l.a2 * r.a2 % mod); // 向上的
    ret.b2 = (1LL * l.a2 * r.b2 % mod + l.b2) % mod;
    return ret;
}
```

```cpp

struct node {
    int a, b;
};

node operator + (const node &l , const node &r) {
    node ret;
    //信息合并
    ret.a = (1LL * l.a * r.a % mod);
    ret.b = (1LL * r.a * l.b % mod + r.b) % mod;
    return ret;
}

Info t[N << 2];
int a[N], b[N], sz[N], top[N], dep[N], in[N], dfn[N], f[N];

struct Segment{
    int n;

    void push_up(int u) {
        t[u] = t[u << 1] + t[u << 1 | 1];
    }

    void build(int u, int l, int r){
        if (l == r){
            t[u].a1 = t[u].a2 = a[dfn[l]];
            t[u].b1 = t[u].b2 = b[dfn[l]];
            return;
        }
        int mid = l + r >> 1;
        build(u << 1, l ,mid);
        build(u << 1 | 1, mid + 1, r);
        push_up(u);
    }

    void modify(int u, int l, int r, int p, const Info & v){
        if (l == r){
            t[u] = v;
            return;
        }
        int mid = l + r >> 1;
        if (p <= mid) modify(u << 1, l, mid, p, v);
        else  modify(u << 1 | 1, mid + 1, r, p, v);
        push_up(u);
    }

    void modify(int u, int l, int r, int ql, int qr, const Info & v){
        if (l == ql && r == qr){
            t[u] = v;
            return;
        }
        int mid = l + r >> 1;
        if (qr <= mid) modify(u << 1, l, mid, ql, qr, v);
        else if (ql > mid) modify(u << 1 | 1, mid + 1, r, ql, qr, v);
        else {
            modify(u << 1, l, mid, ql, mid, v);
            modify(u << 1 | 1, mid + 1, r, mid + 1, qr, v);
        }
        push_up(u);
    }

    Info query(int rt, int l, int r, int ql, int qr) {
        if (l == ql && r == qr) {
            return t[rt];
        }
        int mid = l + r >> 1;
        if (qr <= mid) return query(rt << 1 , l , mid , ql , qr);
        else if (ql > mid) return query(rt << 1 | 1 , mid + 1 , r , ql , qr);
        else {
            return query(rt << 1 , l , mid , ql , mid) + query(rt << 1 | 1 , mid + 1 , r , mid + 1 , qr);
        }
```

```
    }

    Info query(int ql, int qr) {
        if (ql > qr){
            return Info {1, 0, 1, 0};
        }
        return query(1, 1, n, ql, qr);
    }
}tree;

void solve(){
    int n, q;
    cin >> n >> q;
    for (int i = 1; i <= n; ++i){
        cin >> a[i] >> b[i];
    }
    vector<vector<int>> G(n + 1);
    for (int i = 1; i < n; ++i){
        int u, v;
        cin >> u >> v;
        u++, v++;
        G[u].push_back(v);
        G[v].push_back(u);
    }
    auto dfs1 = [&](auto self, int u, int fa) -> void {
        if (fa) G[u].erase(find(G[u].begin(), G[u].end(), fa));
        sz[u]++;
        for (auto &v : G[u]){
            dep[v] = dep[u] + 1;
            f[v] = u;
            self(self, v, u);
            sz[u] += sz[v];
            if (sz[v] > sz[G[u][0]]){
                swap(G[u][0], v);
            }
        }
    };

    int tot = 0 ;
    auto dfs2 = [&](auto self, int u, int fa) -> void {
        in[u] = ++tot;
        dfn[in[u]] = u;
        for (auto & v : G[u]){
            top[v] = (v == G[u][0] ? top[u] : v);
            self(self, v, u);
        }
    };

    dep[1] = 0;
    dfs1(dfs1, 1, 0);
    top[1] = 0;
    dfs2(dfs2, 1, 0);
    tree.n = n;
    tree.build(1, 1, n);

    // 向上的
    auto path1 = [&](auto self, int x, int y) -> node{
        node ans = {1, 0};
        while(top[x] != top[y]){
            if (dep[top[x]] < dep[top[y]]) std::swap(x, y);
            Info res = tree.query(in[top[x]], in[x]);
            ans = (ans + node{res.a2, res.b2});
            x = f[top[x]];
        }
        if (dep[x] > dep[y]) std::swap(x, y);
        Info res = tree.query(in[x] + 1, in[y]);
        ans = (ans + node{res.a2, res.b2});
        return ans;
    };

    auto path2 = [&](auto self, int x, int y) -> node{
```

```
        node ans = {1, 0};
        while(top[x] != top[y]){
            if (dep[top[x]] < dep[top[y]]) std::swap(x, y);
            Info res = tree.query(in[top[x]], in[x]);
            ans = (node{res.a1, res.b1} + ans);
            x = f[top[x]];
        }
        if (dep[x] > dep[y]) std::swap(x, y);
        Info res = tree.query(in[x], in[y]);
        ans = (node{res.a1, res.b1} + ans);
        return ans;
    };

    auto lca = [&](auto self, int u, int v) -> int{
        while (top[u] != top[v]) {
            if (dep[top[u]] > dep[top[v]]) {
                u = f[top[u]];
            } else {
                v = f[top[v]];
            }
        }
        return dep[u] < dep[v] ? u : v;
    };

    while(q--){
        int op, l, r, x;
        cin >> op >> l >> r >> x;
        l++;
        if (op == 0){
            tree.modify(1, 1, n, in[l], Info{r, x, r, x});
        }else{
            r++;
            int lc = lca(lca, l, r);
            node ansL = path1(path1, l, lc);
            node ansR = path2(path2, lc, r);
            int ans = (1ll * ansL.a * x + ansL.b) % mod;
            ans = (1ll * ansR.a * ans + ansR.b) % mod;
            cout << ans << endl;
        }
    }
}

signed main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    //cin >> t;
    while(t--) solve();
    return 0;
}
```

动态开点，权值线段树

```
#include <bits/stdc++.h>
#define endl '\n'

using ll = long long;

constexpr int N = 1e5 + 10;
constexpr int M = 1e7 + 10;
constexpr int mod = 998244353;

using namespace std;

struct SegmentTree{
    int ls, rs;
    int num;
}tr[M];

int n, q, cnt, np1[N], np2[N], f[N];

```

```
ll m, k, a[N];

void update(int u, ll l, ll r, ll p, int sum) {
    tr[u].num = sum;
    if (l == r) return;
    ll mid = l + r >> 1;
    if (p <= mid){
        if (!tr[u].ls){
            tr[u].ls = ++cnt;
        }
        update(tr[u].ls, l, mid, p, sum);
    }else{
        if (!tr[u].rs){
            tr[u].rs = ++cnt;
        }
        update(tr[u].rs, mid + 1, r, p, sum);
    }
}

int query(int u, ll l, ll r, ll ql, ll qr) {
    if (ql <= l && r <= qr) return tr[u].num;
    ll mid = l + r >> 1;
    int ans = n + 2;
    if (ql <= mid) {
        ans = min(ans, query(tr[u].ls, l, mid, ql, qr));
    }
    if (qr > mid) {
        ans = min(ans, query(tr[u].rs, mid + 1, r, ql, qr));
    }
    return ans;
}

void solve(){
    cin >> n >> m >> k;
    for (int i = 1; i <= n; ++i) cin >> a[i];
    cnt = 1, tr[0].num = tr[1].num = n + 1;
    for (int i = n; i >= 1; --i){
        np1[i] = query(1, 1ll, m, a[i], min(m, a[i] + k));
        np2[i] = query(1, 1ll, m, max(1ll, a[i] - k), a[i]);
        update(1, 1ll, m, a[i], i);
    }
    cin >> q;
    while (q--){
        int l, r;
        cin >> l >> r;
        for (int i = l; i <= r; ++i) f[i] = 0;
        int ans = 0;
        for (int i = r; i >= l; --i){
            f[i] = 1;
            if (np1[i] <= r) f[i] = max(f[i], f[np1[i]] + 1);
            if (np2[i] <= r) f[i] = max(f[i], f[np2[i]] + 1);
            ans = max(ans, f[i]);
        }
        cout << (r - l + 1 - ans) << endl;
    }
    for (int i = 0; i <= cnt; ++i) tr[i].num = tr[i].ls = tr[i].rs = 0;
}

signed main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    cin >> t;
    while(t--) solve();
    return 0;
}
```

左右端点限制的最大区间字段和

```
#include <bits/stdc++.h>
#define endl '\n'
```

```cpp

using ll = long long;

constexpr int N = 1e5 + 10;
constexpr int mod = 998244353;

using namespace std;

int a[N];

struct Info{
    int lmax, smax, rmax, sum;
};

Info merge(Info s1, Info s2) {
    Info res;
    res.lmax = max(s1.lmax, s1.sum + s2.lmax);
    res.smax = max({s1.smax, s2.smax, s1.rmax + s2.lmax});
    res.rmax = max(s2.rmax, s2.sum + s1.rmax);
    res.sum = s1.sum + s2.sum;
    return res;
}

struct Node {
    int l, r;
    Info dat;
}tr[N << 2];

void push_up(int u) {
    tr[u].dat = merge(tr[u << 1].dat, tr[u << 1 | 1].dat);
}

void build(int u, int l, int r) {
    tr[u].l = l, tr[u].r = r;
    if (l == r) {
        tr[u].dat.lmax = tr[u].dat.rmax = tr[u].dat.smax = tr[u].dat.sum = a[l];
        return;
    }
    int mid = (l + r) >> 1;
    build(u << 1, l, mid);
    build(u << 1 | 1, mid + 1, r);
    push_up(u);
}

Info query_sub(int u, int l, int r) {
    if (l > r) {
        return Info{0, 0, 0, 0};
    }
    if (tr[u].l == l && tr[u].r == r) {
        return tr[u].dat;
    }
    int mid = (tr[u].l + tr[u].r) >> 1;
    if (r <= mid) return query_sub(u << 1, l, r);
    else if (l > mid) return query_sub(u << 1 | 1, l, r);
    return merge(query_sub(u << 1, l, mid), query_sub(u << 1 | 1, mid + 1, r));
}

int query(int l1, int r1, int l2, int r2) {
    if (r1 < l2) {
        int tmp = query_sub(1, l1, r1).rmax;
        tmp += query_sub(1, r1 + 1, l2 - 1).sum;
        tmp += query_sub(1, l2, r2).lmax;
        return tmp;
    }
    int ans = query_sub(1, l2, r1).smax;
    if (l1 < l2) ans = max(ans, query_sub(1, l1, l2).rmax + query_sub(1, l2, r2).lmax - a[l2]);
    if (r2 > r1) ans = max(ans, query_sub(1, l1, r1).rmax + query_sub(1, r1, r2).lmax - a[r1]);
    return ans;
}

void solve(){
```

```
    int n;
    cin >> n;
    for (int i = 1; i <= n; ++i) cin >> a[i];
    build(1, 1, n);
    int m;
    cin >> m;
    while (m--) {
        int x1, x2, y1, y2;
        cin >> x1 >> x2 >> y1 >> y2;
        cout << query(x1, x2, y1, y2) << endl;
    }
}

signed main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    cin >> t;
    while(t--) solve();
    return 0;
}
```

染色 (树上路径颜色段数量)

11122211 -> 111 222 11

```
#include <bits/stdc++.h>
#define endl '\n'

using ll = long long;

constexpr int N = 2e5 + 10;
constexpr int mod = 998244353;

using namespace std;

int col[N];

vector<int> G[N];

struct Node {
    int l, r;
    int num, lc, rc;
    int tag;
}tr[N << 2];

void build(int u, int l, int r) {
    tr[u].l = l, tr[u].r = r, tr[u].num = 0;
    if (l == r) return;
    int mid = (l + r) >> 1;
    build(u << 1, l, mid);
    build(u << 1 | 1, mid + 1, r);
}

void push_up(int u) {
    tr[u].lc = tr[u << 1].lc;
    tr[u].rc = tr[u << 1 | 1].rc;
    int tmp = tr[u << 1].num + tr[u << 1 | 1].num;
    if (tr[u << 1].rc == tr[u << 1 | 1].lc) tmp--;
    tr[u].num = tmp;
}

void push_down(int u) {
    if (tr[u].tag) {
        tr[u << 1].tag = tr[u << 1 | 1].tag = tr[u].tag;
        tr[u << 1].num = tr[u << 1 | 1].num = 1;
        tr[u << 1].lc = tr[u << 1].rc = tr[u << 1 | 1].lc = tr[u << 1 | 1].rc = tr[u].lc;
        tr[u].tag = 0;
    }
}

```

```
void update(int u, int l, int r, int v) {
    if (tr[u].l == l && tr[u].r == r) {
        tr[u].num = tr[u].tag = 1;
        tr[u].lc = tr[u].rc = v;
        return;
    }
    push_down(u);
    int mid = (tr[u].l + tr[u].r) >> 1;
    if (r <= mid) update(u << 1, l, r, v);
    else if (l > mid) update(u << 1 | 1, l, r, v);
    else update(u << 1, l, mid, v), update(u << 1 | 1, mid + 1, r, v);
    push_up(u);
}

int LC, RC;

int query(int u, int ql, int qr, int L, int R) {
    if (tr[u].l == L) LC = tr[u].lc;
    if (tr[u].r == R) RC = tr[u].rc;
    if (tr[u].l == ql && tr[u].r == qr) {
        return tr[u].num;
    }
    push_down(u);
    int mid = (tr[u].l + tr[u].r) >> 1;
    if (qr <= mid) return query(u << 1, ql, qr, L, R);
    else if (ql > mid) return query(u << 1 | 1, ql, qr, L, R);
    else {
        int ans = query(u << 1, ql, mid, L, R) + query(u << 1 | 1, mid + 1, qr, L, R);
        if (tr[u << 1].rc == tr[u << 1 | 1].lc) ans--;
        return ans;
    }

}

void solve(){
    int n, m;
    cin >> n >> m;
    for (int i = 1; i <= n; ++i) cin >> col[i];
    for (int i = 1; i < n; ++i) {
        int u, v;
        cin >> u >> v;
        G[u].push_back(v);
        G[v].push_back(u);
    }

    vector<int> sz(n + 1), dfn(n + 1), top(n + 1), dep(n + 1), son(n + 1), f(n + 1);
    int tot = 0;
    auto dfs1 = [&](auto self, int u, int fa) -> void {
        sz[u] = 1; dep[u] = dep[fa] + 1;
        f[u] = fa;
        for (auto v : G[u]) {
            if (v == fa) continue;
            self(self, v, u);
            sz[u] += sz[v];
            if (sz[v] > sz[son[u]]) {
                son[u] = v;
            }
        }
    };

    auto dfs2 = [&](auto self, int u, int t) -> void {
        dfn[u] = ++tot;
        top[u] = t;
        if (son[u]) {
            self(self, son[u], top[u]);
        }
        for (auto v : G[u]) {
            if (v == f[u] || v == son[u]) continue;
            self(self, v, v);
        }
    };
```

```
    dfs1(dfs1, 1, 0);
    dfs2(dfs2, 1, 1);
    build(1, 1, n);

    for (int i = 1; i <= n; ++i) update(1, dfn[i], dfn[i], col[i]);

    auto calc = [&](int op, int u, int v, int c) -> int {
        int ans = 0;
        if (op == 1) {
            while (top[u] != top[v]) {
                if (dep[top[u]] < dep[top[v]]) swap(u, v);
                update(1, dfn[top[u]], dfn[u], c);
                u = f[top[u]];
            }
            if (dep[u] > dep[v]) swap(u, v);
            update(1, dfn[u], dfn[v], c);
        }else {
            int ans1 = -1, ans2 = -1;
            while (top[u] != top[v]) {
                if (dep[top[u]] < dep[top[v]]) swap(u, v), swap(ans1, ans2);
                ans += query(1, dfn[top[u]], dfn[u], dfn[top[u]], dfn[u]);
                if (RC == ans1) ans--;
                ans1 = LC;
                u = f[top[u]];
            }
            if (dep[u] < dep[v]) swap(u, v), swap(ans1, ans2);
            ans += query(1, dfn[v], dfn[u], dfn[v], dfn[u]);
            if (RC == ans1) ans--;
            if (LC == ans2) ans--;
        }
        return ans;
    };

    while (m--) {
        char op;
        int x, y, v;
        cin >> op;
        if (op == 'C') {
            cin >> x >> y >> v;
            calc(1, x, y, v);
        }else {
            cin >> x >> y;
            cout << calc(2, x, y, 0) << endl;
        }
    }
}

signed main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    //cin >> t;
    while(t--) solve();
    return 0;
}
```

线段树区间排序，整体查询

```
#include <bits/stdc++.h>
#define endl '\n'

using ll = long long;

constexpr int N = 2e5 + 10;
constexpr int M = 27;
constexpr int mod = 998244353;

using namespace std;

string s;
```

```
int n, q;

struct Info {
    int sum[M];
}empty_Info;

struct Node {
    int l, r;
    int tag[M];
    Info res;
}tr[N << 2];

void push_down(int u) {
    for (int i = 1; i <= 26; ++i) {
        if (tr[u].tag[i]) {
            for (int j = 1; j <= 26; ++j) {
                tr[u << 1].tag[j] = 0;
                tr[u << 1].res.sum[j] = 0;
                tr[u << 1 | 1].tag[j] = 0;
                tr[u << 1 | 1].res.sum[j] = 0;
            }
            tr[u << 1].tag[i] = 1;
            tr[u << 1].res.sum[i] = tr[u << 1].r - tr[u << 1].l + 1;
            tr[u << 1 | 1].tag[i] = 1;
            tr[u << 1 | 1].res.sum[i] = tr[u << 1 | 1].r - tr[u << 1 | 1].l + 1;
            tr[u].tag[i] = 0;
            break;
        }
    }
}

Info merge(Info ls, Info rs) {
    Info res = empty_Info;
    for (int i = 1; i <= 26; ++i) {
        res.sum[i] = ls.sum[i] + rs.sum[i];
    }
    return res;
}

void push_up(int u) {
    tr[u].res = merge(tr[u << 1].res, tr[u << 1 | 1].res);
}

void build(int u, int l, int r) {
    tr[u].l = l, tr[u].r = r;
    if (l == r) {
        tr[u].res.sum[s[l] - 'a' + 1] = 1;
        return;
    }
    int mid = (l + r) >> 1;
    build(u << 1, l, mid);
    build(u << 1 | 1, mid + 1, r);
    push_up(u);
}


Info query(int u, int ql, int qr) {
    if (ql > qr) {
        return empty_Info;
    }
    if (tr[u].l == ql && tr[u].r == qr) {
        return tr[u].res;
    }
    push_down(u);
    int mid = (tr[u].l + tr[u].r) >> 1;
    if (qr <= mid) return query(u << 1, ql, qr);
    else if (ql > mid) return query(u << 1 | 1, ql, qr);
    return merge(query(u << 1, ql, mid), query(u << 1 | 1, mid + 1, qr));
}

void update(int u, int ql, int qr, int val) {
```

```
    if (ql > qr) return;
    if (tr[u].l == ql && tr[u].r == qr) {
        for (int i = 1; i <= 26; ++i) {
            tr[u].res.sum[i] = 0;
            tr[u].tag[i] = 0;
        }
        tr[u].tag[val] = 1;
        tr[u].res.sum[val] = tr[u].r - tr[u].l + 1;
        return;
    }
    push_down(u);
    int mid = (tr[u].l + tr[u].r) >> 1;
    if (qr <= mid) update(u << 1, ql, qr, val);
    else if (ql > mid) update(u << 1 | 1, ql, qr, val);
    else {
        update(u << 1, ql, mid, val);
        update(u << 1 | 1, mid + 1, qr, val);
    }
    push_up(u);
}

void push_all(int u) {
    if (tr[u].l == tr[u].r) return;
    push_down(u);
    push_all(u << 1);
    push_all(u << 1 | 1);
}

void print(int u) {
    if (tr[u].l == tr[u].r) {
        for (int i = 1; i <= 26; ++i) {
            if (tr[u].res.sum[i]) {
                cout << char('a' + i - 1);
            }
        }
        return;
    }
    print(u << 1);
    print(u << 1 | 1);
}

void solve(){
    cin >> n >> q;
    cin >> s; s = " " + s;
    build(1, 1, n);
    while (q--) {
        int op, l, r;
        cin >> l >> r >> op;
        Info tmp = query(1, l, r);

        // for (int i = 1; i <= 26; ++i) {
        //  cerr << tmp.sum[i] << " \n"[i == 26];
        // }
        if (op == 1) {
            for (int i = 1; i <= 26; ++i) {
                update(1, l, l + tmp.sum[i] - 1, i);
                l += tmp.sum[i];
            }
        }else {
            for (int i = 26; i >= 1; --i) {
                update(1, l, l + tmp.sum[i] - 1, i);
                l += tmp.sum[i];
            }
        }
    }
    push_all(1);
    print(1);
    cout << endl;
}

signed main(){
```

```
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    //cin >> t;
    while(t--) solve();
    return 0;
}
```

线段树区间最小值数量

```
#include <bits/stdc++.h>
#define endl '\n'

using ll = long long;

constexpr int N = 2e5 + 10;
constexpr int mod = 998244353;
constexpr int inf = 1e9;

using namespace std;

struct Node{
    ll sum; int len;
    Node() : sum(0), len(1) {}
    Node(ll sum, int len) : sum(sum), len(len) {a}
};

using Tag = ll;

Node operator + (const Node &a, const Node &b){

    if(a.sum < b.sum) return a;
    if(a.sum > b.sum) return b;
    Node c;
    c.sum = a.sum, c.len = a.len + b.len;
    return c;
}

void apply(Node &x, Tag &a, Tag f){
    x.sum += f;
    a += f;
}

template<class Node, class Tag, class Merge = plus<Node>>
struct Tree{
    int n;
    Merge merge;
    vector<Node> info;
    vector<Tag> tag;
    Tree(int n = 0) : n(n), merge(Merge()), info((n << 2) + 1), tag((n << 2) + 1){}
    Tree(vector<Node> init) : Tree((int)init.size()){
        function<void(int, int, int)> build = [&](int p, int l, int r){
            if (l == r){
                info[p] = init[l - 1];
                return;
            }
            int m = (l + r) / 2;
            build(2 * p, l, m);
            build(2 * p + 1, m + 1, r);
            push_up(p);
        };
        build(1, 1, n);
    }
    void push_up(int p){
        info[p] = merge(info[2 * p], info[2 * p + 1]);
    }
    void apply(int p, const Tag &v){
        ::apply(info[p], tag[p], v);
    }
    void push_down(int p){
        apply(2 * p, tag[p]);
```

```
        apply(2 * p + 1, tag[p]);
        tag[p] = Tag();
    }
    void modify(int p, int l, int r, int x, const Node &v){
        if (l == r){
            info[p] = v;
            return;
        }
        int m = (l + r) / 2;
        push_down(p);
        if (x <= m){
            modify(2 * p, l, m, x, v);
        }
        else{
            modify(2 * p + 1, m + 1, r, x, v);
        }
        push_up(p);
    }
    void modify(int p, const Node &v){
        modify(1, 1, n, p, v);
    }
    Node query(int p, int l, int r, int x, int y){
        if (l > y || r < x){
            return Node(inf,1);
        }
        if (l >= x && r <= y){
            return info[p];
        }
        int m = (l + r) / 2;
        push_down(p);
        return merge(query(2 * p, l, m, x, y), query(2 * p + 1, m + 1, r, x, y));
    }
    Node query(int l, int r){
        return query(1, 1, n, l, r);
    }
    void modify(int p, int l, int r, int x, int y, const Tag &v){
        if (l > y || r < x){
            return;
        }
        if (l >= x && r <= y){
            apply(p, v);
            return;
        }
        int m = (l + r) / 2;
        push_down(p);
        modify(2 * p, l, m, x, y, v);
        modify(2 * p + 1, m + 1, r, x, y, v);
        push_up(p);
    }
    void modify(int l, int r, const Tag &v){
        return modify(1, 1, n, l, r, v);
    }
};
using T = Tree<Node, Tag>;

void solve(){
    int n, k;
    cin >> n >> k;
    vector<ll> a(n + 1), prek(n + 1), lst(n + 1);
    map<ll, vector<int>> pos;
    for (int i = 1; i <= n; ++i){
        cin >> a[i];
        if (pos[a[i]].size()) lst[i] = pos[a[i]].back();
        pos[a[i]].push_back(i);
    }
    for (auto [x, v] : pos){
        for (int i = k - 1; i < v.size(); ++i){
            prek[v[i]] = v[i - k + 1];
        }
    }

```

```
    ll ans = 0;
    map<ll, int> mp;
    vector<Node> init(n);
    T tr(init);

    for (int i = 1; i <= n; ++i){
        if (mp[a[i]]){
            if (prek[mp[a[i]]] < mp[a[i]]){
                tr.modify(prek[mp[a[i]]] + 1, mp[a[i]], -1);
            }
            if (lst[prek[mp[a[i]]]]){
                tr.modify(1, lst[prek[mp[a[i]]]], -1);
            }
        }

        if (prek[i] < i){
            tr.modify(prek[i] + 1, i, 1);
        }
        if (lst[prek[i]]){
            tr.modify(1, lst[prek[i]], 1);
        }
        auto qry = tr.query(1, i);
        if (!qry.sum) ans += qry.len;
        mp[a[i]] = i;
    }
    cout << ans << endl;
}

signed main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    cin >> t;
    while(t--) solve();
    return 0;
}
```

线段树区间最小 x 的 y 和

```
#include <bits/stdc++.h>
#define endl '\n'

using ll = long long;

constexpr int N = 2e5 + 10;
constexpr int mod = 998244353;

using namespace std;

struct Node {
    ll sumtag, sumval;
    ll tag;
};

struct LazySegmentTree {
    Node merge(Node a, Node b) {
        a.tag = b.tag = 0;
        if (a.sumtag == b.sumtag) a.sumval = (a.sumval + b.sumval) % mod;
        if (a.sumtag < b.sumtag) swap(a, b);
        return a;
    }

    vector<Node> Info;

    void init(int n = 2e5 + 10) {
        Info.resize(4 * n);
    }

    void push_down(int u){
        if (Info[u].tag) {
            Info[u << 1].tag += Info[u].tag;
```

```
            Info[u << 1].sumtag += Info[u].tag;
            Info[u << 1 | 1].tag += Info[u].tag;
            Info[u << 1 | 1].sumtag += Info[u].tag;
            Info[u].tag = 0;
        }
    }

    void push_up(int u) {
        Info[u] = merge(Info[u << 1], Info[u << 1 | 1]);
    }

    void modify(int u, int l, int r, int ql, int qr, ll val){
        if (ql <= l && r <= qr) {
            Info[u].tag += val;
            Info[u].sumtag += val;
            return;
        }
        push_down(u);
        int mid = (l + r) >> 1;
        if (ql <= mid) modify(u << 1, l, mid, ql, qr, val);
        if (qr > mid) modify(u << 1 | 1, mid + 1, r, ql, qr, val);
        push_up(u);
    }

    void updateDp(int u, int l, int r, int pos, ll val){
        if (l == r) {
            Info[u].sumval = val;
            return;
        }
        push_down(u);
        int mid = (l + r) >> 1;
        if (pos <= mid) updateDp(u << 1, l, mid, pos, val);
        else updateDp(u << 1 | 1, mid + 1, r, pos, val);
        push_up(u);
    }
};

void solve(){
    int n, m;
    cin >> n >> m;
    LazySegmentTree Tr;
    Tr.init();
    vector<int> a(n + 1);
    vector<vector<int>> pos(n + 1);
    for (int i = 1; i <= n; ++i) {
        cin >> a[i];
        pos[i].push_back(0);
    }
    vector<int> p(m + 1);
    vector<ll> dp(n + 1);
    for (int i = 1; i <= m; ++i) cin >> p[i];
    Tr.updateDp(1, 0, n, 0, 1);
    for (int i = 1; i <= n; ++i) {
        pos[a[i]].push_back(i);
        int u = pos[a[i]].size() - 1;
        for (int j = 1; j <= m; ++j){
            if (u >= p[j]) Tr.modify(1, 0, n, pos[a[i]][u - p[j]], pos[a[i]][u - p[j] + 1] - 1, -1);
            if (u >= p[j] + 1) Tr.modify(1, 0, n, pos[a[i]][u - p[j] - 1], pos[a[i]][u - p[j]] - 1, 1);
        }
        dp[i] = Tr.Info[1].sumval;
        Tr.updateDp(1, 0, n, i, dp[i]);
    }
    cout << dp[n] << endl;
}

signed main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    //cin >> t;
    while(t--) solve();
```

```
    return 0;
}
```

线性 dp

最长单峰子序列

```
#include <bits/stdc++.h>
#define endl '\n'

using ll = long long;

constexpr int N = 2e5 + 10;
constexpr int mod = 998244353;

using namespace std;

struct SegmentTree
{
    ll tr[N << 2];

    void push_up(int u) {
        tr[u] = max(tr[u << 1], tr[u << 1 | 1]);
    }

    void modify(int u, int l, int r, int pos, ll val) {
        if (l == r) {
            tr[u] = val;
            return;
        }
        int mid = (l + r) >> 1;
        if (pos <= mid) modify(u << 1, l, mid, pos, val);
        else modify(u << 1 | 1, mid + 1, r, pos, val);
        push_up(u);
    }

    ll query_max(int u, int l, int r, int ql, int qr) {
        if (ql <= l && r <= qr) return tr[u];
        int mid = (l + r) >> 1;
        ll ans = 0;
        if (ql <= mid) {
            ll tmp = query_max(u << 1, l, mid, ql, qr);
            ans = max(ans, tmp);
        }
        if (qr > mid) {
            ll tmp = query_max(u << 1 | 1, mid + 1, r, ql, qr);
            ans = max(ans, tmp);
        }
        return ans;
    }
}T0, T1;

ll a[N];

void solve(){
    int n, m;
    cin >> n >> m;
    for (int i = 1; i <= n; ++i) {
        int l, r;
        cin >> l >> r;
        a[l]++;
        a[r + 1]--;
    }
    for (int i = 1; i <= m; ++i) a[i] += a[i - 1];
    for (int i = 1; i <= m; ++i) {
        ll v = T0.query_max(1, 0, n, 0, a[i]);
        T0.modify(1, 0, n, a[i], v + 1);
        T1.modify(1, 0, n, a[i], max(T1.query_max(1, 0, n, a[i], n), v) + 1);
    }
    cout << (T0.query_max(1, 0, n, 0, n), T1.query_max(1, 0, n, 0, n)) << endl;
}
```

```
65
66  signed main(){
67      ios::sync_with_stdio(false);
68      cin.tie(nullptr);
69      int t = 1;
70      //cin >> t;
71      while(t--) solve();
72      return 0;
73  }
```